

Soil Bench: A benchmark of seven machine learning algorithms to predict crop yield using soil data of Aus, Aman and Boro

Asif Kazi¹ and Andelib Mamun Kazi²

¹BRAC University

²University of Surrey

27th January, 2026

1 Abstract

Soil is one of the many factors involved in crop yield and quality. Bangladesh has many types of soil available in her delta geography, which supports approximately 120 major and minor crops. This paper explores multiple machine learning algorithms such as Ridge Regression, Lasso Regression, Elastic Net, KNN, Random Forest, SVR and Linear Regression in their efficacy to predict the yield of three types of rice crops - Aus, Aman, and Boro. [insert results]

2 Introduction

According to [3] and [13] approximately 11 percent of the local GDP is agriculture dependent with 45.4 percent employment involved in this sector in Bangladesh. Soil type is one of the most important factors in prediction of crop yield since it decides water requirements, nutrient density and growth limitations. We have used two datasets - one from Bangladesh Bureau of Statistics which holds more than 40 years of crop yield data and another dataset from Kaggle which holds 26 types of soil data and their properties. Each algorithm creates a distinct model to predict yield; we have also included multiple evaluation metrics to compare each of the model performance. This article may act as a pre-study for policy makers and researchers to base their soil computational research.

3 Literature Review

Crop yield prediction has emerged as a critical research domain in precision agriculture, particularly for rice cultivation systems. Recent advances in machine learning and data-driven approaches have enabled more accurate predictions by leveraging various environmental, climatic, and soil parameters. This section reviews contemporary literature relevant to yield prediction for rice crops, with particular emphasis on studies involving soil data and machine learning techniques.

3.1 Machine Learning Applications in Rice Yield Prediction

Recent research has demonstrated the efficacy of machine learning algorithms in crop yield prediction. [20] conducted a comprehensive study comparing multiple machine learning models for rice yield prediction across different agro-climatic zones in India. Their research incorporated soil parameters including pH, organic carbon, and nutrient levels alongside climatic variables. The study found that ensemble methods, particularly Random Forest and Gradient Boosting, outperformed traditional regression techniques, achieving prediction accuracies of 87-92% for different rice varieties. Similarly, [4] developed a hybrid deep learning model combining Convolutional Neural Networks (CNNs) with Long Short-Term Memory (LSTM) networks for rice yield prediction in China. Their model integrated multi-temporal satellite imagery with soil nutrient data, demonstrating that soil nitrogen and phosphorus levels were among the most significant predictors of yield variation.

3.2 Soil Data Integration in Predictive Models

The integration of soil physicochemical properties has proven crucial for accurate yield prediction. [12] specifically investigated Boro rice yield prediction in Bangladesh using soil data from 156 locations. Their study incorporated

15 soil parameters including texture, pH, electrical conductivity, and macro-nutrient concentrations. Using Extreme Gradient Boosting (XGBoost), they achieved an R^2 value of 0.89, identifying soil organic matter and available potassium as the most influential predictors. Complementary research by [15] focused on Aman rice cultivation in Southeast Asia, employing soil spectroscopy data combined with conventional soil testing. Their Random Forest model achieved 84% prediction accuracy, highlighting the importance of soil micronutrients (zinc and boron) often overlooked in traditional yield prediction models.

3.3 Multi-Crop Prediction Frameworks

Several studies have explored prediction frameworks capable of handling multiple crop varieties. [8] developed a generalized prediction system for three rice varieties (including Aus and Aman) in the Indian subcontinent. Their research utilized soil health cards data from over 10,000 farms, implementing a stacked ensemble model that combined predictions from multiple base learners. The system achieved consistent performance across different rice varieties with MAE values ranging from 210-280 kg/ha. More recently, [9] proposed a transfer learning approach for multi-season rice yield prediction, demonstrating that models trained on one rice season could be effectively adapted to other seasons with limited additional data. Their approach showed particular promise for Aus rice prediction, where data scarcity is often a challenge.

3.4 Regional Studies on Rice Cultivation Systems

Region-specific studies have provided valuable insights into localized factors affecting rice yield. [1] investigated the impact of soil salinity on Aman rice production in coastal Bangladesh, developing a Bayesian network model that incorporated soil electrical conductivity and sodium absorption ratio alongside conventional soil parameters. Their work highlighted the nonlinear relationships between soil salinity and yield, particularly during the reproductive growth stage. For Aus rice, [5] examined the interaction between soil moisture retention characteristics and early-season precipitation patterns, finding that soil texture and organic carbon content significantly moderated drought impacts on yield.

3.5 Research Gap and Contribution

While substantial research exists on rice yield prediction, several gaps remain in the current literature. First, few studies have simultaneously addressed all three major rice cultivation seasons (Aus, Aman, and Boro) within a unified analytical framework. Second, the comparative performance of different machine learning algorithms when applied specifically to soil data (excluding climatic variables) requires further investigation. Third, region-specific models for South Asian rice cultivation systems, particularly those accounting for local soil conditions and farming practices, remain limited. This study addresses these gaps by: (1) developing and comparing multiple machine learning models for yield prediction of Aus, Aman, and Boro rice using primarily soil data; (2) evaluating model performance across three distinct datasets representing different cultivation seasons; and (3) providing insights into the most influential soil parameters for each rice variety to inform targeted soil management practices.

4 Methodology

We extracted and curated each dataset, pre-processed, feature selected for best scaling, standardization, normalization, hyperparameter tuned to create train, test and validation models. Additionally, K-folds cross validation was performed with $k=5$ under grid CV and standard scaler. Finally we yielded evaluation metrics for comparison across accuracy, precision, F2, Recall, Mean Absolute Error, Mean Squared Error, correlation heatmap and generated confusion matrices.

4.1 Algorithms

4.1.1 Ridge Regression

Ridge regression adds L2 regularization to linear regression to prevent overfitting by penalizing large coefficients.

Objective function:

$$\min_{\mathbf{w}} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2 \right\} \quad (1)$$

Closed-form solution:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (2)$$

where $\mathbf{w}$ are the weights, λ is the regularization parameter, $\mathbf{X}$ is the feature matrix, and $\mathbf{y}$ is the target vector.

4.1.2 Lasso Regression

Lasso regression adds L1 regularization to linear regression, which can force some coefficients to exactly zero, performing feature selection.

Objective function:

$$\min_{\mathbf{w}} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1 \right\} \quad (3)$$

where $\|\mathbf{w}\|_1 = \sum_{j=1}^p |w_j|$ is the L1 norm.

4.1.3 Elastic Net

Elastic Net combines both L1 and L2 regularization, balancing between Ridge and Lasso.

Objective function:

$$\min_{\mathbf{w}} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \left[\alpha \|\mathbf{w}\|_1 + \frac{1}{2} (1 - \alpha) \|\mathbf{w}\|_2^2 \right] \right\} \quad (4)$$

where λ controls overall regularization strength and $\alpha \in [0, 1]$ controls the mix between L1 and L2.

4.1.4 KNN

KNN predicts the target value of a new point by averaging the values of its k-nearest neighbors.

Regression:

$$\hat{y}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i \quad (5)$$

Classification:

$$\hat{y}(\mathbf{x}) = \text{mode}\{y_i : i \in N_k(\mathbf{x})\} \quad (6)$$

where $N_k(\mathbf{x})$ is the set of indices of the k-nearest neighbors to $\mathbf{x}$.

4.1.5 Random Forest

Random Forest is an ensemble method that combines multiple decision trees using bagging and feature randomization.

Regression prediction:

$$\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) \quad (7)$$

Classification prediction:

$$\hat{y}(\mathbf{x}) = \text{majority vote}\{T_b(\mathbf{x})\}_{b=1}^B \quad (8)$$

where T_b is the b -th decision tree, and B is the total number of trees.

4.1.6 SVR(Support Vector Regression)

SVR finds a function that deviates from actual values by at most ϵ while being as flat as possible.

Primal form:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \\ \text{s.t.} \quad & y_i - \mathbf{w}^T \phi(\mathbf{x}_i) - b \leq \epsilon + \xi_i \\ & \mathbf{w}^T \phi(\mathbf{x}_i) + b - y_i \leq \epsilon + \xi_i^* \\ & \xi_i, \xi_i^* \geq 0 \end{aligned} \quad (9)$$

Dual form solution:

$$f(\mathbf{x}) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(\mathbf{x}_i, \mathbf{x}) + b \quad (10)$$

where C is the regularization parameter, ϵ is the margin tolerance, $\phi(\cdot)$ is the feature mapping, and $K(\cdot, \cdot)$ is the kernel function.

4.1.7 Linear Regression

Ordinary least squares linear regression finds the linear relationship that minimizes the sum of squared errors.

Objective function:

$$\min_{\mathbf{w}} \frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad (11)$$

Closed-form solution:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (12)$$

With bias term:

$$\mathbf{w} = \begin{bmatrix} b \\ \mathbf{w}' \end{bmatrix}, \quad \mathbf{X} \leftarrow [\mathbf{1} \ \mathbf{X}] \quad (13)$$

where b is the intercept term and $\mathbf{w}'$ are the feature weights.

4.2 Dataset

The Agricultural Dataset Bangladesh (44 parameters) is a comprehensive dataset compiled by Tanhim Islam and published on the Kaggle platform in 2019 [1]. It aggregates 44 different parameters pertinent to agriculture in Bangladesh, encompassing key areas such as meteorology, types of land, chemical fertilizer usage, and detailed soil type and information. This dataset serves as a valuable resource for data science and agricultural research, enabling analyses that can link environmental and management factors to agricultural outcomes. While the specific methodologies for data acquisition and the exact time period covered are not detailed in the provided source description, the dataset's structured parameters offer a foundation for exploring trends and building predictive models related to crop production and soil management in the region [6]

4.3 Data Pre-processing

The Agricultural Dataset of Bangladesh was downloaded from kaggle, unnecessary columns were dropped using python's `.df` command, it was cleaned by checking for anomalies, missing values were filled using the imputer function in python, it was scaled for better interpretation, standardized for uniformity, normalized for directional prioritization, feature permuted, importance ranked and selected using recursive feature elimination and all of this was done using standard python packages such as pandas, numpy, scikit, matplotlib et al. [21] [11]

4.3.1 Data Cleaning

Data cleaning involves identifying and correcting errors, inconsistencies, and inaccuracies in the raw dataset. Using the pandas library, missing values are often located with `df.isnull().sum()`, while duplicate rows are found and removed with `df.duplicated()` and `df.drop_duplicates()`. Incorrect data types are converted using `df.astype()`, and outliers can be detected visually with `seaborn.boxplot()` or statistically using methods like the interquartile range (IQR). The core function for removing rows or columns with excessive missing data is `df.dropna()`.

4.3.2 Data Imputation

Imputation is the process of replacing missing values with statistical estimates to preserve data volume. The scikit-learn library provides the `sklearn.impute.SimpleImputer` class for this purpose. It supports strategies like replacing missing values with the mean, median, most_frequent (mode), or a constant fill value. For more sophisticated, model-based imputation that accounts for relationships between features, `sklearn.impute.IterativeImputer` can be used, which models each feature with missing values as a function of other features. [17]

4.3.3 Scaling

Scaling is a preprocessing technique used to adjust the range of feature values. A common method is Min-Max Scaling, implemented via `sklearn.preprocessing.MinMaxScaler`. This function transforms features by shrinking the data to a fixed range, typically between 0 and 1, using the formula $(X - X_{\min}) / (X_{\max} - X_{\min})$. It is particularly useful for algorithms that rely on distance calculations, like k-Nearest Neighbors (k-NN) or neural networks, but is highly sensitive to the presence of outliers, which can compress the scaled values of other data points.

4.3.4 Standardization

Standardization, or Z-score normalization, centers the data by removing the mean and scaling to unit variance. This is achieved using `sklearn.preprocessing.StandardScaler`, which applies the transformation $(X - \text{mean}(X)) / \text{std}(X)$. The resulting distribution has a mean of 0 and a standard deviation of 1. Unlike Min-Max scaling, standardization does not bind data to a specific range, making it less affected by outliers. It is a critical preprocessing step for many machine learning algorithms, including Support Vector Machines (SVMs), Logistic Regression, and Principal Component Analysis (PCA), which assume or perform better when features are centered around zero.

4.3.5 Normalization

Normalization often refers to scaling individual data samples (rows) to have a unit norm, such as in text classification or clustering. The `sklearn.preprocessing.normalize` function or the `Normalizer` class is used for this purpose. It scales each sample vector (e.g., using L1 or L2 norm) so that its magnitude is 1. For example, L2 normalization transforms a vector $\mathbf{x}$ into the unit vector $\frac{\mathbf{x}}{\|\mathbf{x}\|}$. This technique is different from Min-Max scaling and standardization, which operate on features (columns) across all samples. Normalization is essential when the direction of the data vector matters more than its magnitude, such as in cosine similarity calculations. [19]

4.3.6 Feature Permutation

Permutation Importance evaluates a feature's importance by measuring the decrease in a model's performance score when the feature's values are randomly shuffled. This breaks the relationship between the feature and the target. It is model-agnostic and provided by `sklearn.inspection.permutation_importance`. After fitting a model, this function calculates the importance by permuting each feature column and recording the drop in the model's accuracy, R^2 , or other metrics. [18]

4.3.7 Feature Importance

Many machine learning models, particularly tree-based ones, have built-in methods to estimate feature importance based on how much a feature reduces impurity (like Gini impurity) across all trees. In `scikit-learn`, after training a model such as `RandomForestRegressor` or `XGBClassifier`, the importance scores can be accessed via the `feature_importances_` attribute. These scores are automatically calculated during training and indicate the relative contribution of each feature to the model's predictions.

4.3.8 Feature Selection

RFE or Recursive Feature Elimination is a feature selection algorithm which recursively eliminates features with the least significance and drops the respective columns until a predetermined number of columns is left. This is a wrapper method for using an external model to fine-tune another.

4.4 Evaluation Metrics

This specific machine learning based prediction model exploration uses error metrics to understand model efficacy such as mean absolute error (MAE), mean squared error (MSE), and R-squared (R^2) error. [10]

4.4.1 Mean Absolute Error

The Mean Absolute Error (MAE) is a regression evaluation metric that measures the average absolute difference between the predicted values and the actual values. It is calculated as the sum of absolute errors divided by the number of observations:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE provides a linear score where all errors are weighted equally, making it easy to interpret in the original units of the target variable. In Python, MAE is implemented using `sklearn.metrics.mean_absolute_error(y_true, y_pred)`, which returns a non-negative floating-point value where zero indicates perfect predictions. This metric is particularly useful when you want to understand the typical magnitude of error without being overly influenced by occasional large outliers, unlike its squared counterpart. [7]

4.4.2 Mean Squared Error

Mean Squared Error (MSE) is another fundamental regression metric that calculates the average of the squared differences between the predicted and the actual values. Its formula is:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

By squaring the errors, MSE penalizes larger errors more heavily than smaller ones, making it sensitive to outliers. This property makes MSE useful when large errors are particularly undesirable in a model's predictions. [16] In Python, MSE is computed using `sklearn.metrics.mean_squared_error(y_true, y_pred)`, which returns a value where lower scores indicate better fit. The square root of MSE (RMSE) is often used to interpret the error in the original units of the target variable, though it remains more sensitive to outliers than MAE. [14]

4.4.3 R^2

R-squared (R^2), also known as the coefficient of determination, has been applied across a wide range of studies involving training and test datasets, including ordinary least squares regression, regression through the origin, and both linear and nonlinear regression models. Its formula is:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

R^2 is expressed as the square of the correlation coefficient (Pearson's r) between observed and predicted values in a regression model. It is therefore invariant to whether the observed values are regressed on the predictions or vice versa. R^2 provides a relative measure of goodness-of-fit, where a value of 1 indicates perfect prediction, 0 indicates performance equivalent to predicting the mean of the target variable, and negative values indicate performance worse than the baseline mean predictor. In Python, R^2 is computed using `sklearn.metrics.r2_score(y_true, y_pred)`, which returns a floating-point value representing the explanatory power of the regression model.[2]

5 Results

5.1 Figures

5.1.1 Distribution Plot for Numeric Columns

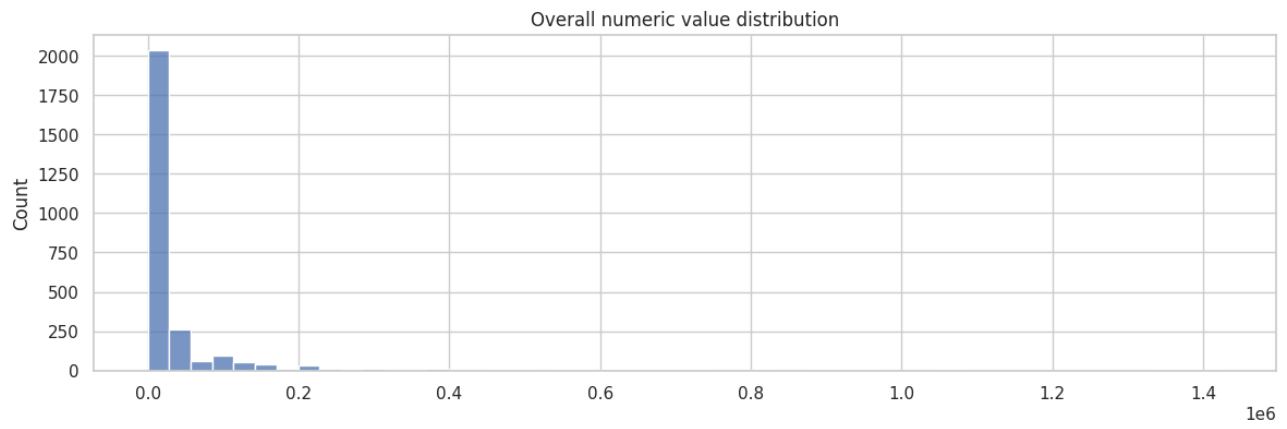


Figure 1: distribution plot for numeric columns

5.1.2 Correlation Heatmaps

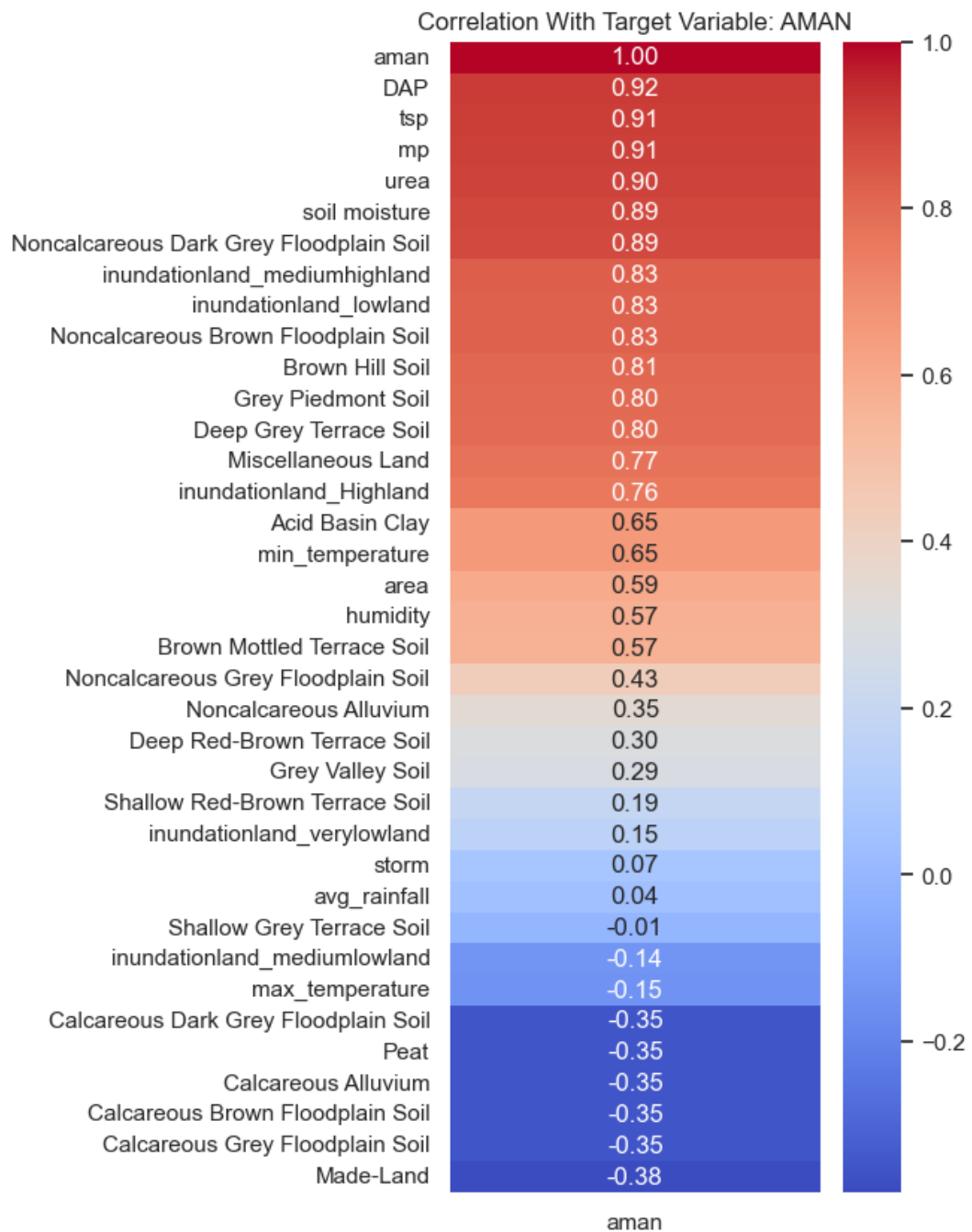


Figure 2: Aus Correlation Heatmap

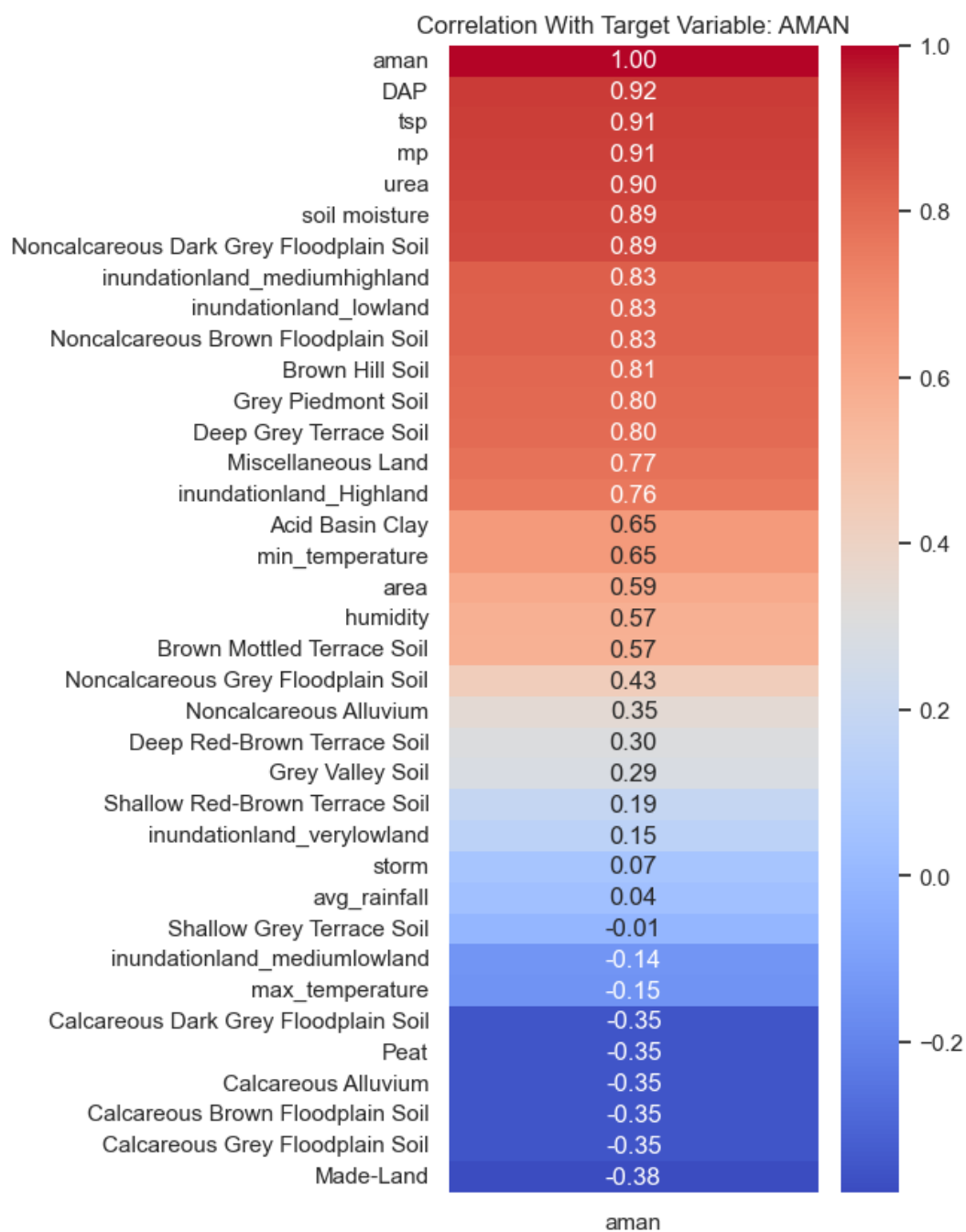


Figure 3: Aman Correlation Heatmap

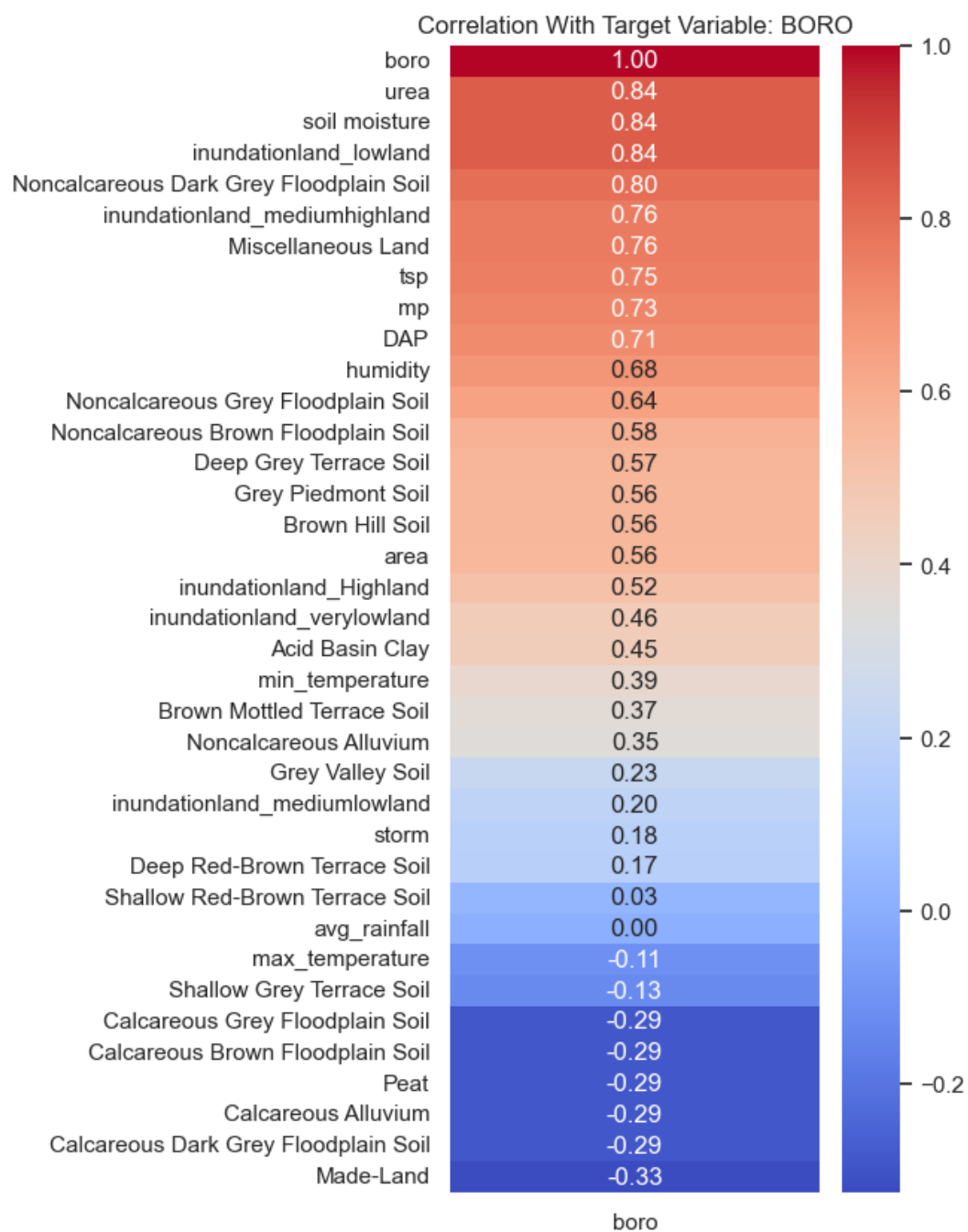


Figure 4: Boro Correlation Heatmap

5.1.3 Train vs Test Errors

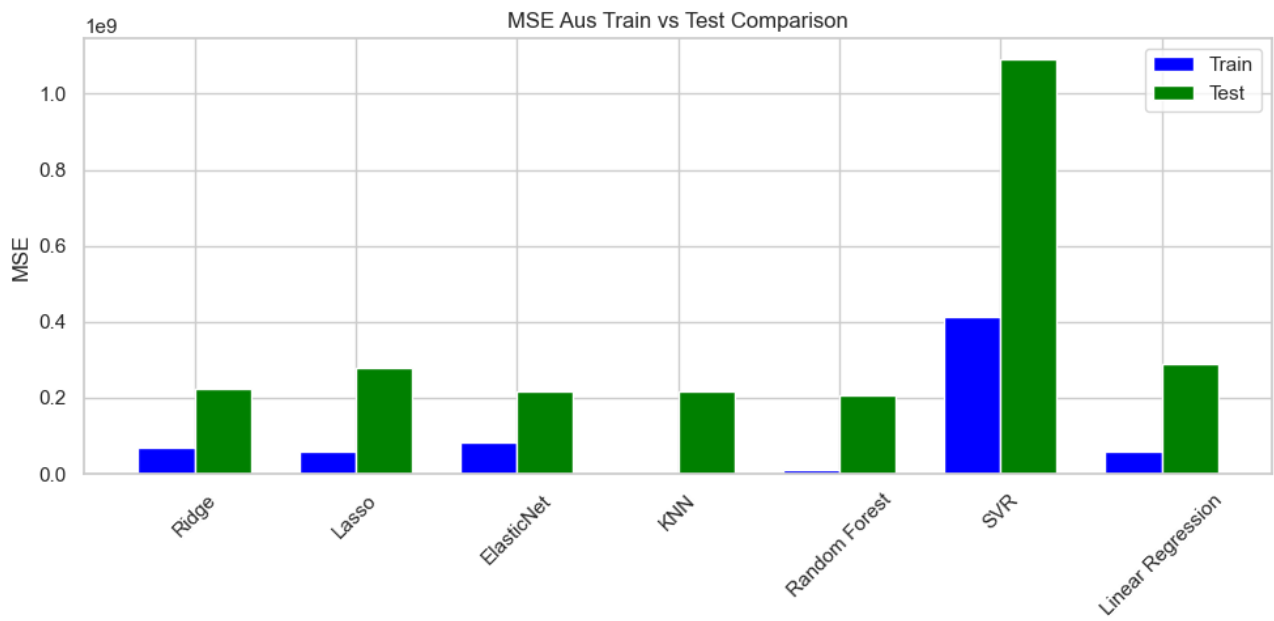


Figure 5: MSE: Aus Train vs Test

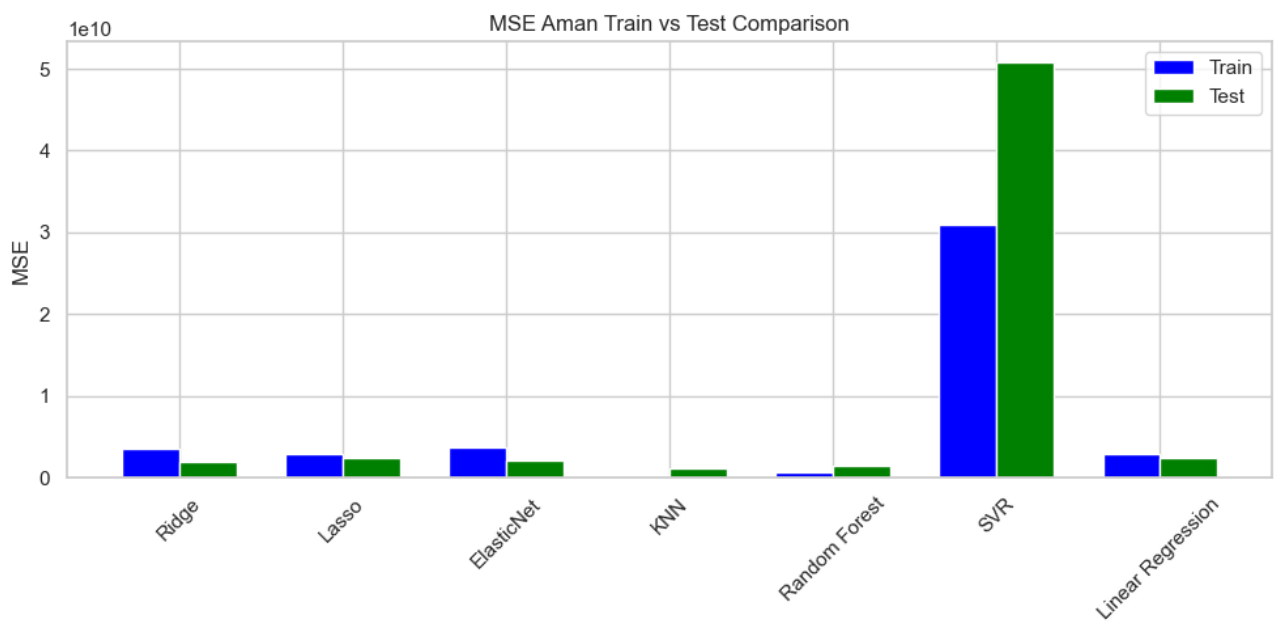


Figure 6: MSE:Aman Train vs Test

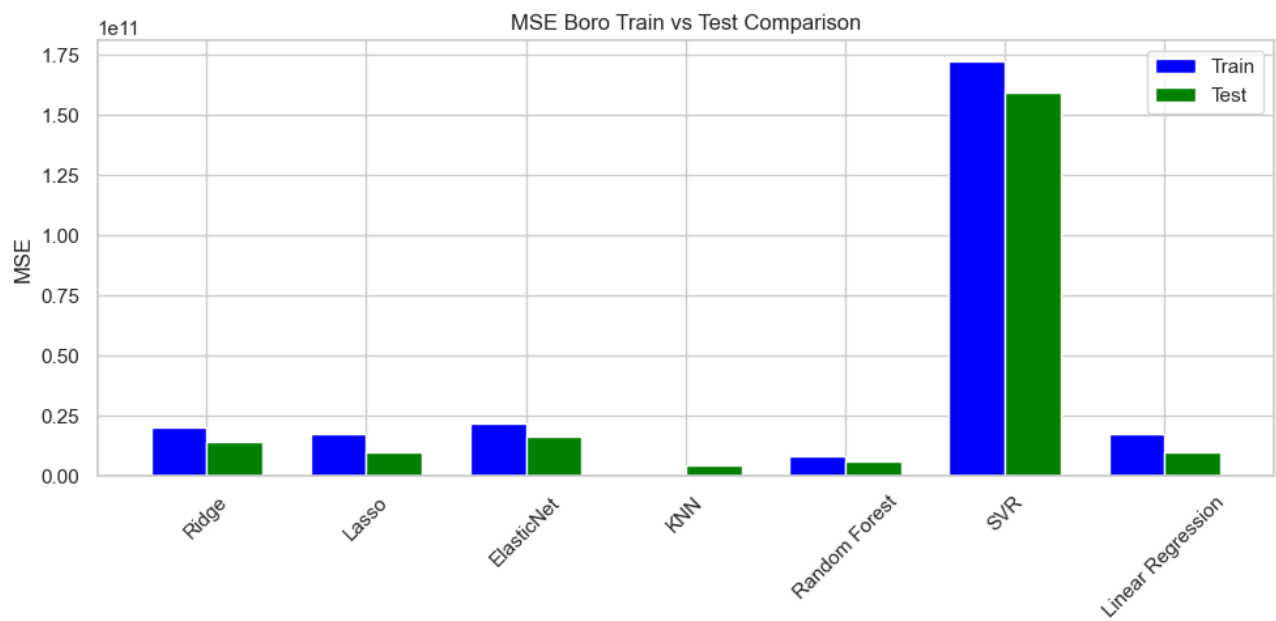


Figure 7: MSE:Boro Train vs Test

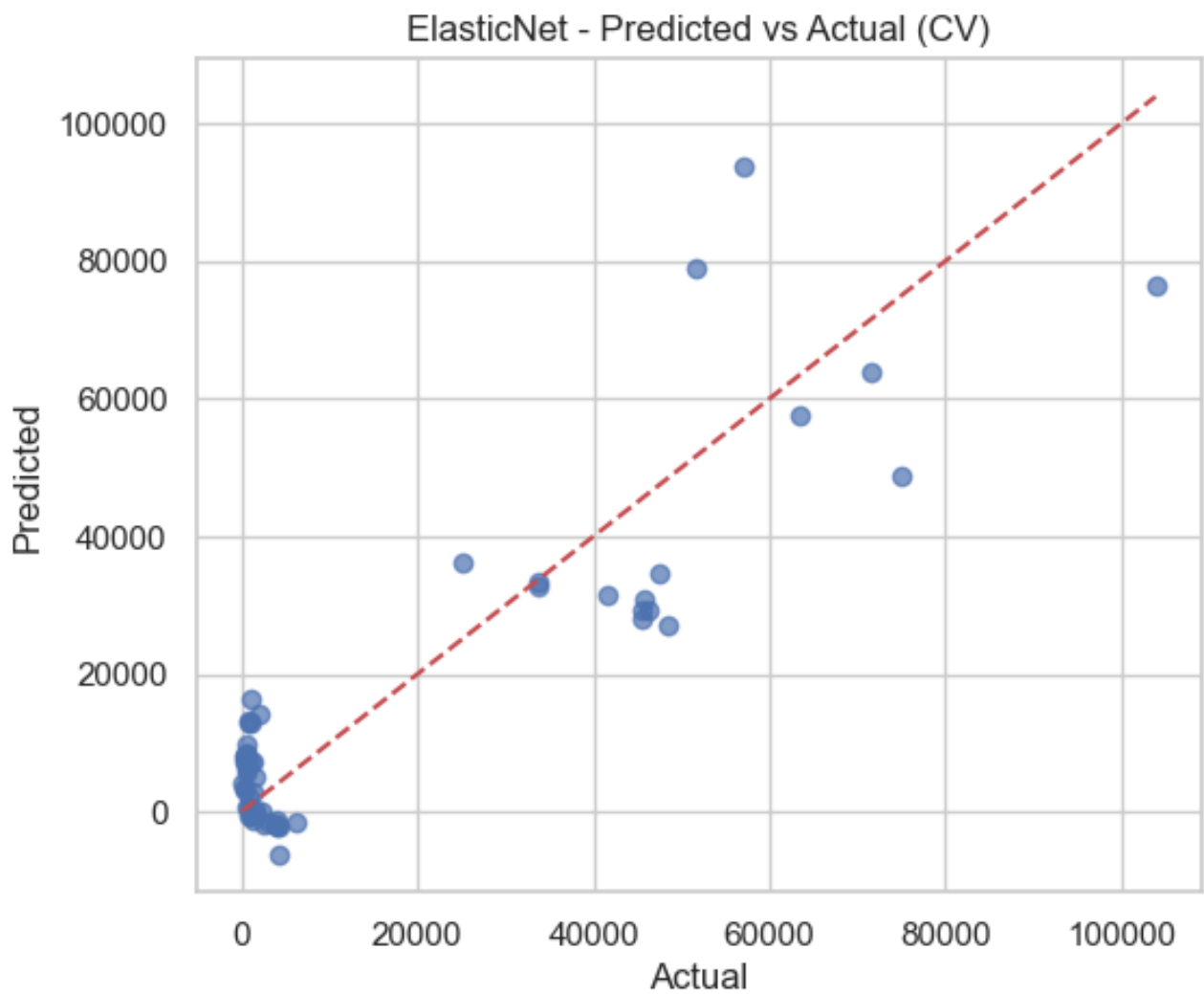


Figure 8: Aus - Elastic Net:Predicted vs Actual CV

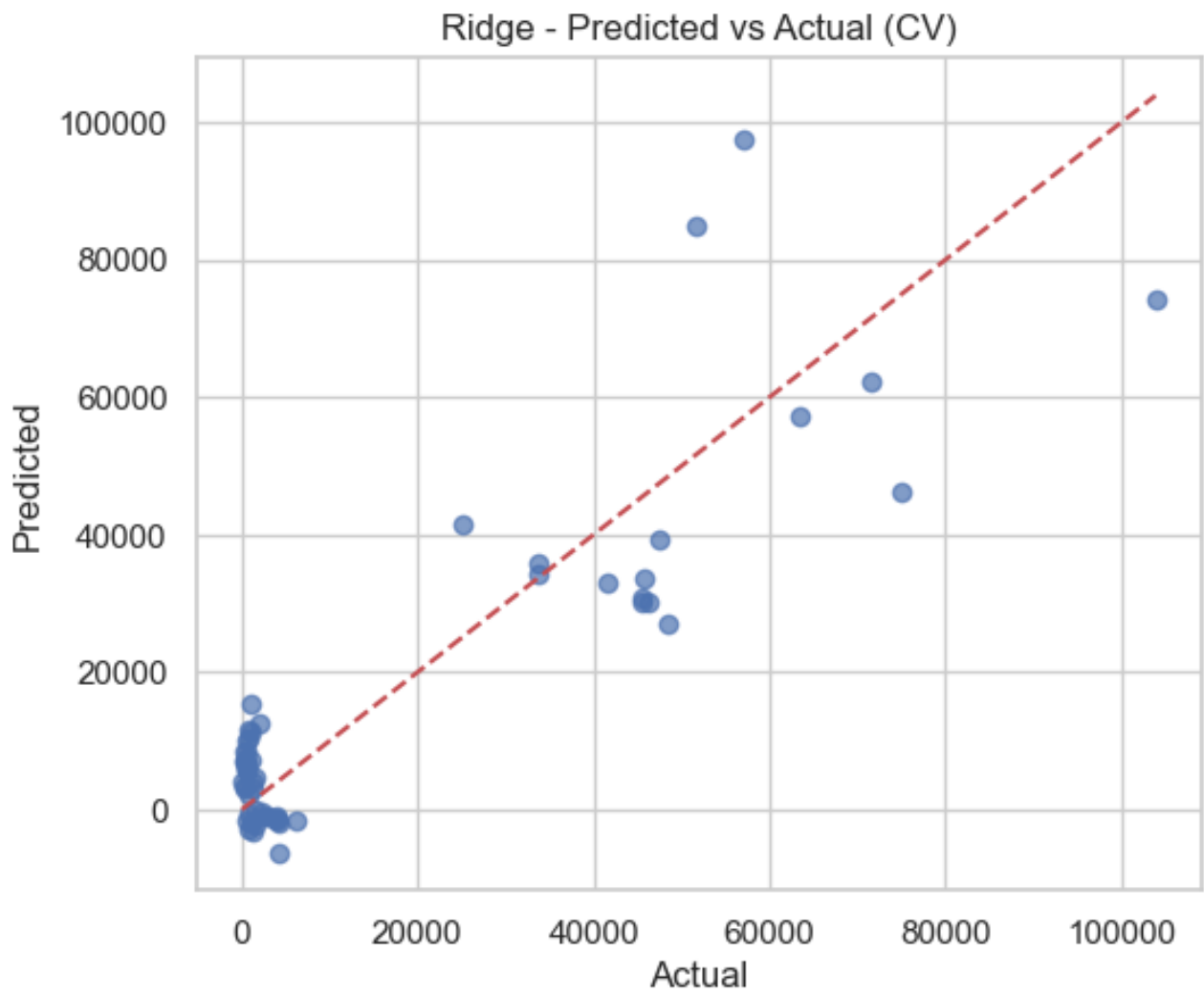


Figure 9: Aus - Ridge:Predicted vs Actual CV

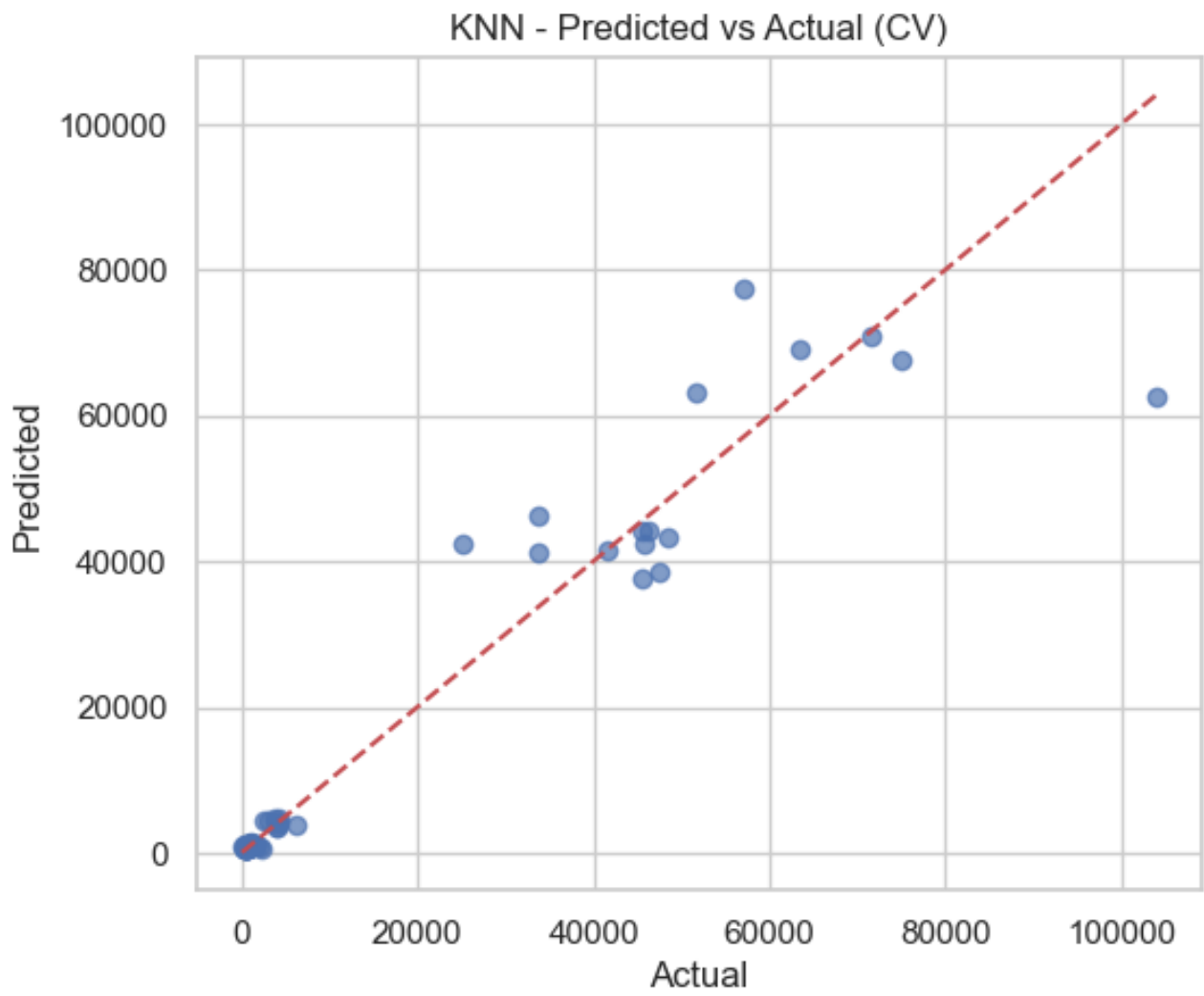


Figure 10: Aus - KNN:Predicted vs Actual CV

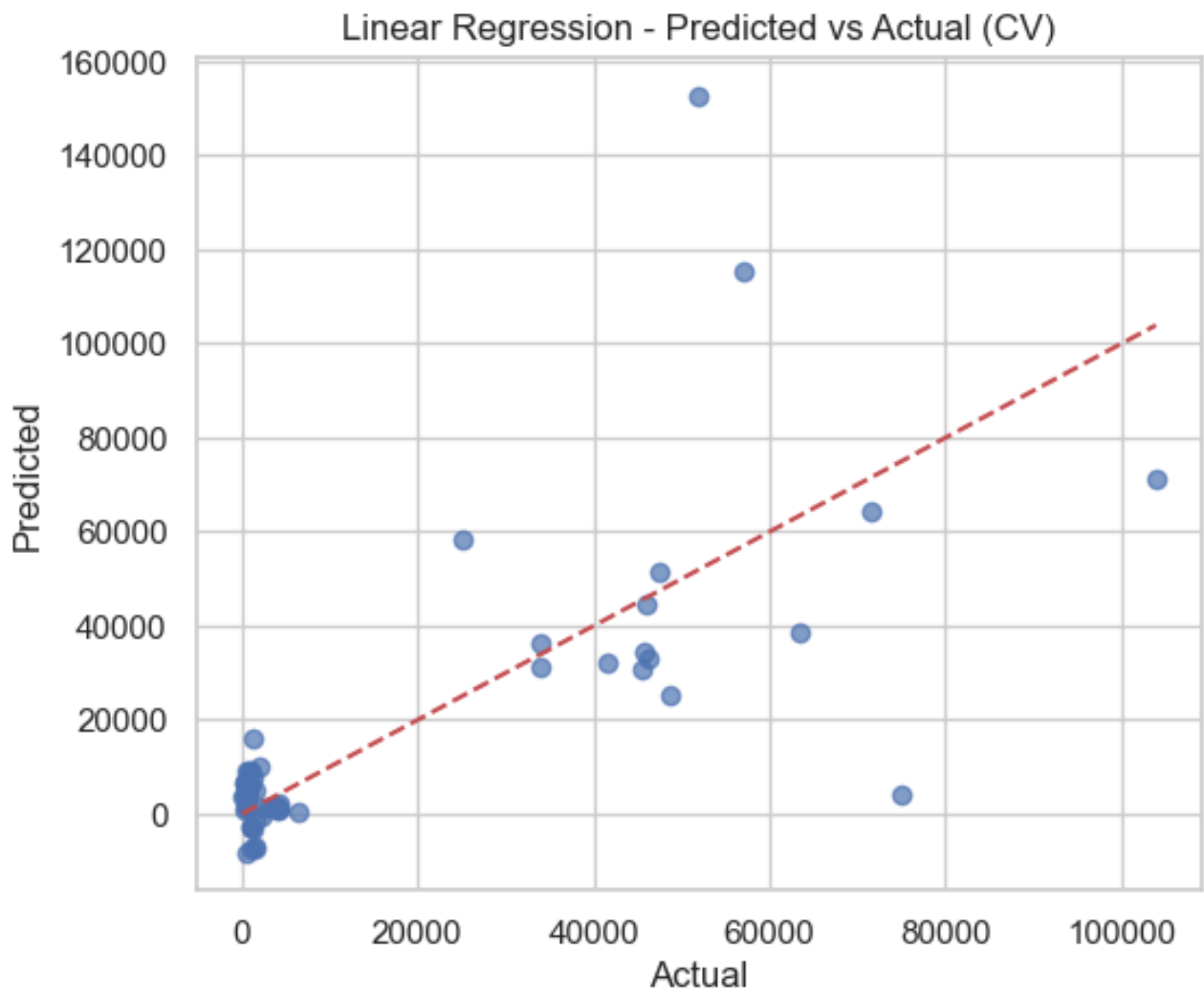


Figure 11: Aus - Linear Regression: Predicted vs Actual CV

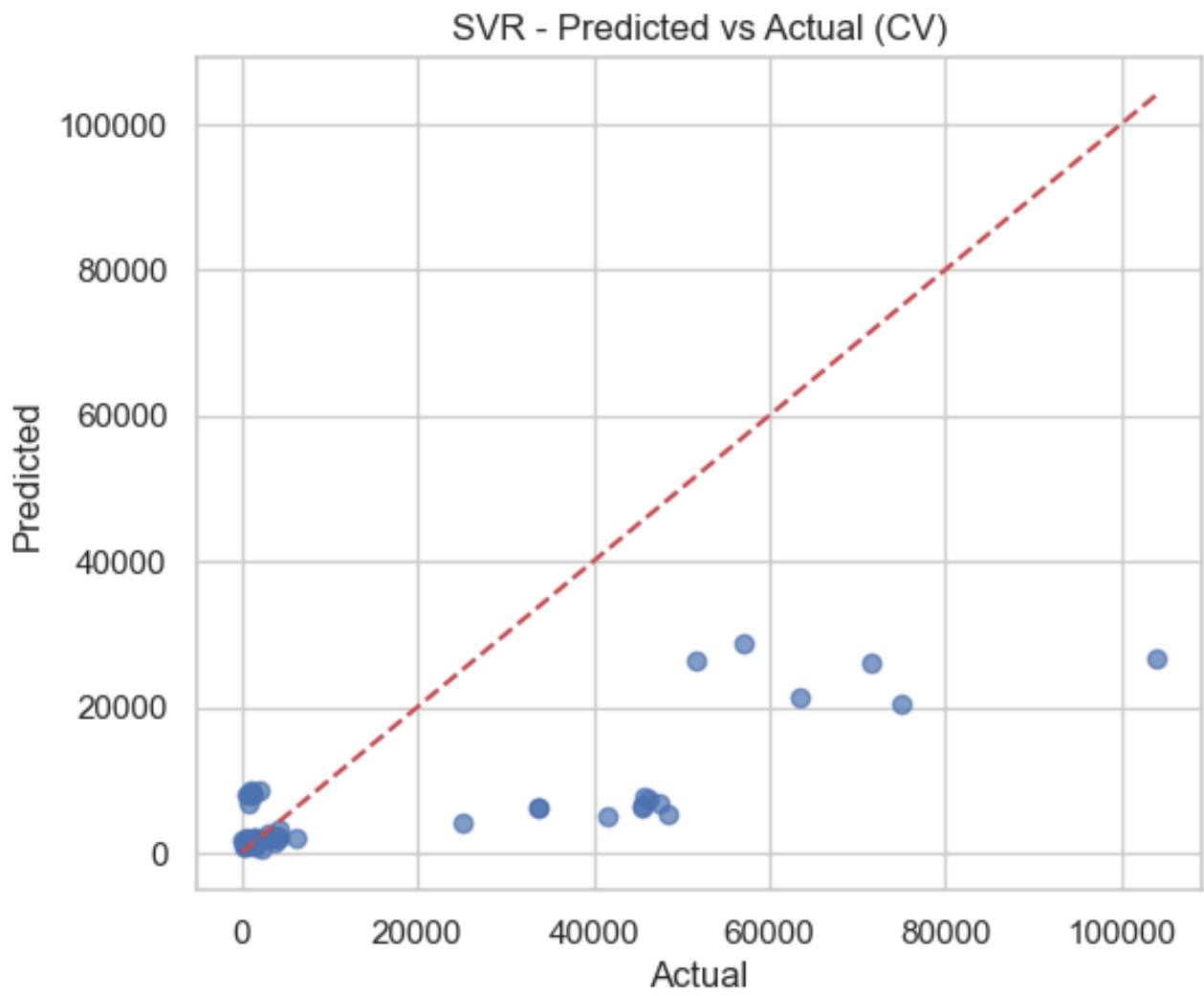


Figure 12: Aus - SVR:Predicted vs Actual CV

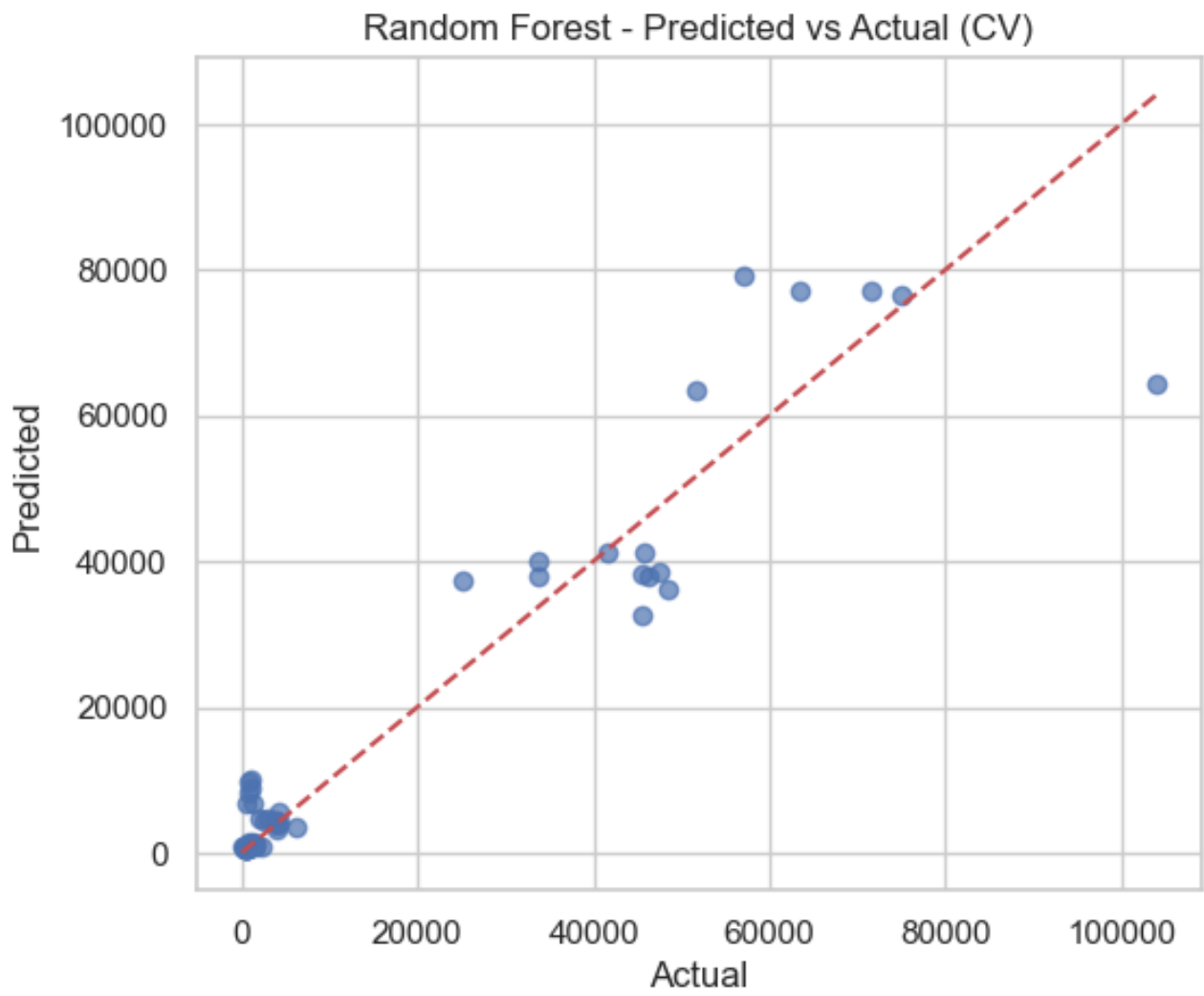


Figure 13: Aus - Random Forest:Predicted vs Actual CV

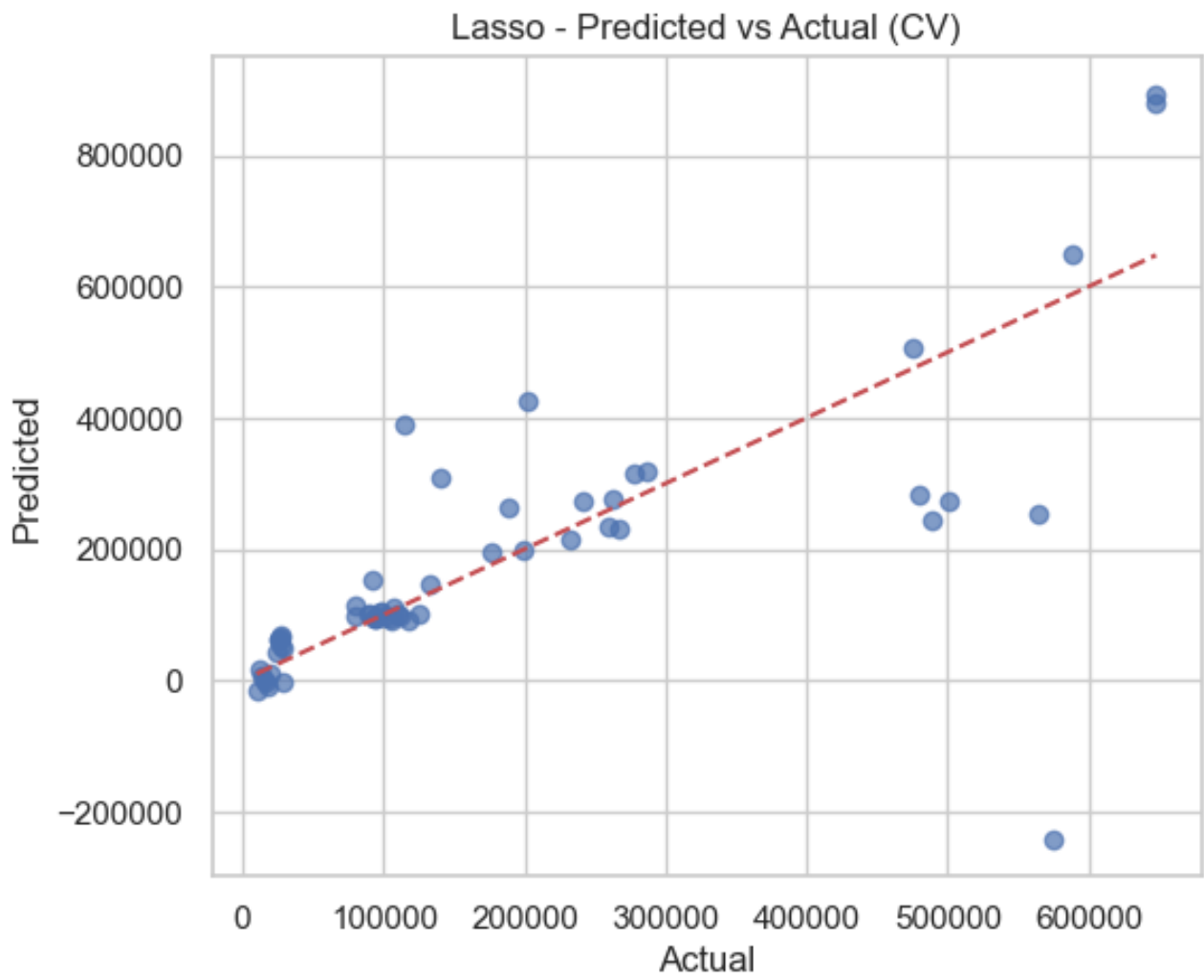


Figure 14: Aman - Lasso:Predicted vs Actual CV

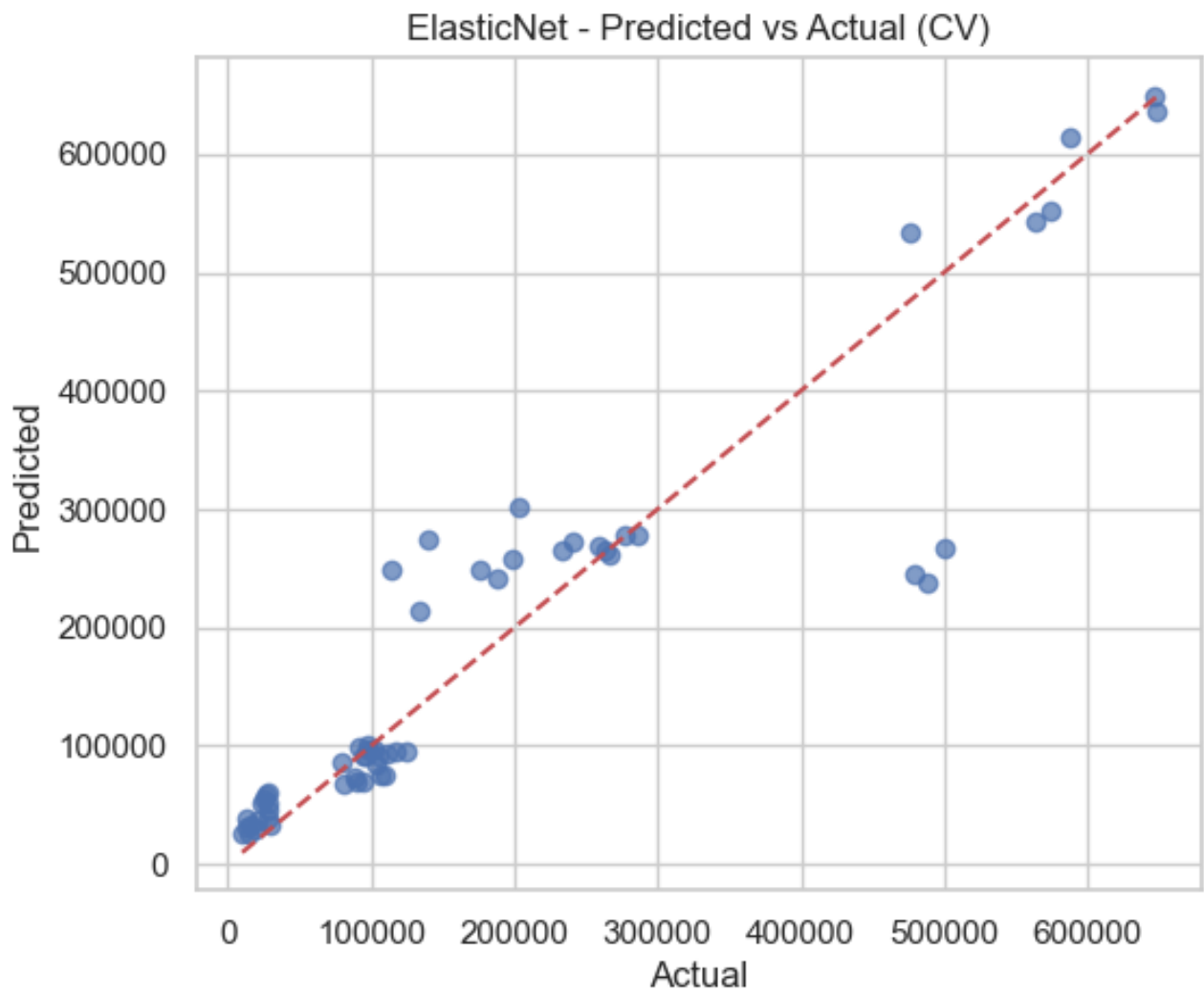


Figure 15: Aman - Elastic:Predicted vs Actual CV

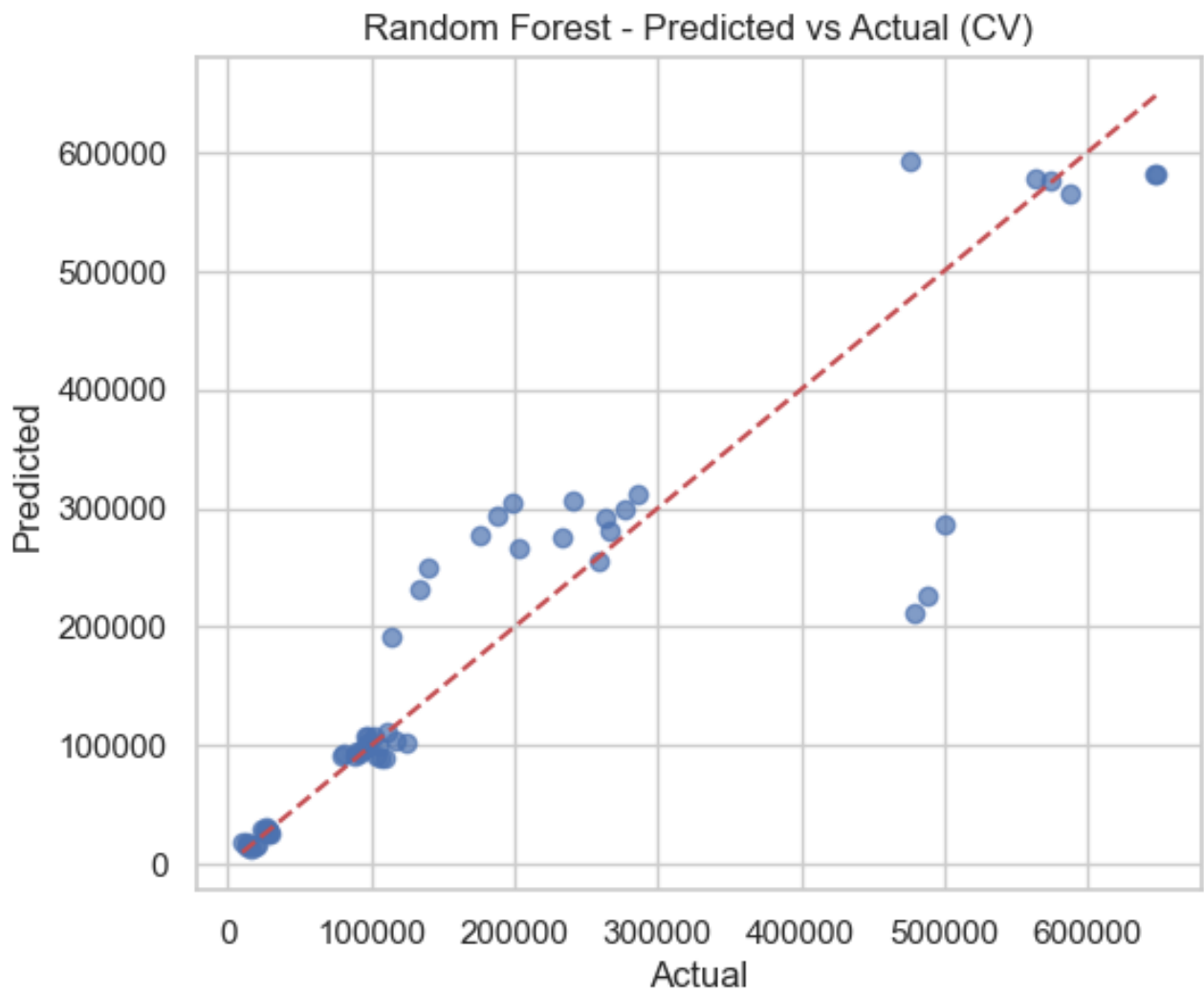


Figure 16: Aman - Random Forest:Predicted vs Actual CV

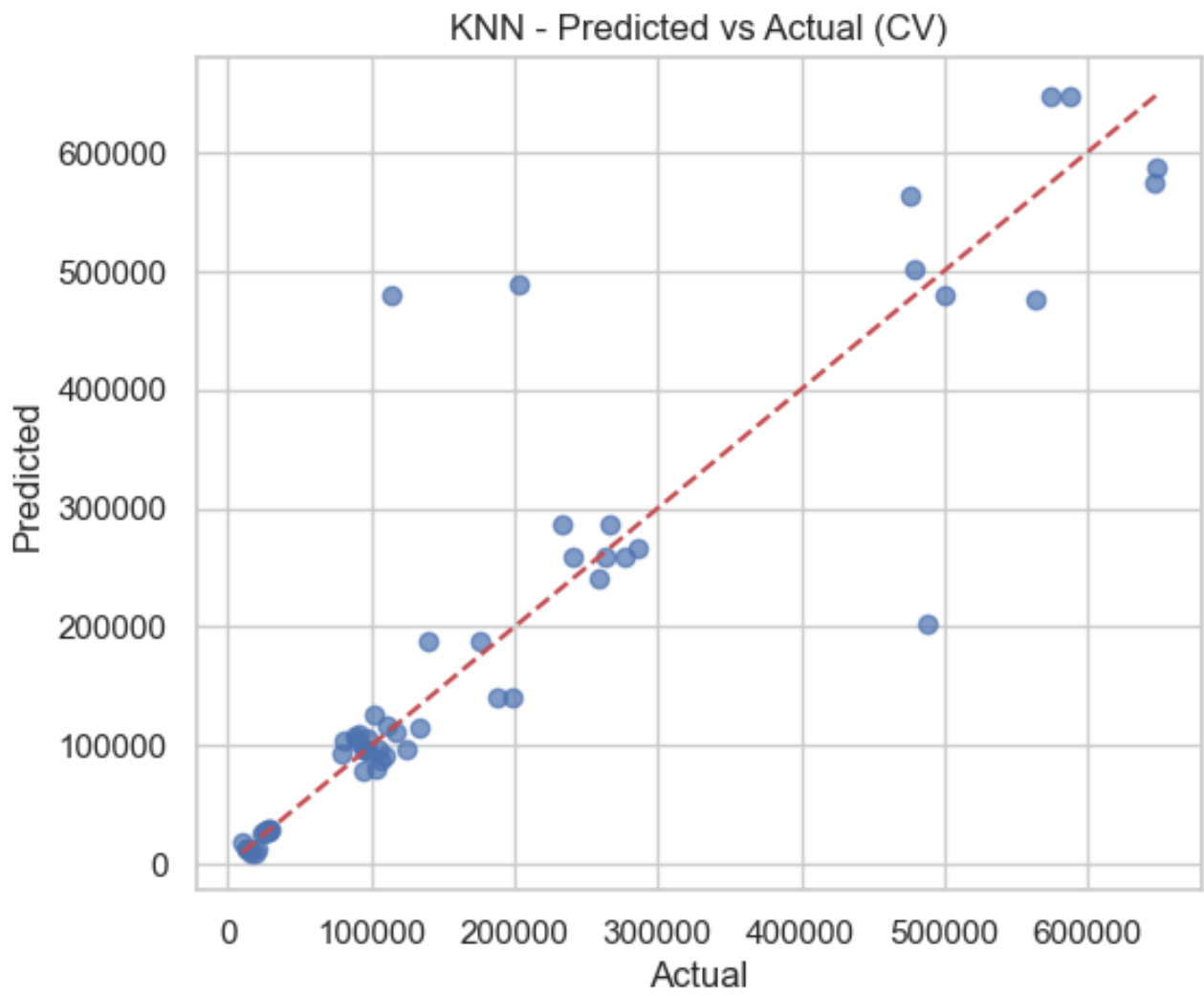


Figure 17: Aman - KNN:Predicted vs Actual CV

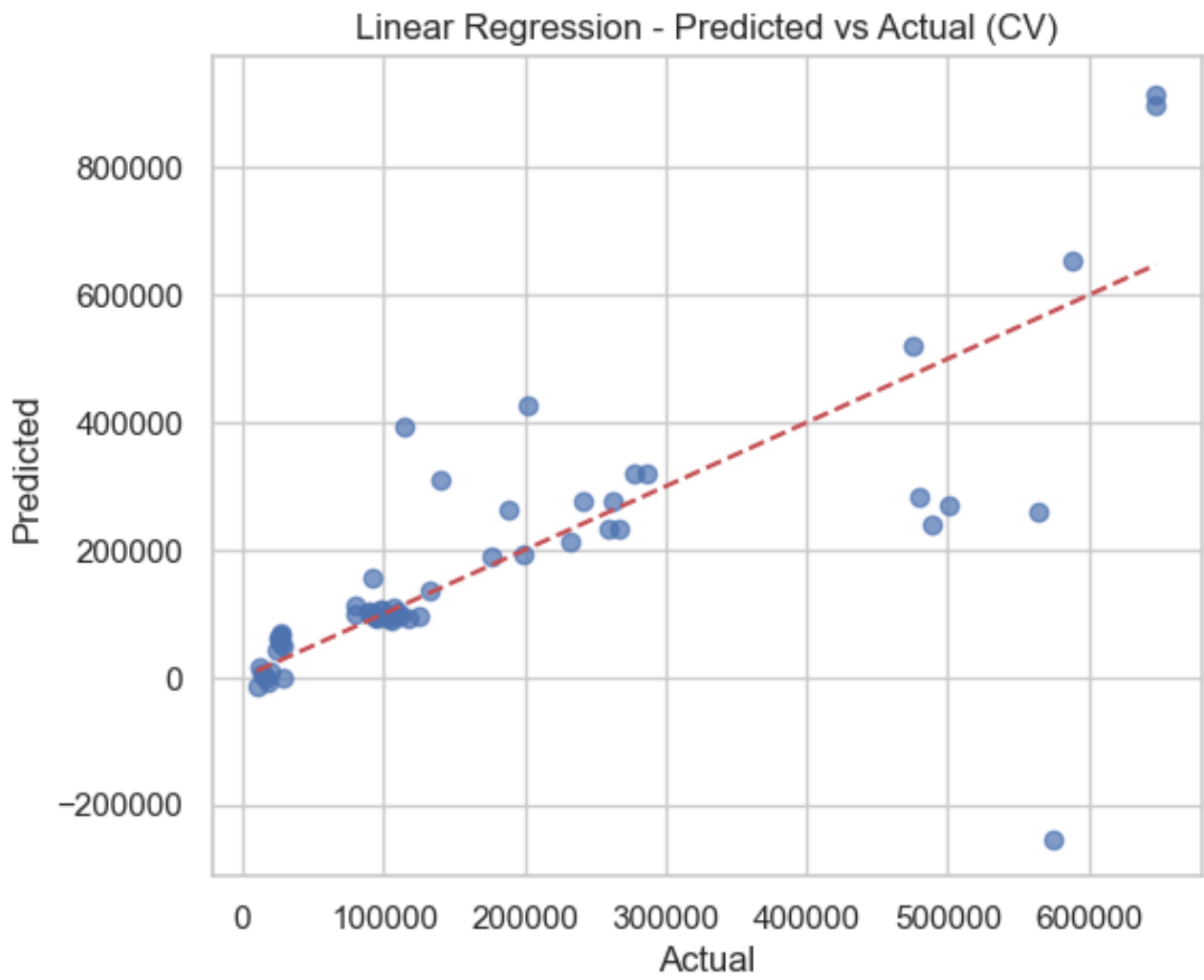


Figure 18: Aman - Linear Regression:Predicted vs Actual CV

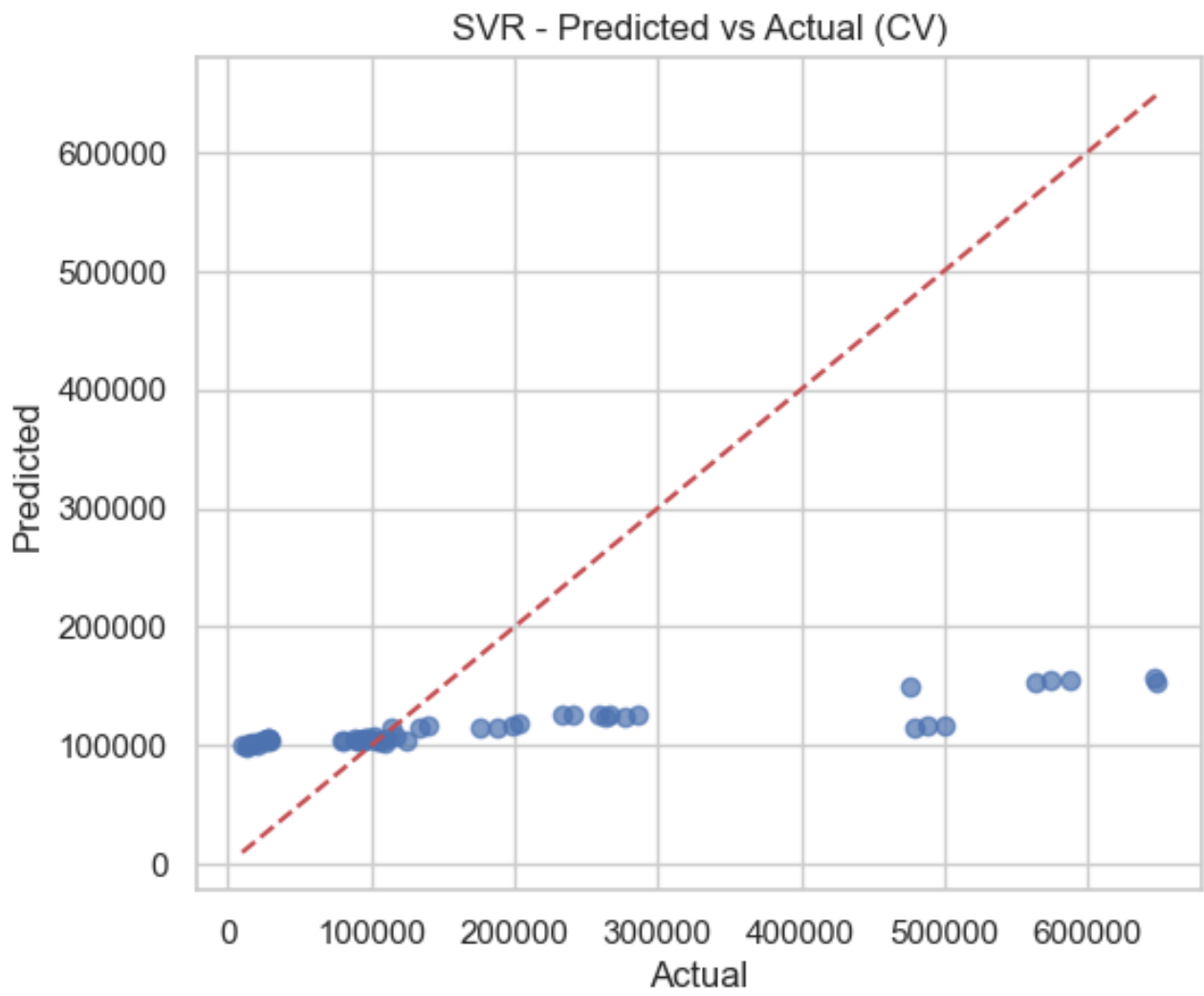


Figure 19: Aman - SVR:Predicted vs Actual CV

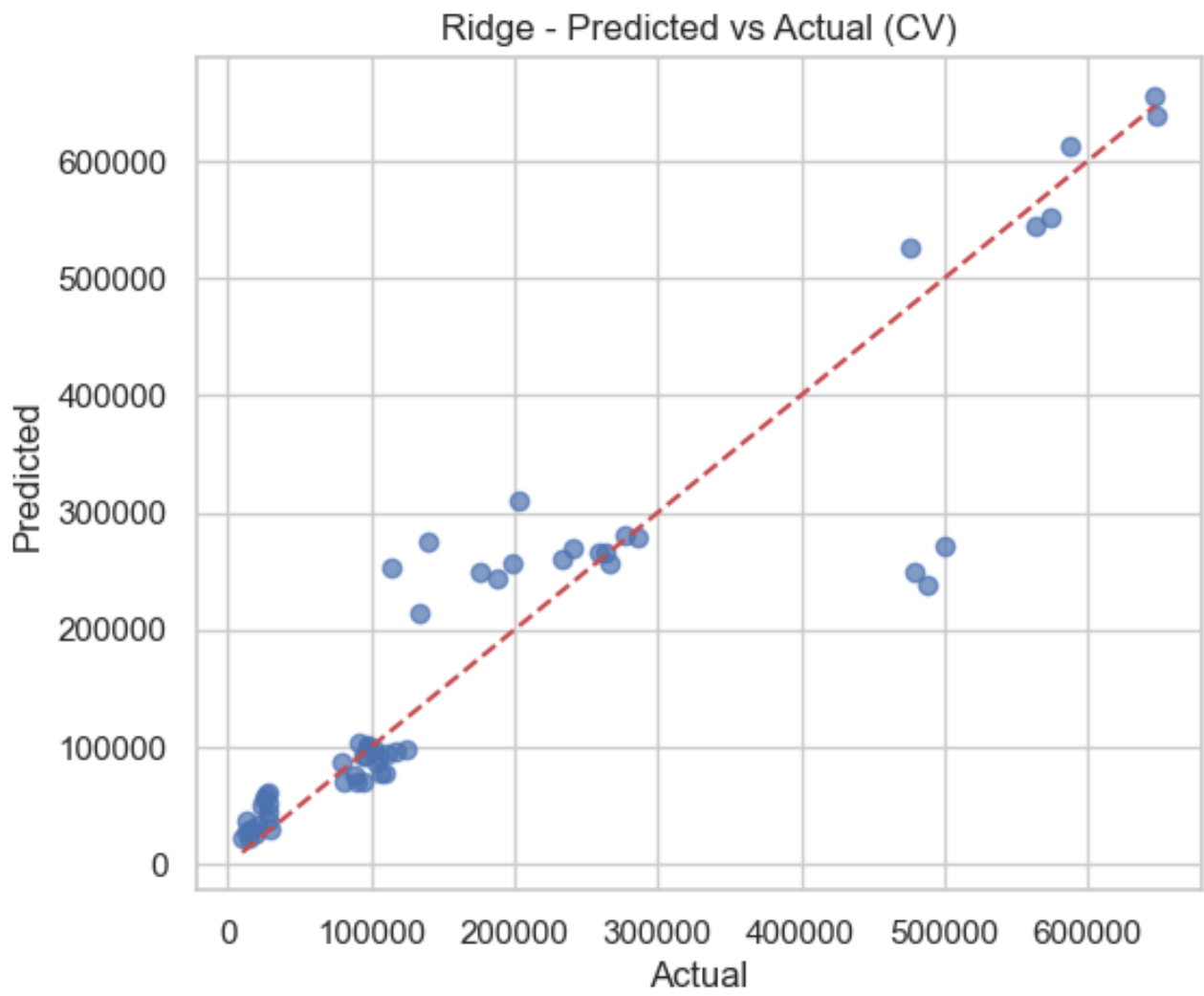


Figure 20: Aman - Ridge:Predicted vs Actual CV

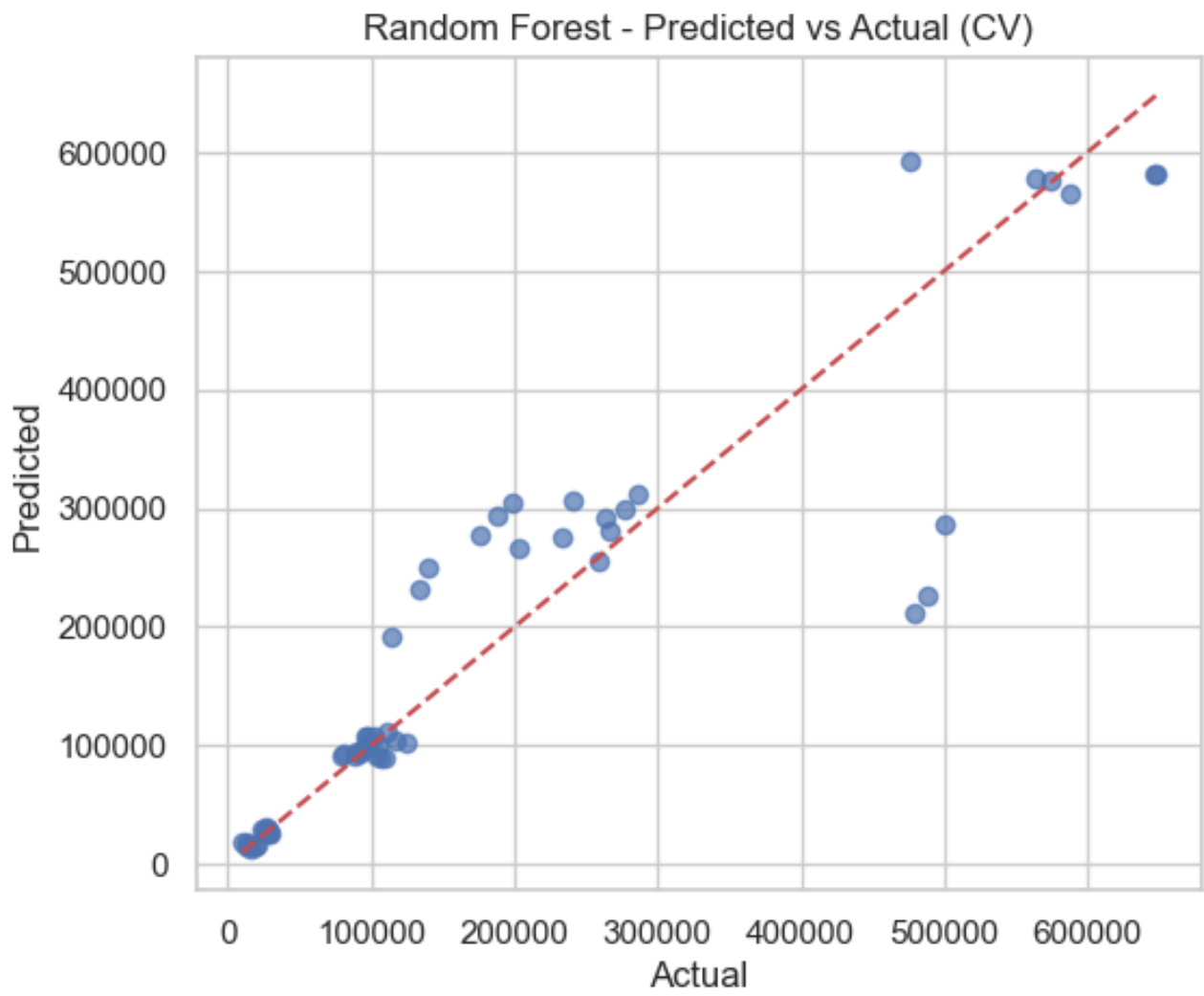


Figure 21: Aman - Random Forest:Predicted vs Actual CV

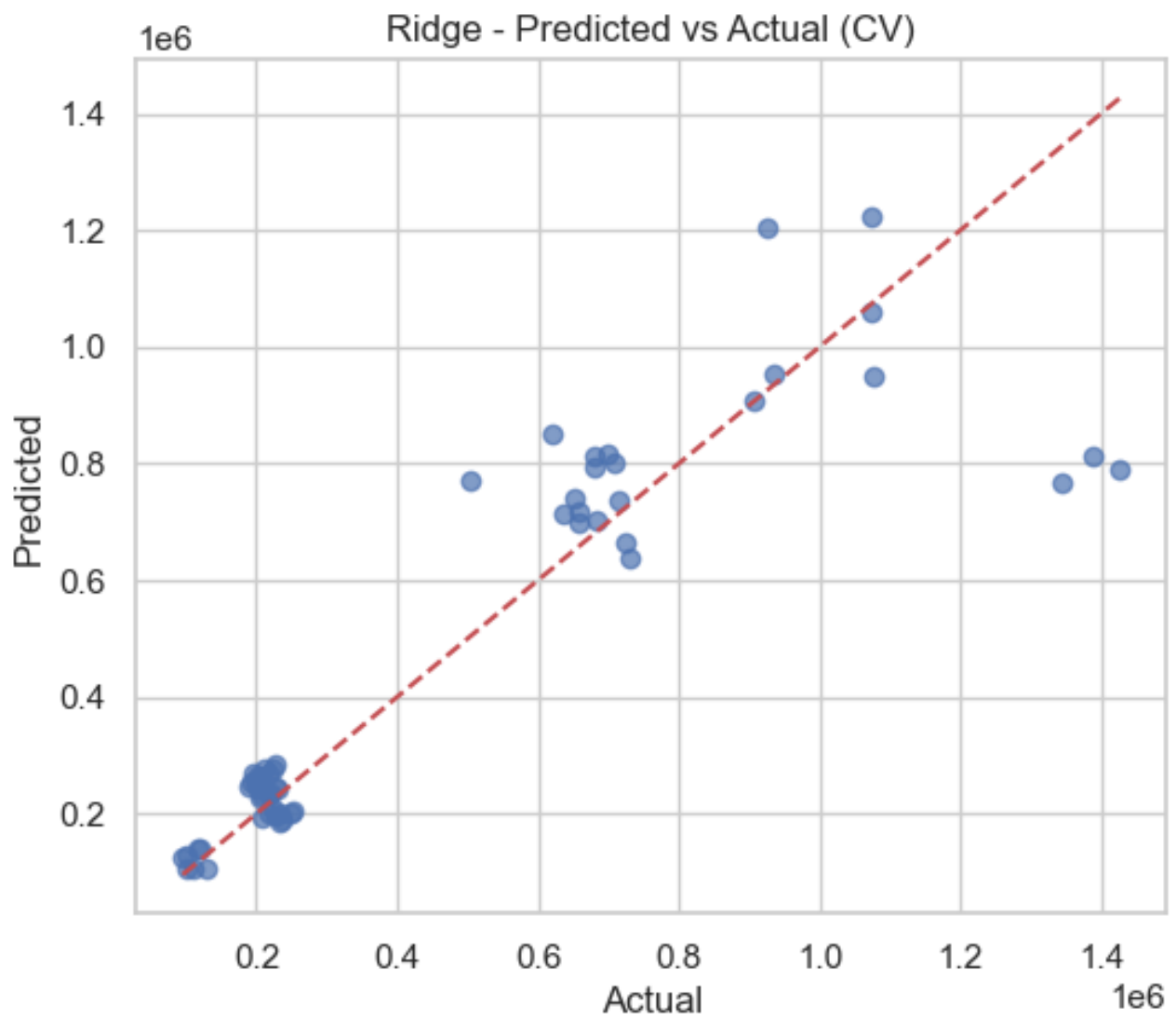


Figure 22: Boro - Ridge:Predicted vs Actual CV

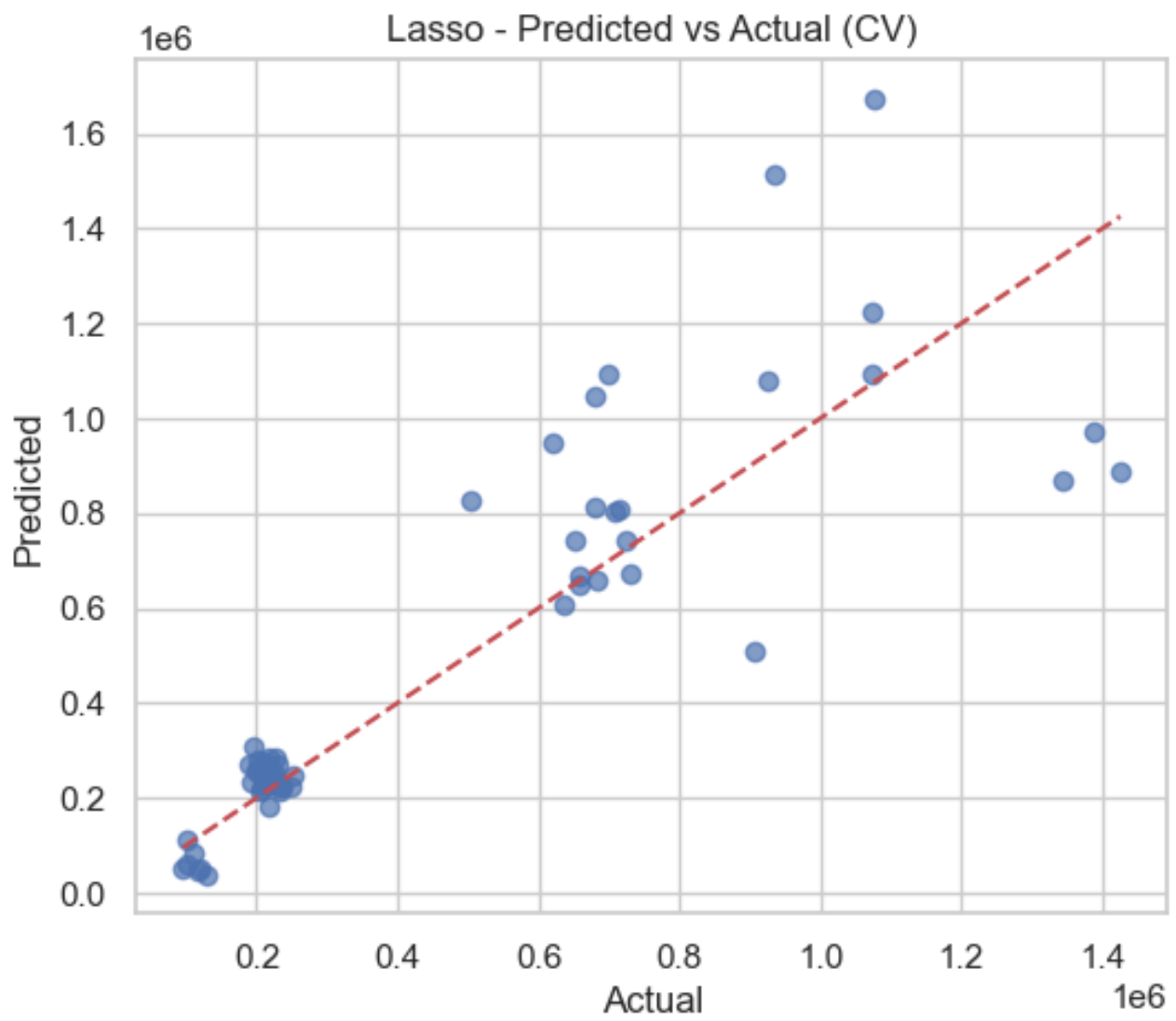


Figure 23: Boro - Lasso:Predicted vs Actual CV

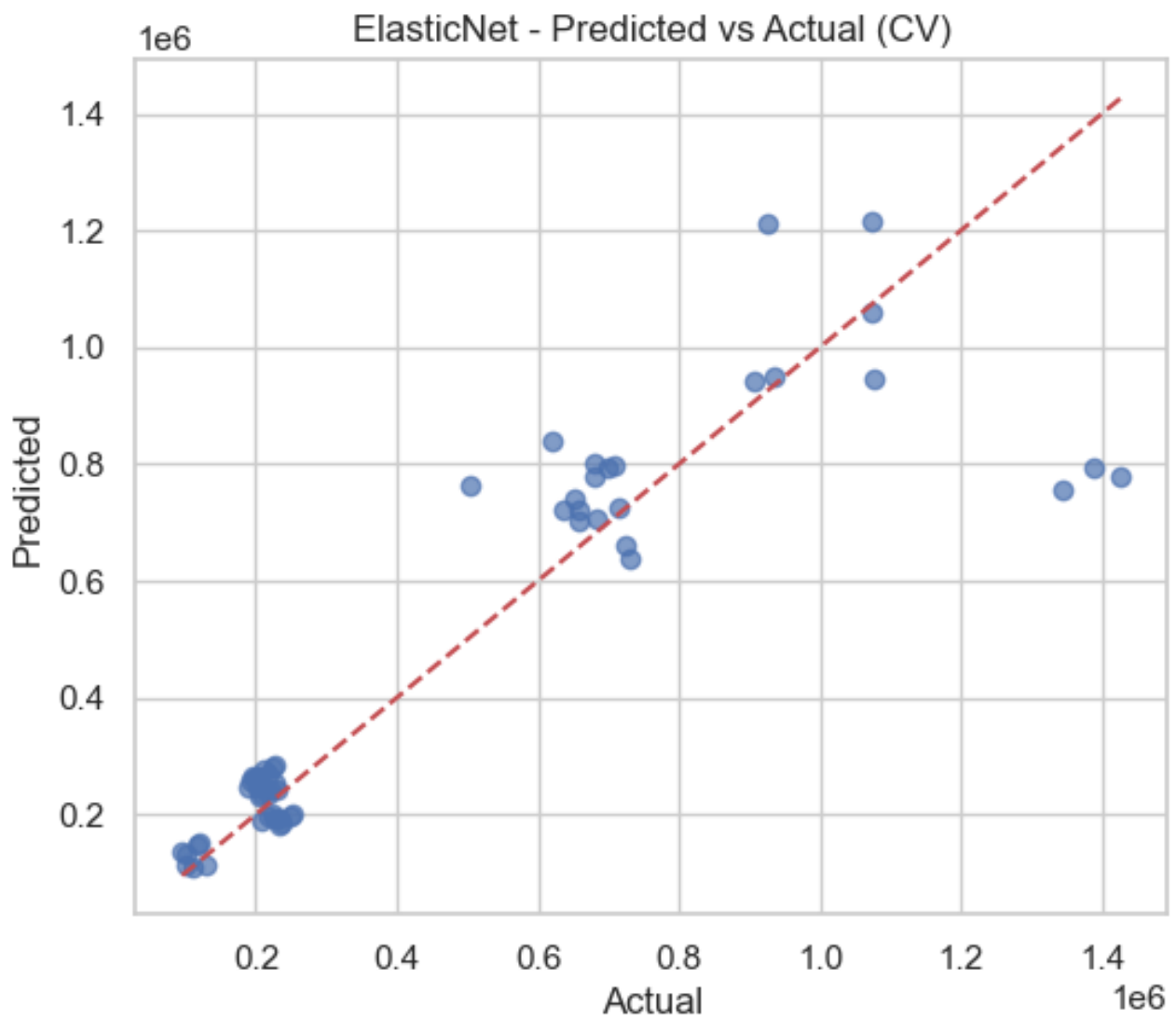


Figure 24: Boro - Elastic Net:Predicted vs Actual CV

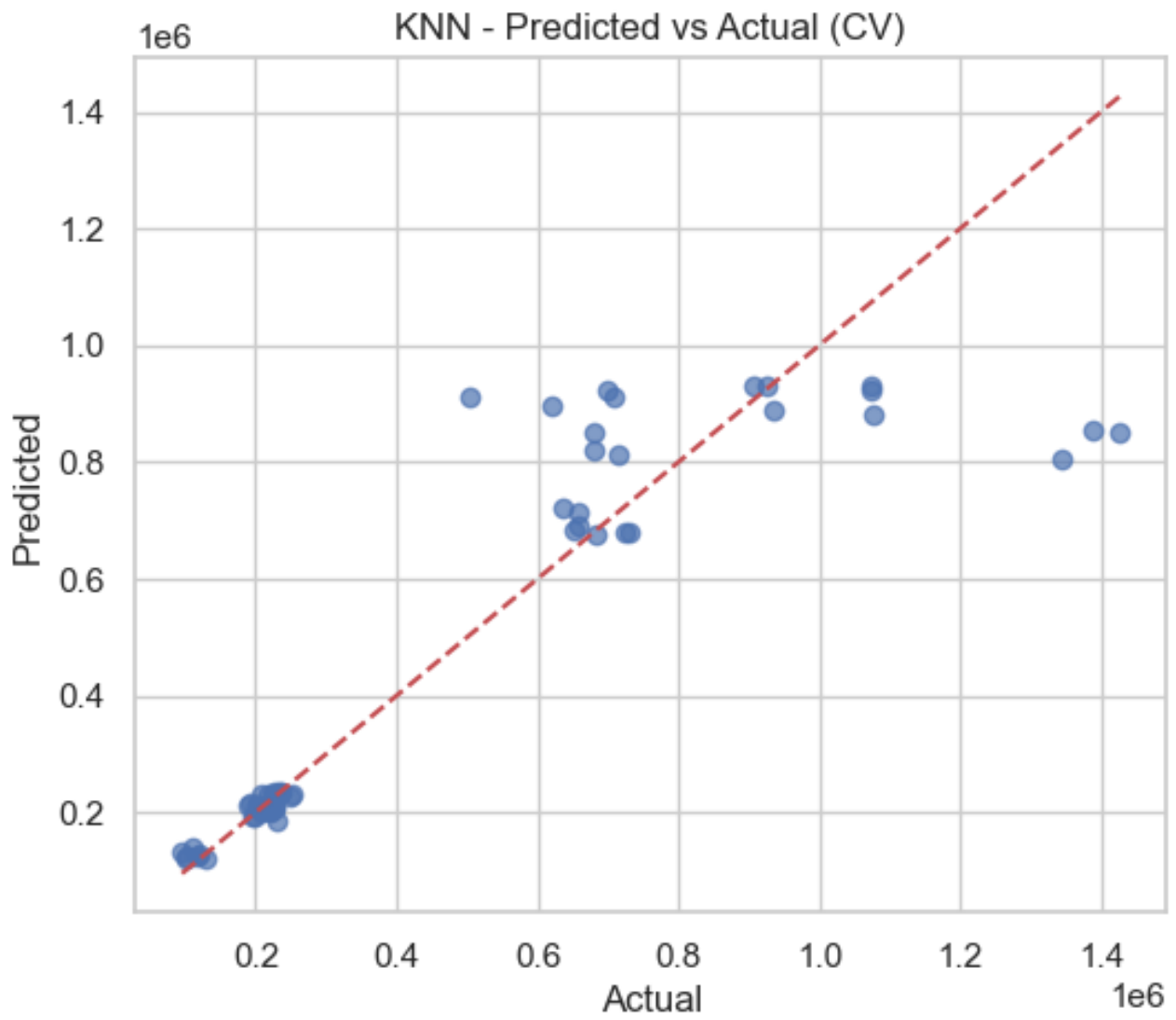


Figure 25: Boro - KNN:Predicted vs Actual CV

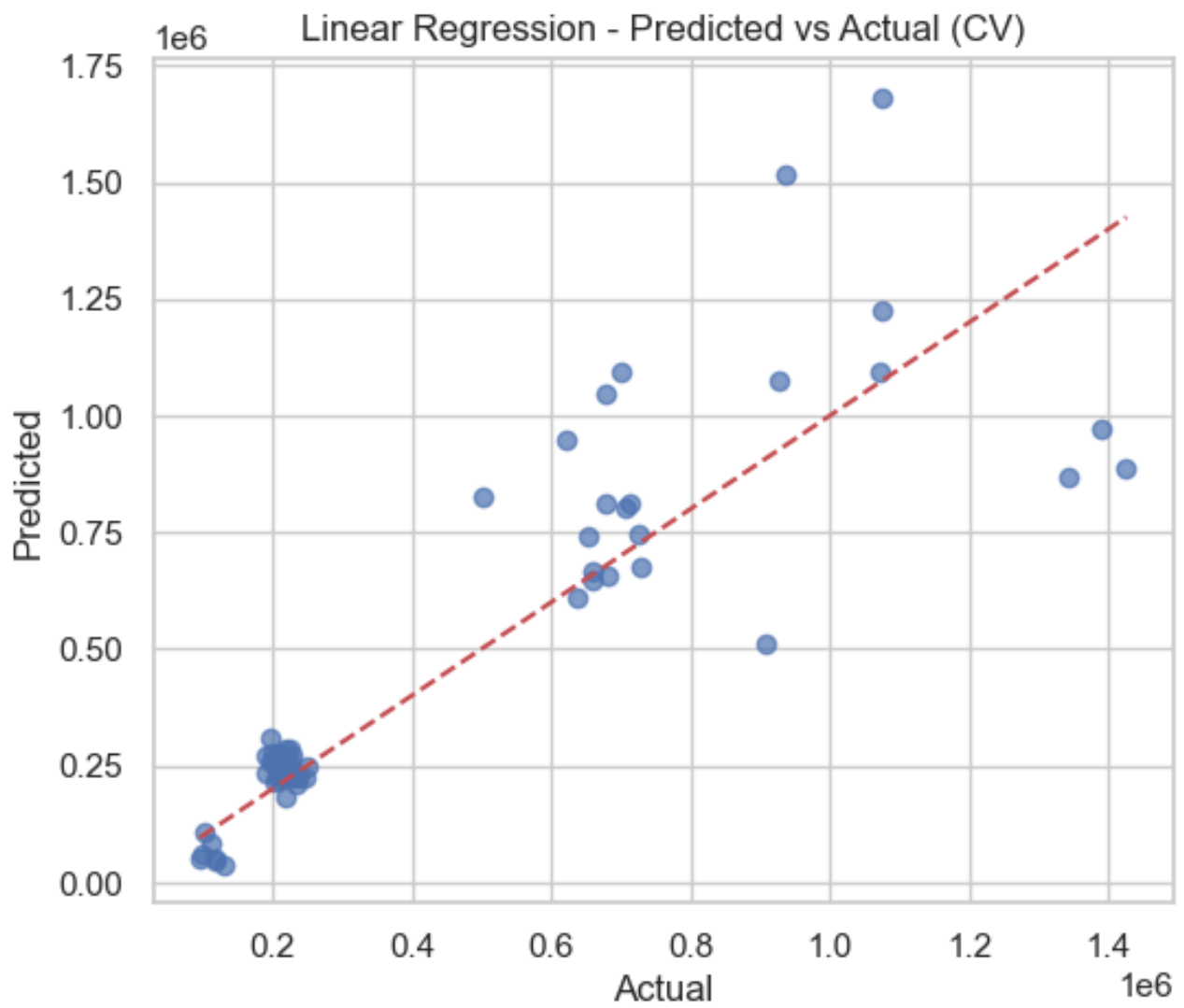


Figure 26: Boro - Linear Regression:Predicted vs Actual CV

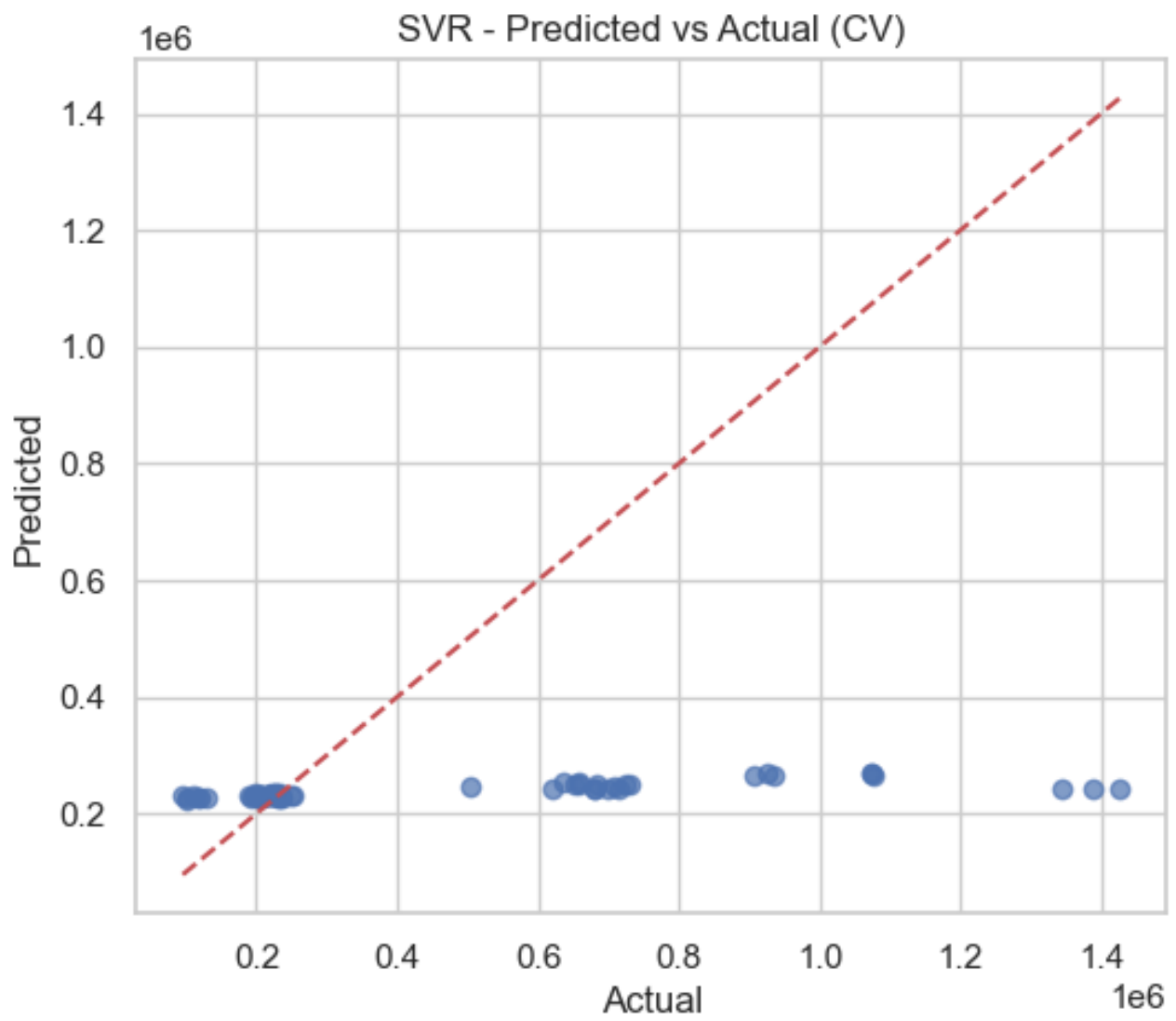


Figure 27: Boro - SVR:Predicted vs Actual CV

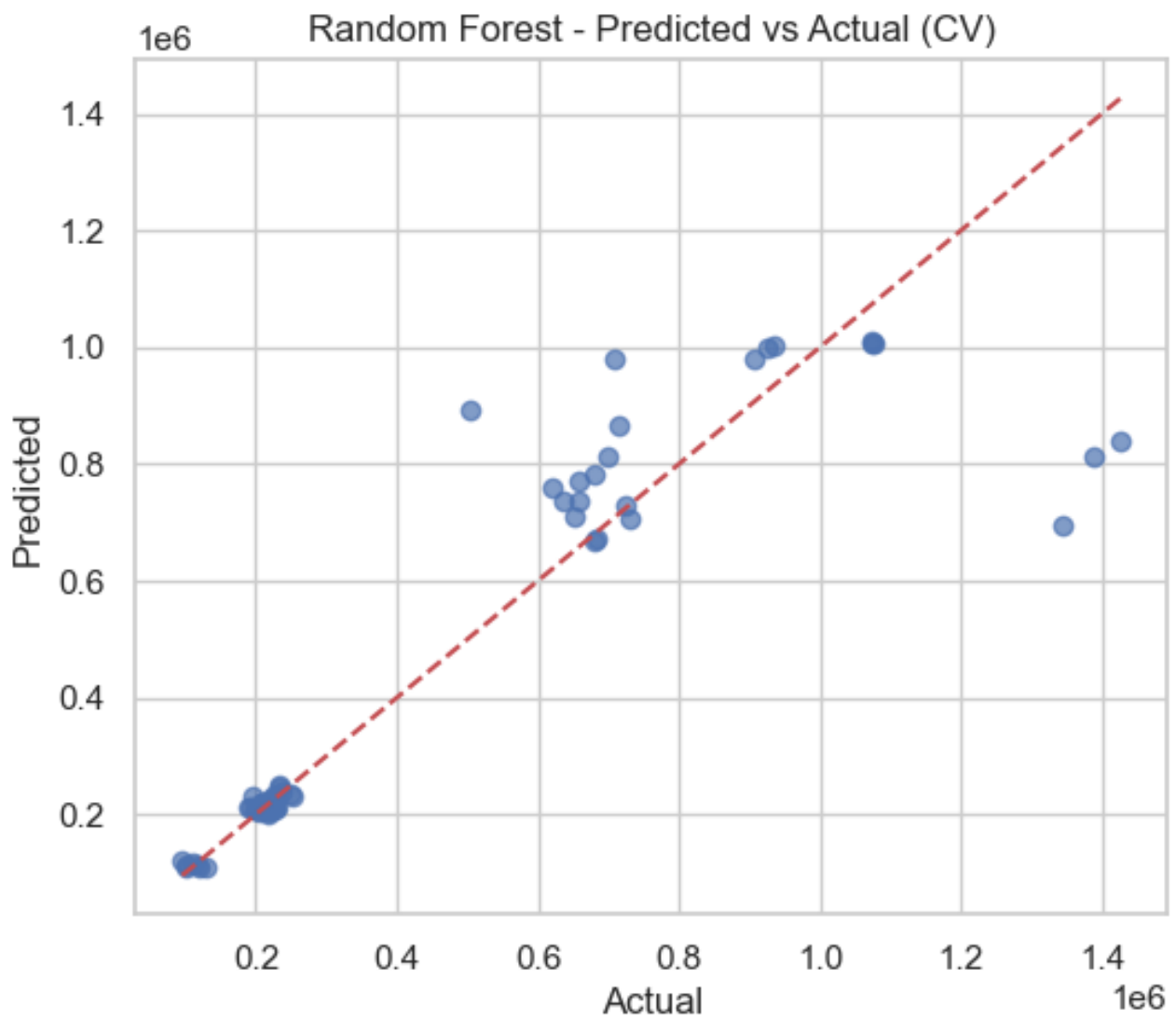


Figure 28: Boro - Random Forest:Predicted vs Actual CV

5.1.4 Permutation Importance

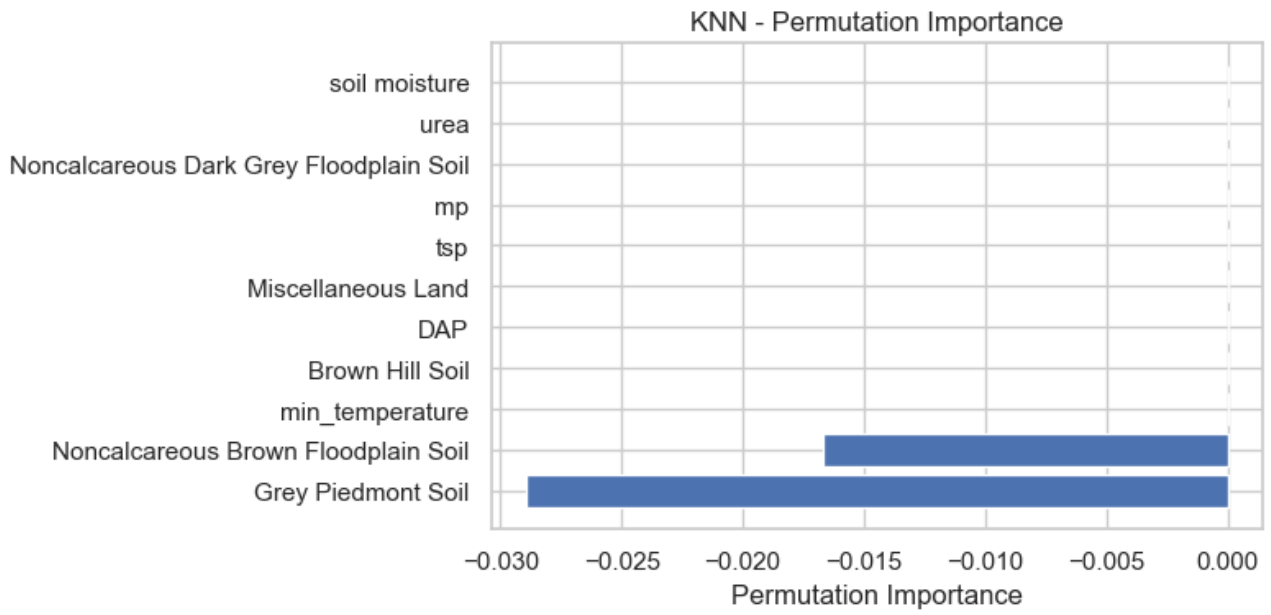


Figure 29: Aus - KNN:Permutation Importance

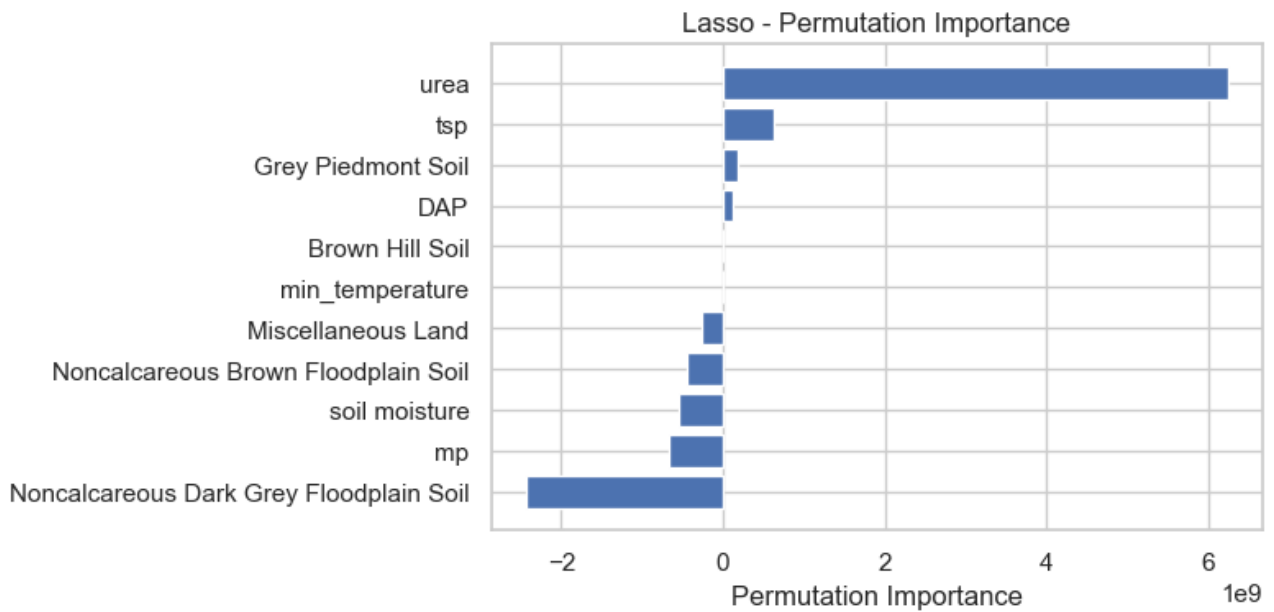


Figure 30: Aus - Lasso:Permutation Importance

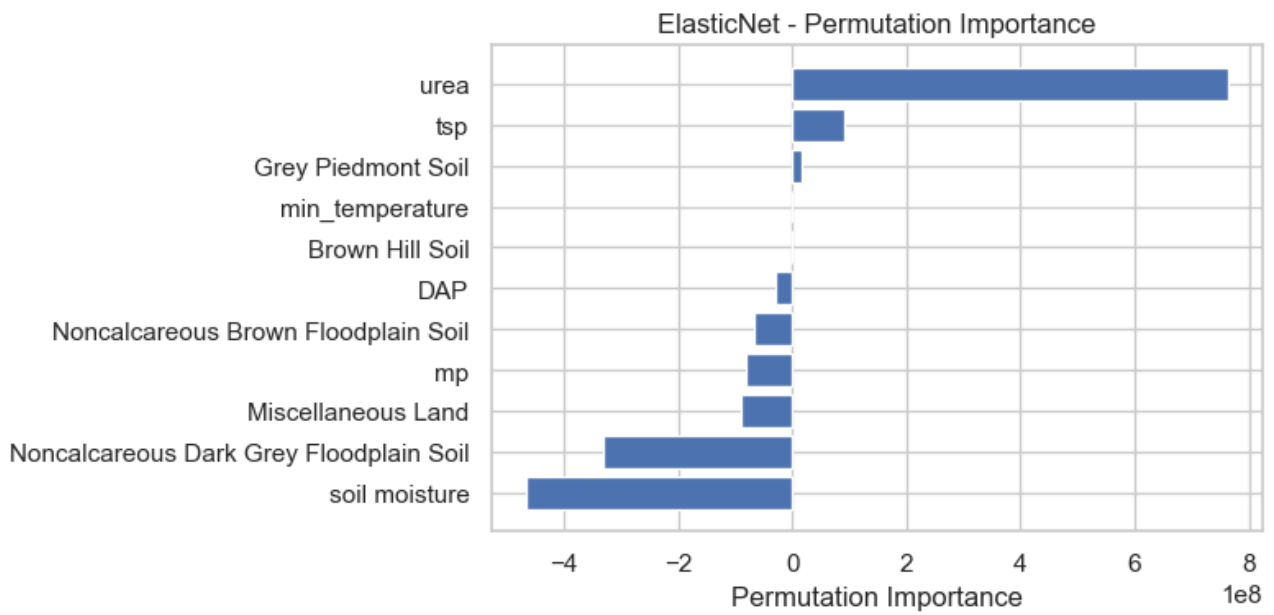


Figure 31: Aus - Elastic Net: Permutation Importance

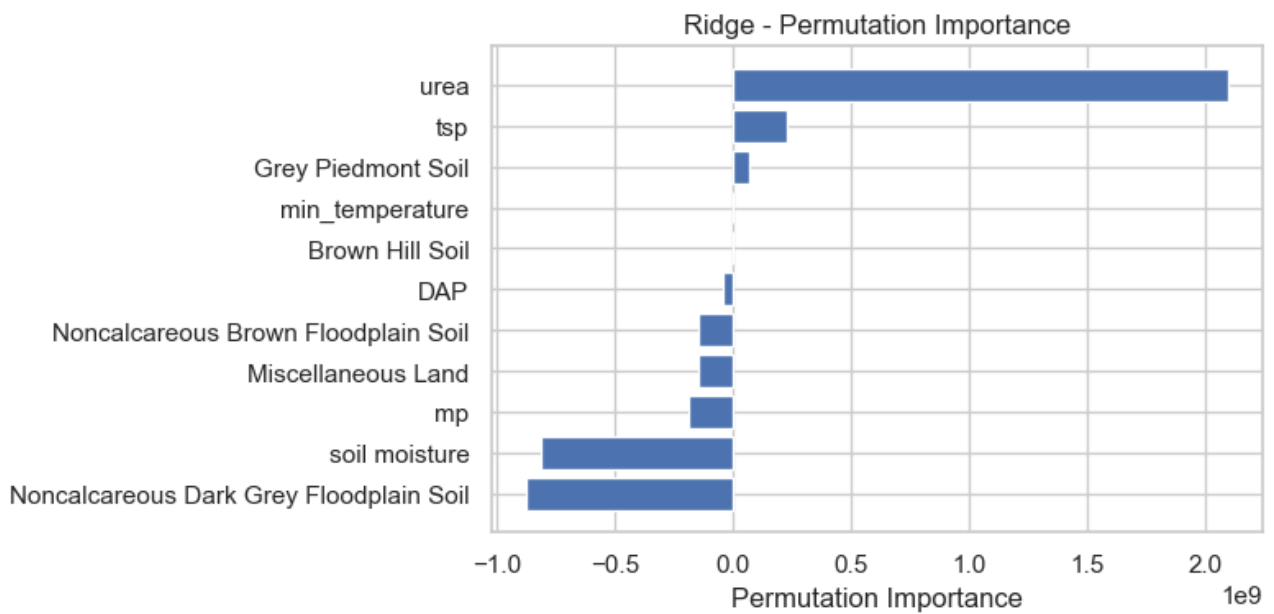


Figure 32: Aus - Ridge:Permutation Importance

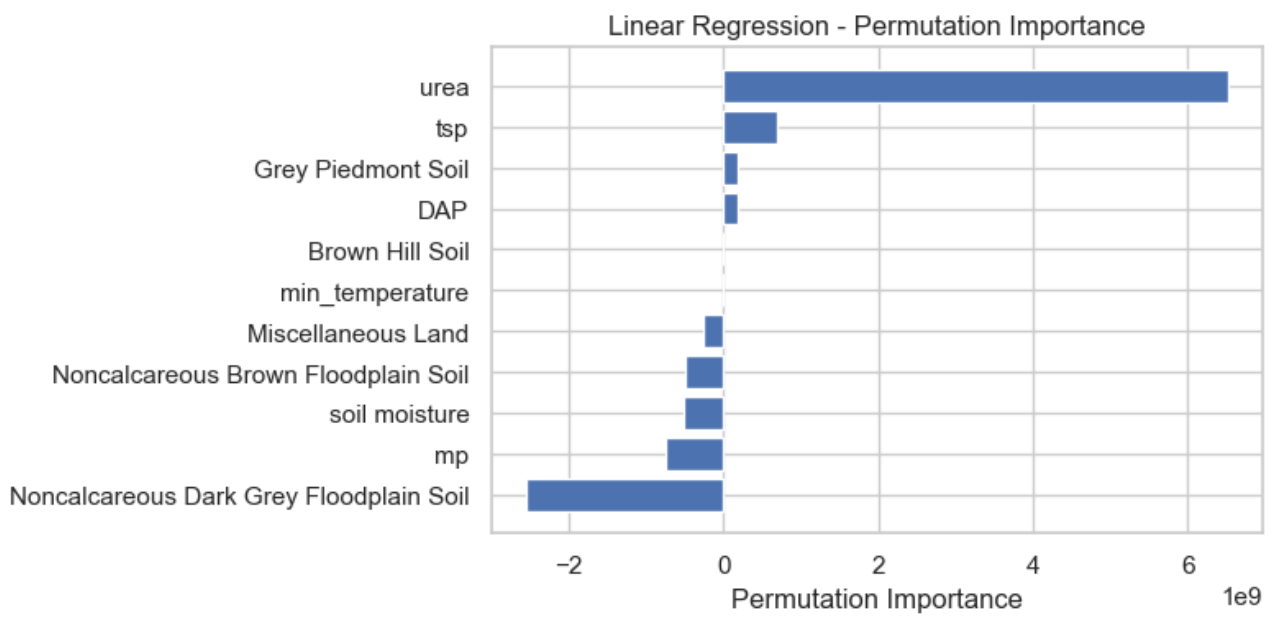


Figure 33: Aus - Linear Regression:Permutation Importance

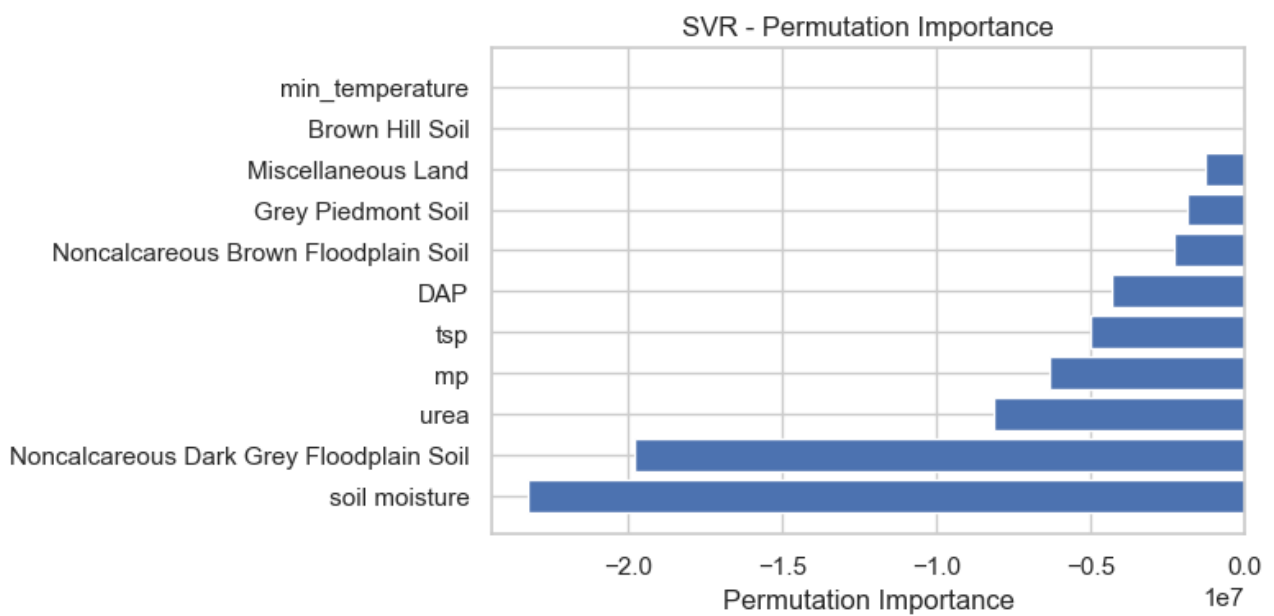


Figure 34: Aus - SVR:Permutation Importance

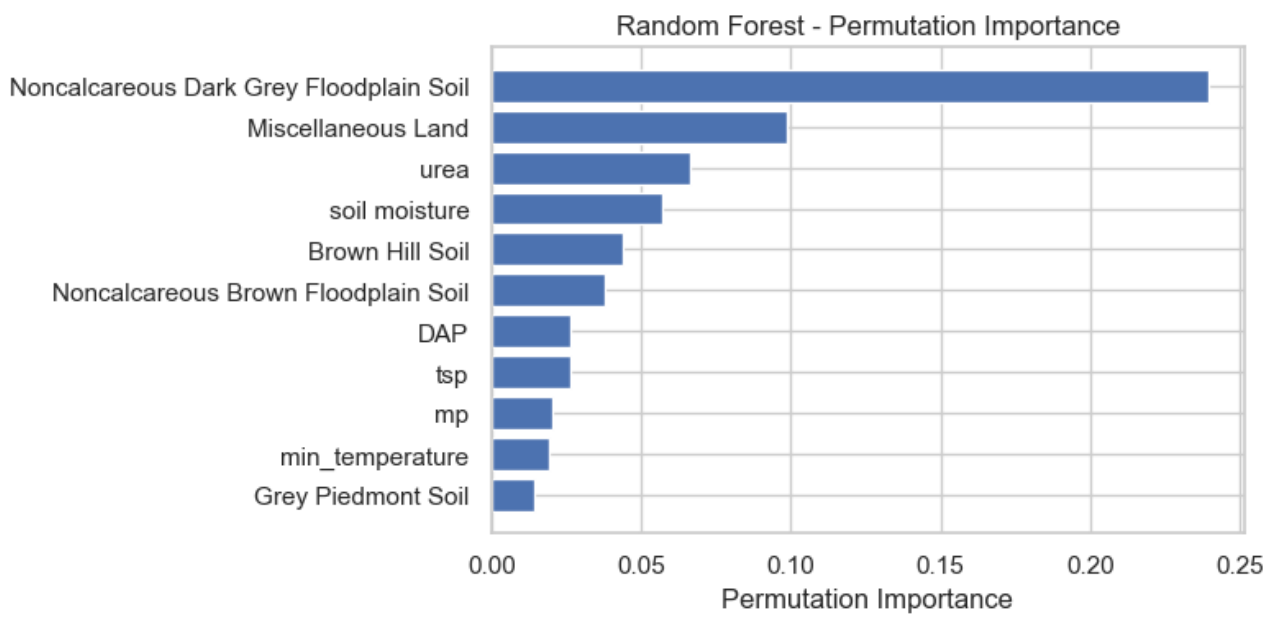


Figure 35: Aus - Random Forest:Permutation Importance

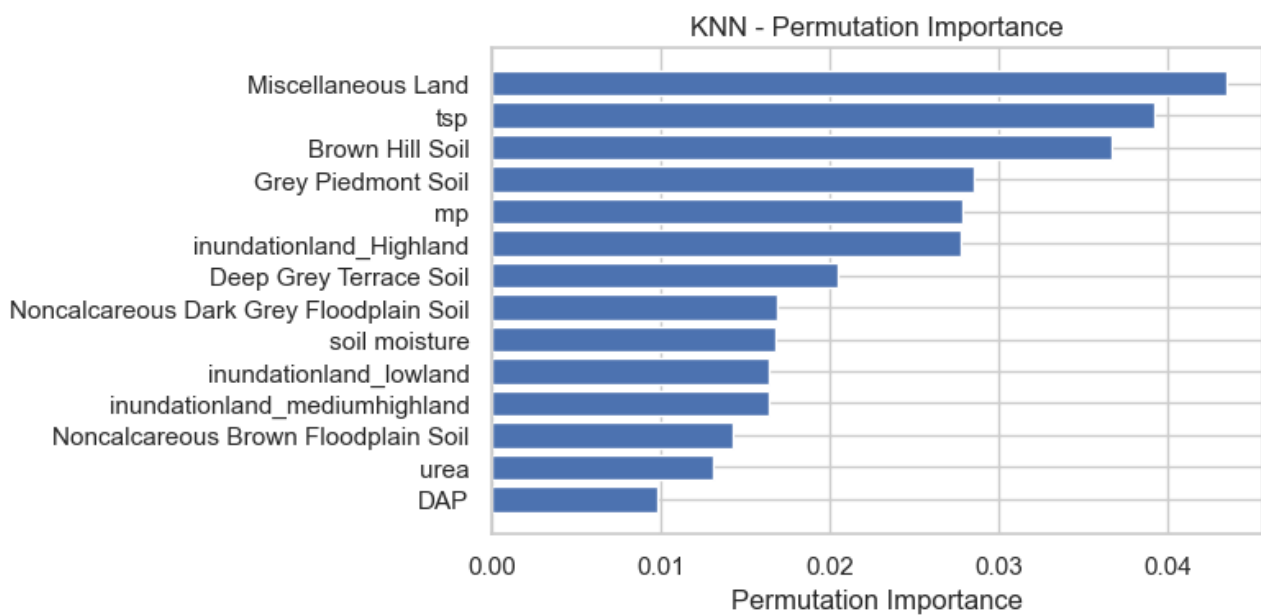


Figure 36: Aman - KNN:Permutation Importance

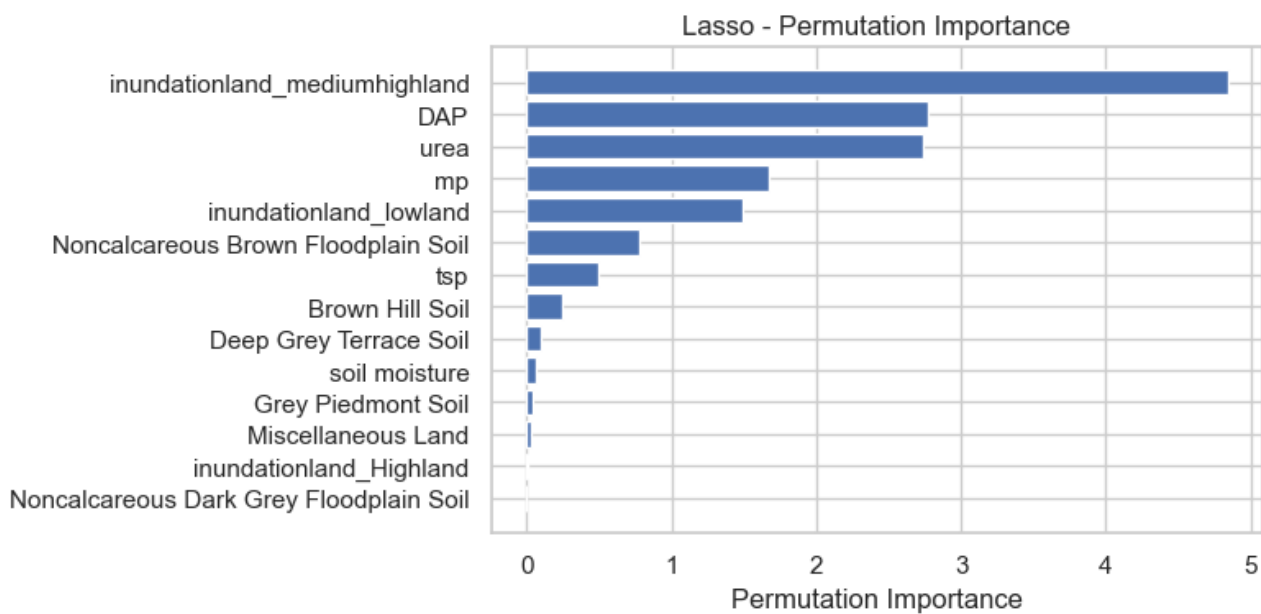


Figure 37: Aman - Lasso:Permutation Importance

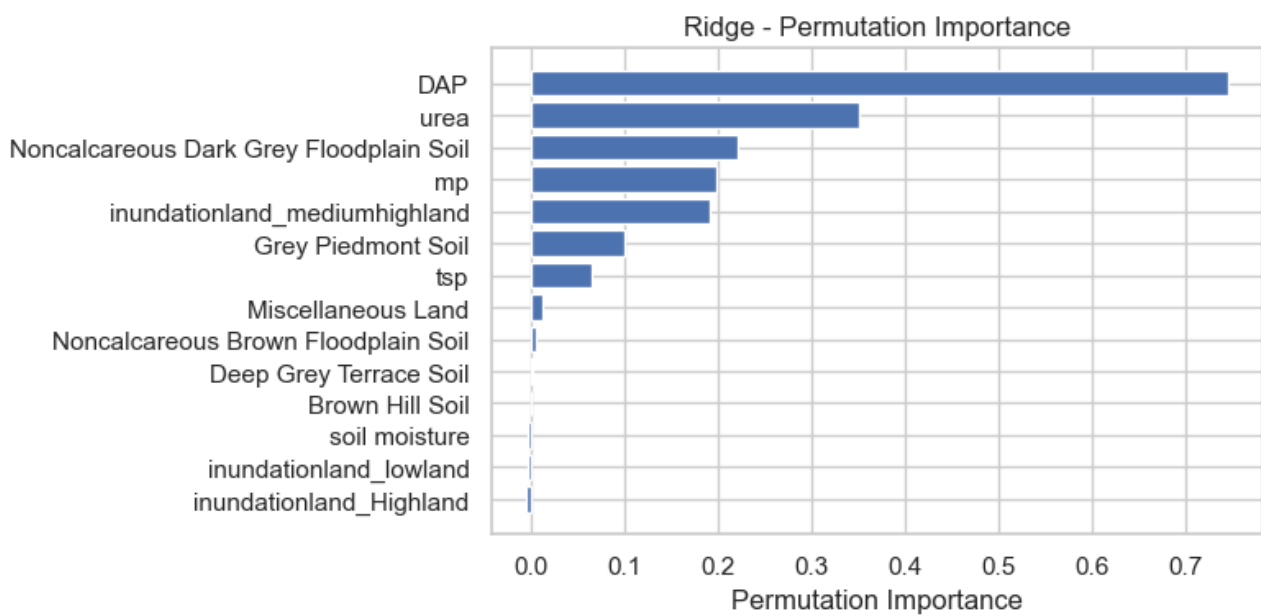


Figure 38: Aman - Ridge:Permutation Importance

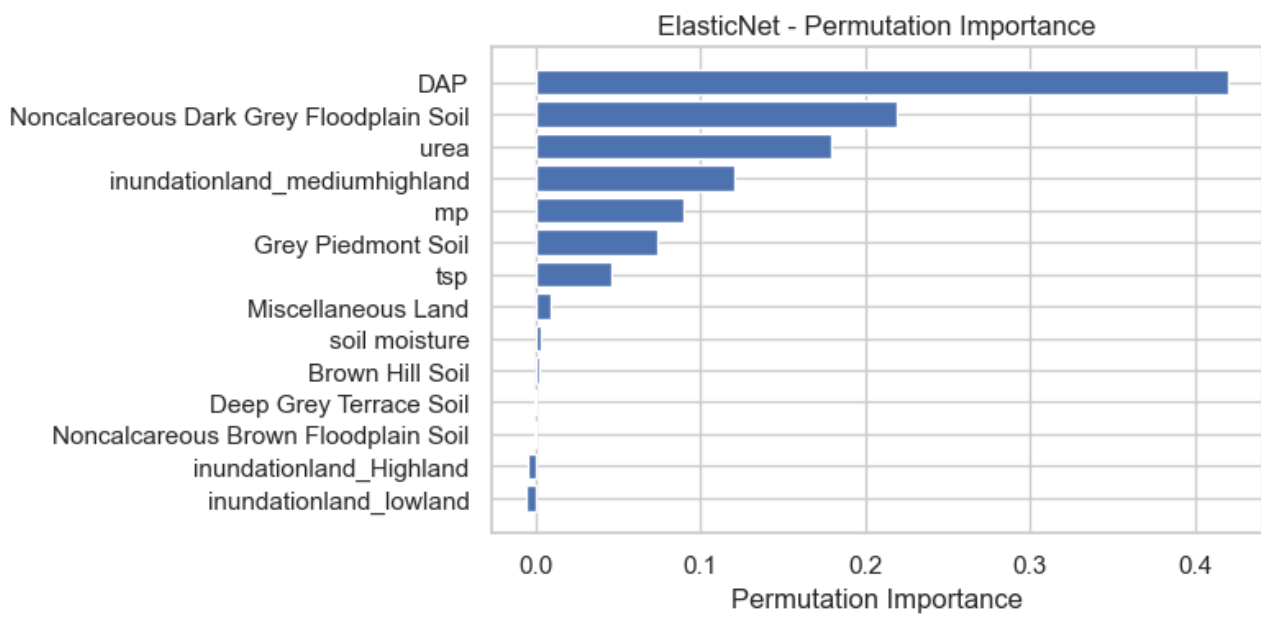


Figure 39: Aman - Elastic Net:Permutation Importance

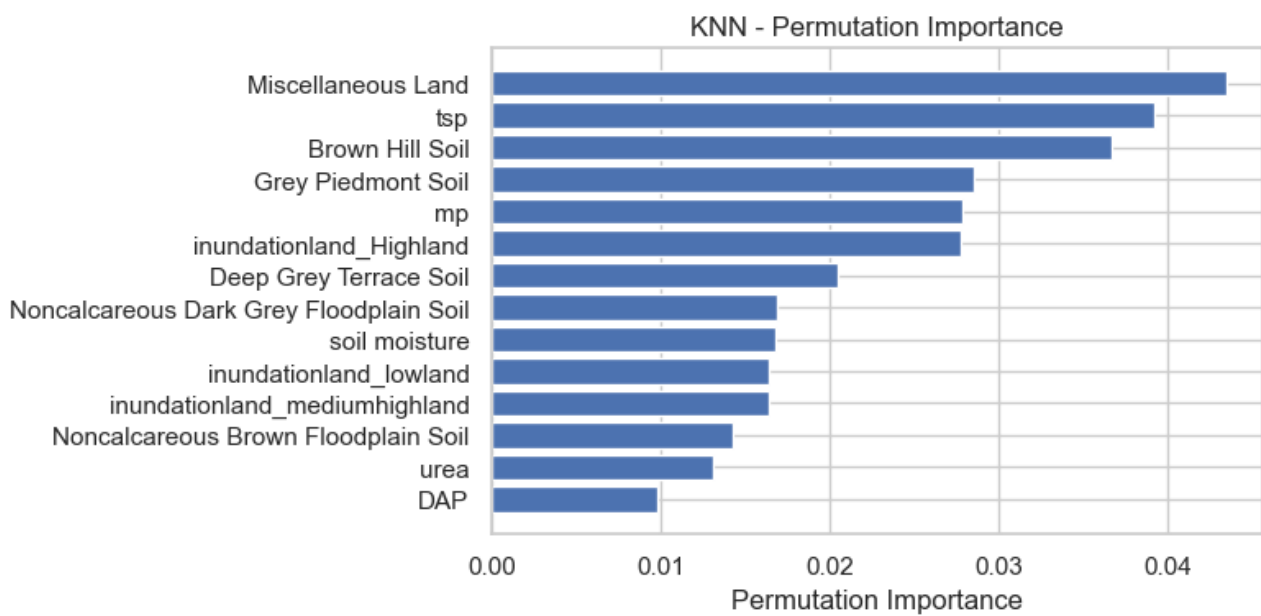


Figure 40: Aman - KNN: Permutation Importance

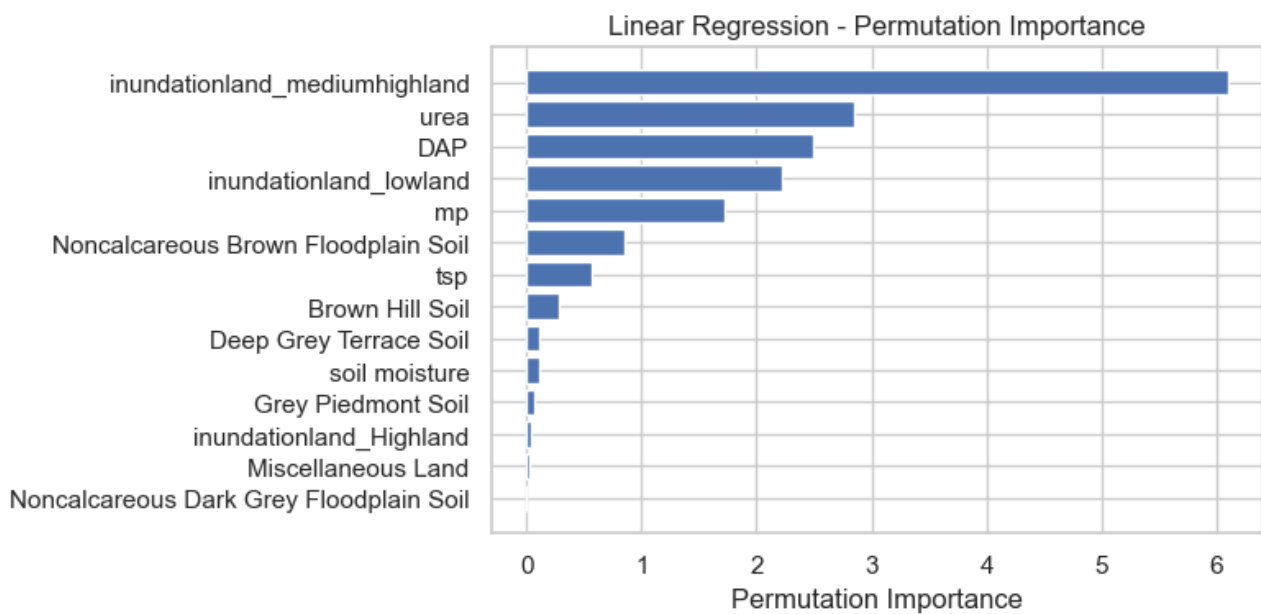


Figure 41: Aman - Linear Regression:Permutation Importance

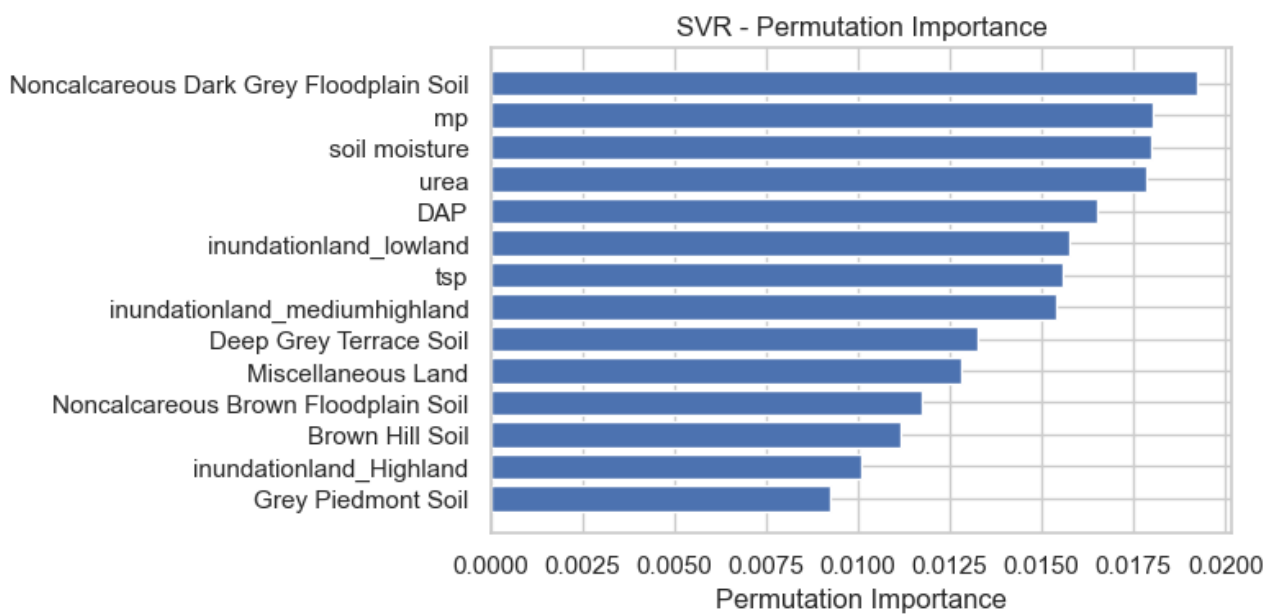


Figure 42: Aman - SVR:Permutation Importance

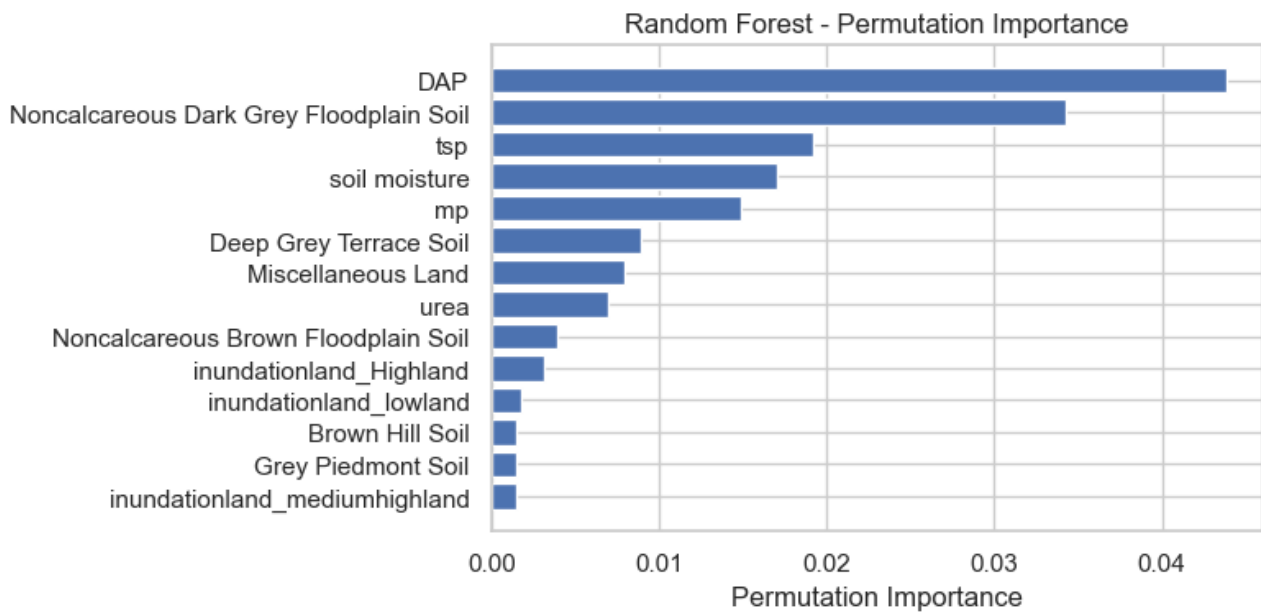


Figure 43: Aman - Random Forest:Permutation Importance

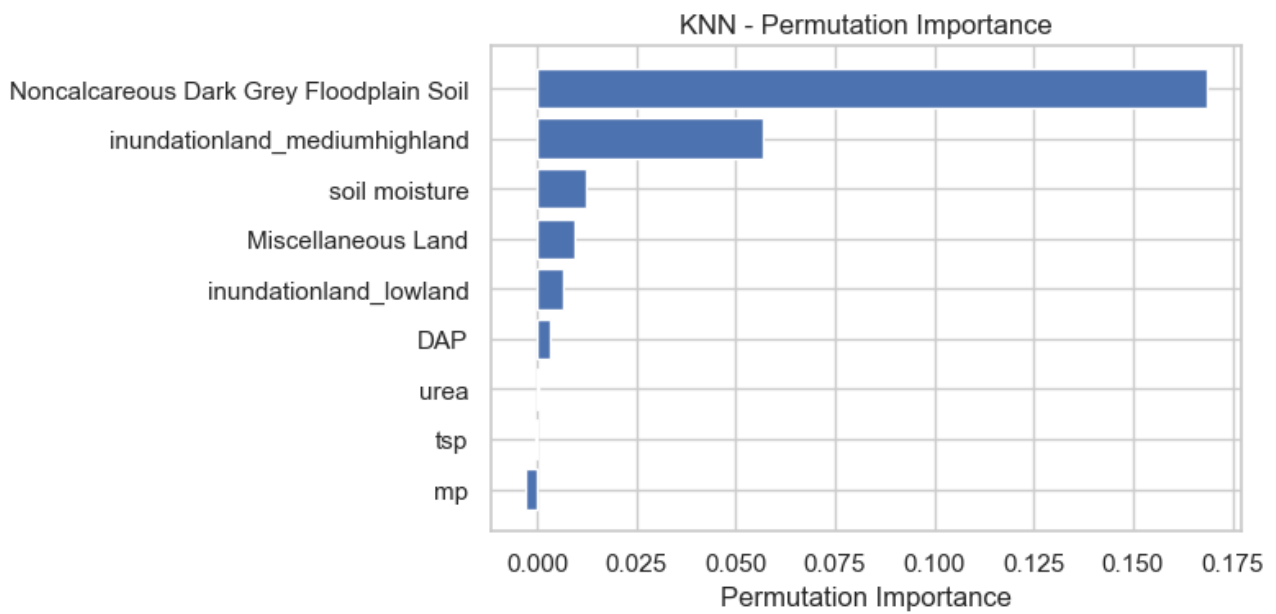


Figure 44: Boro - KNN:Permutation Importance

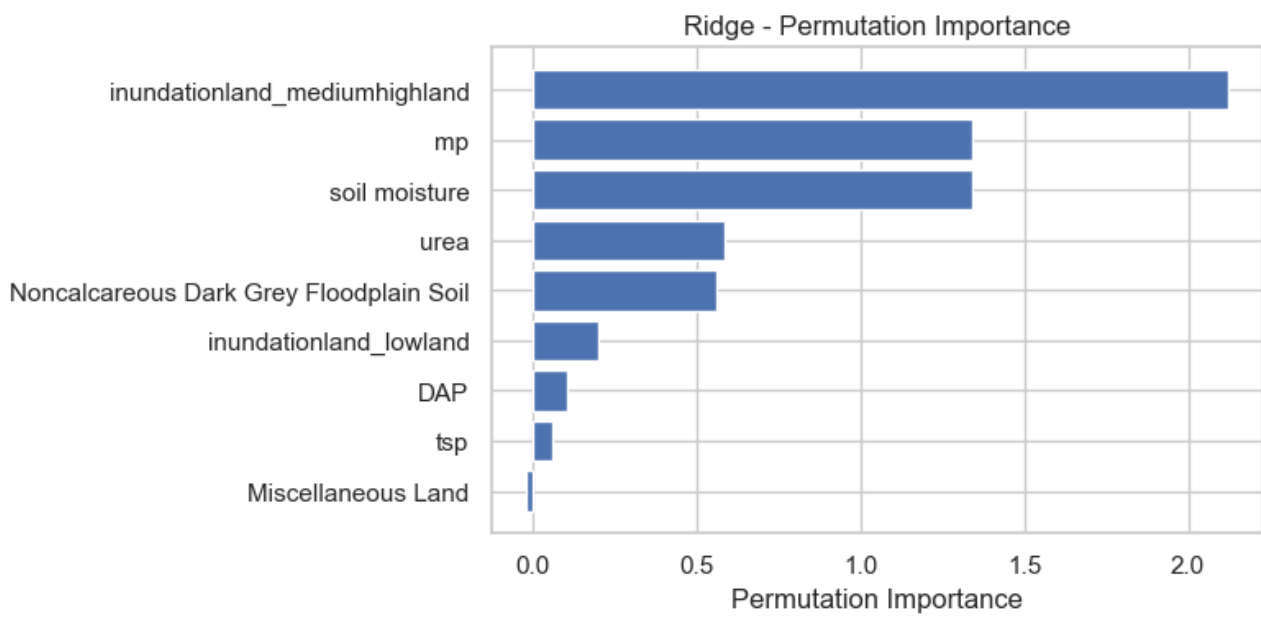


Figure 45: Boro - Ridge:Permutation Importance

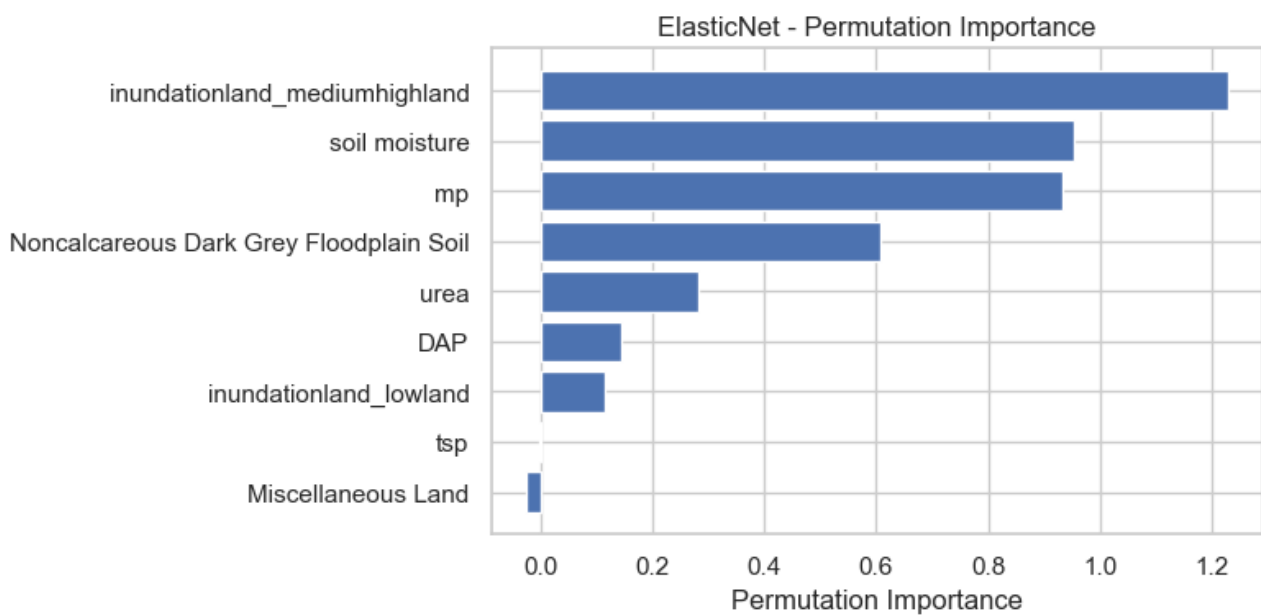


Figure 46: Boro - Elastic Net:Permutation Importance

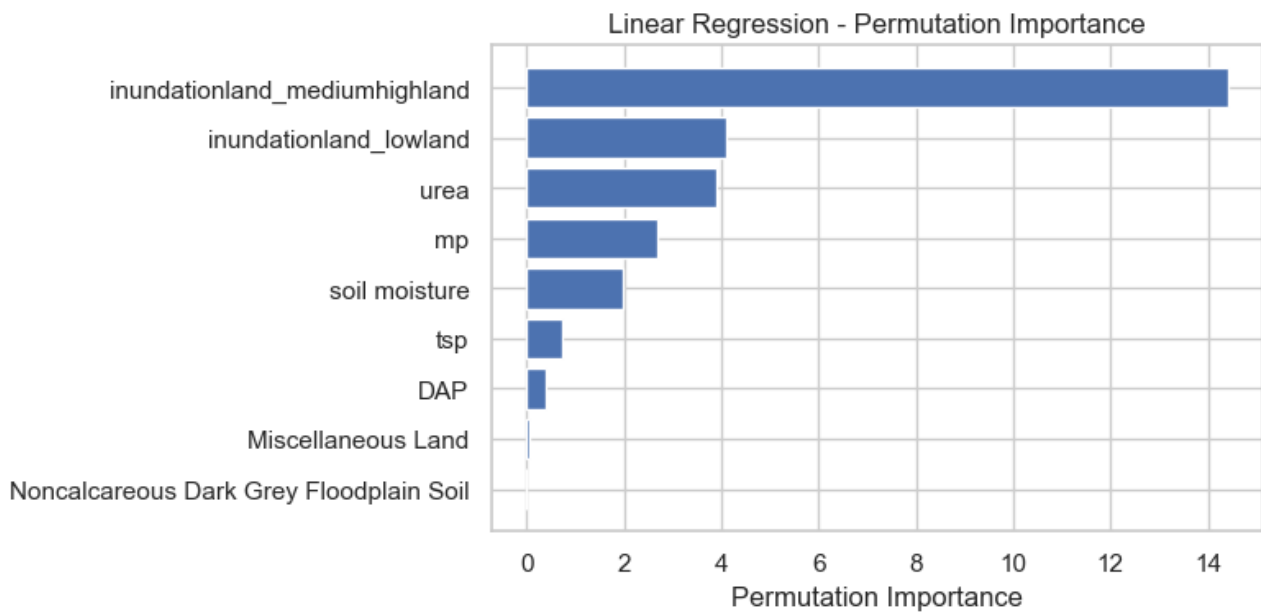


Figure 47: Boro - Linear Regression:Permutation Importance

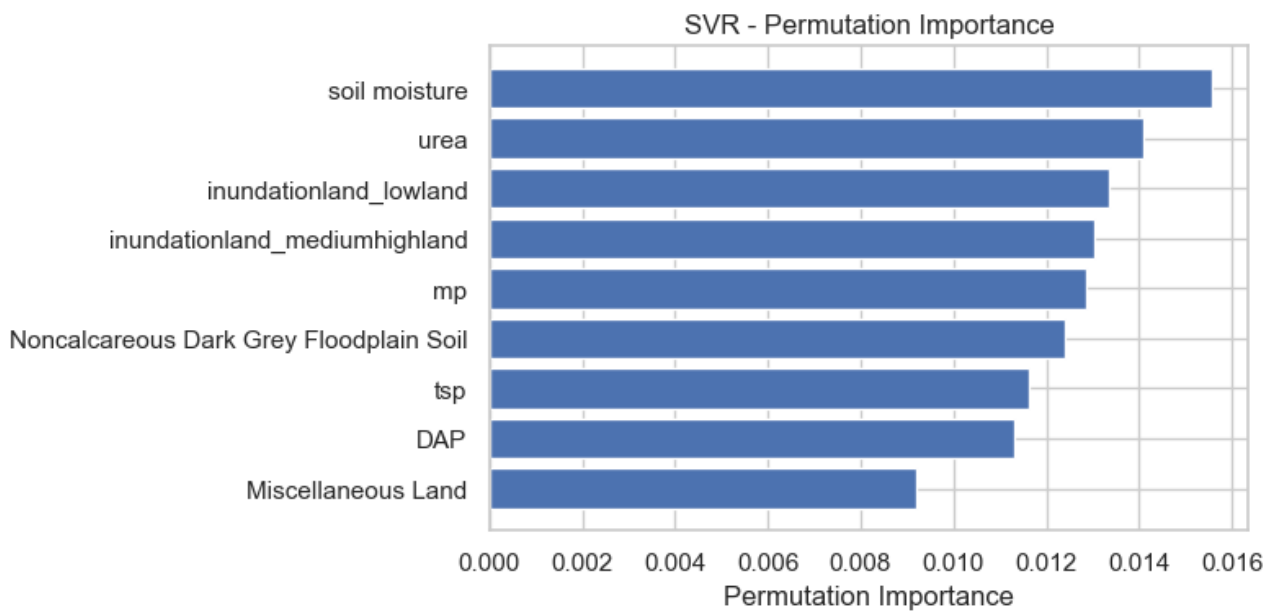


Figure 48: Boro - SVR:Permutation Importance

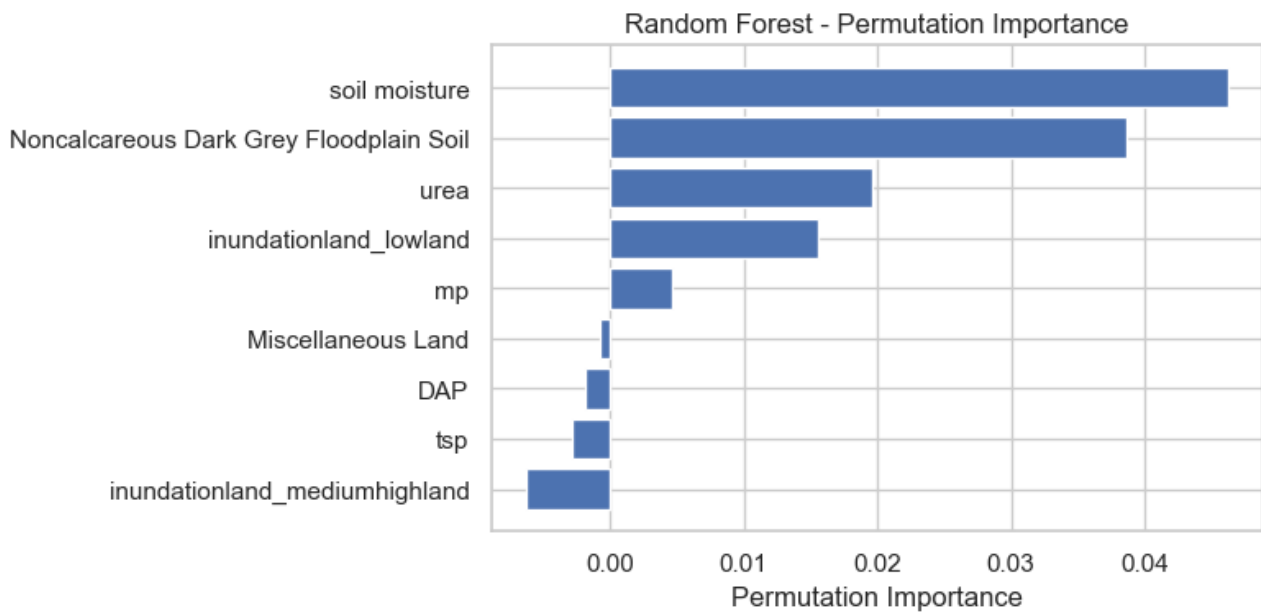


Figure 49: Boro - Random Forest:Permutation Importance

5.1.5 Best Models

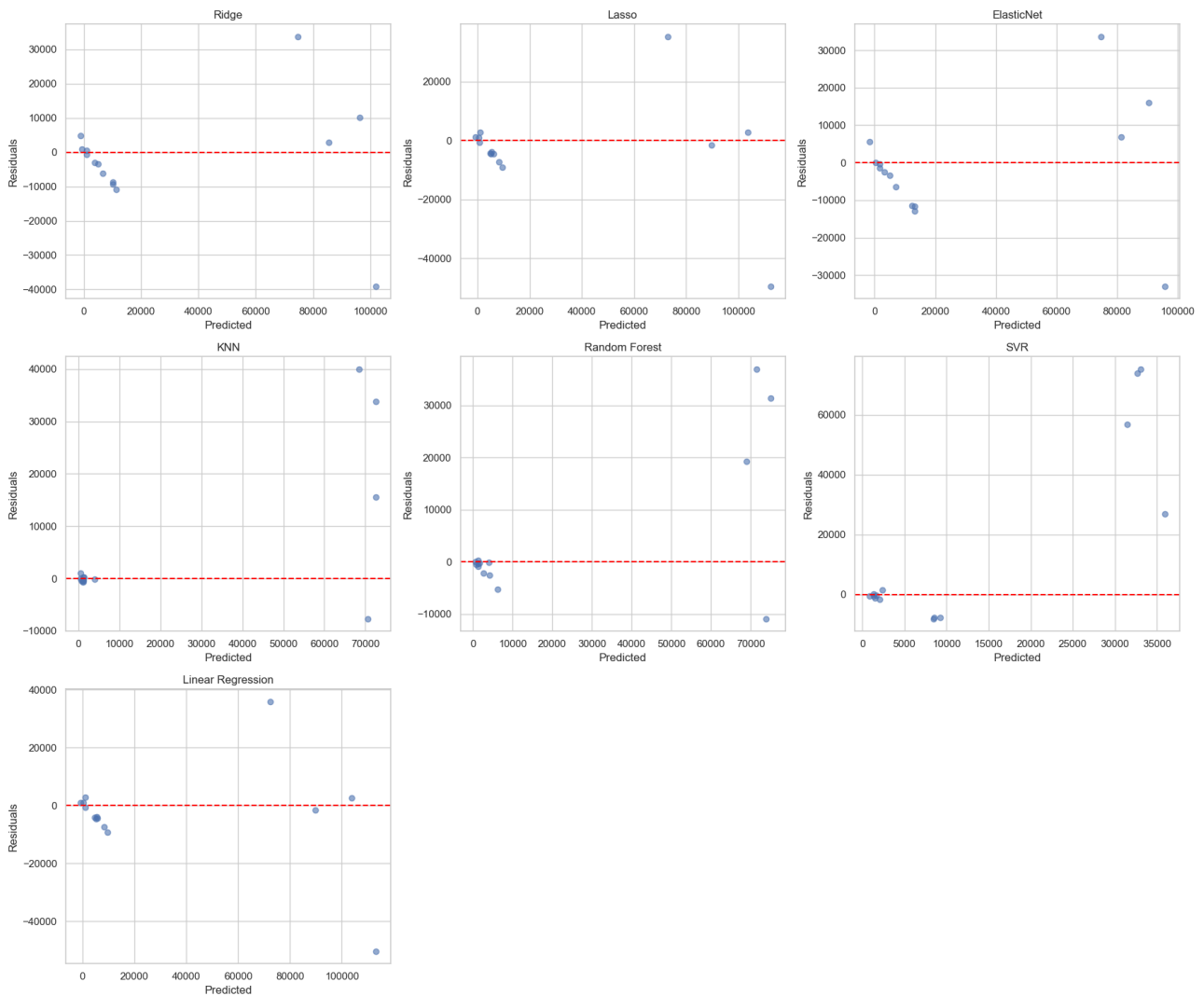


Figure 50: Aus - Best Models

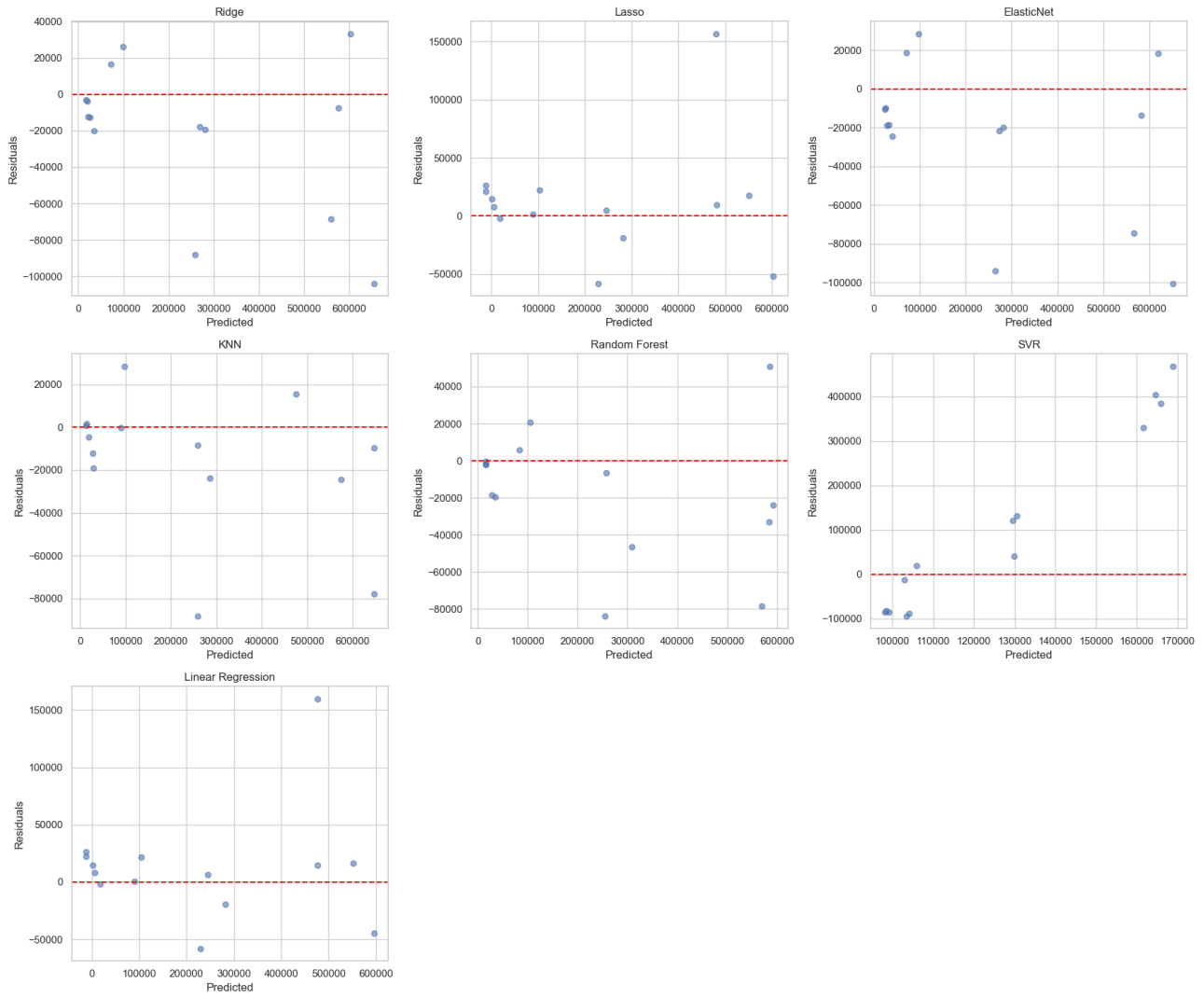


Figure 51: Aman - Best Models

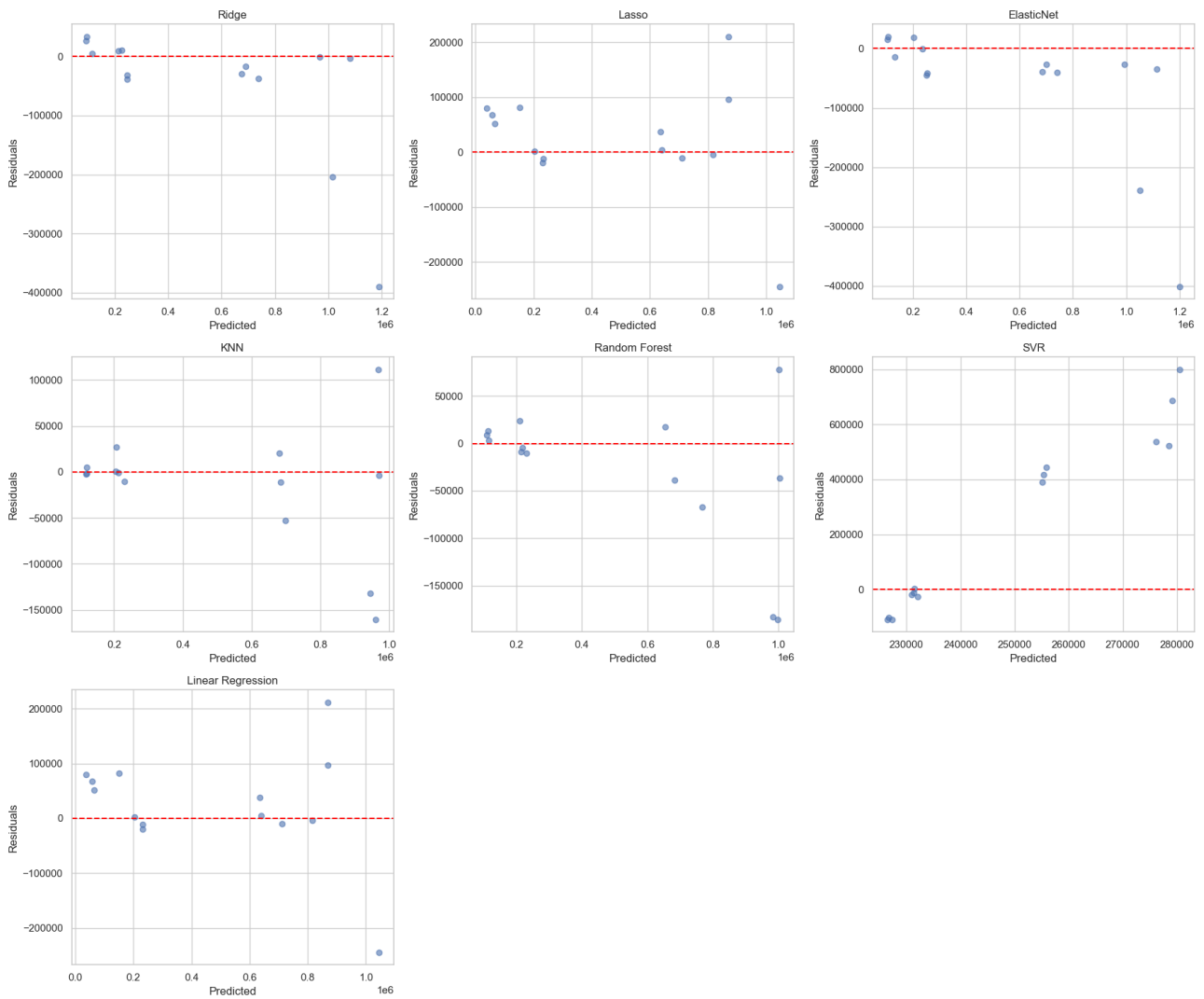


Figure 52: Boro - Best Models

5.1.6 Feature Importance

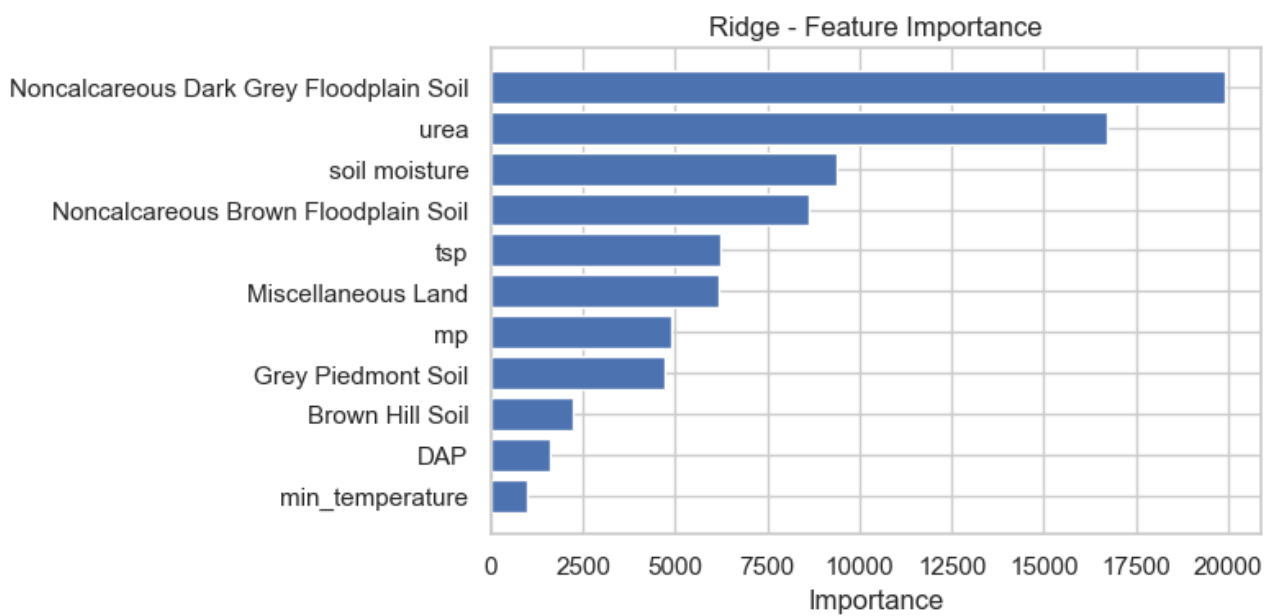


Figure 53: Aus - Ridge: Feature Importance

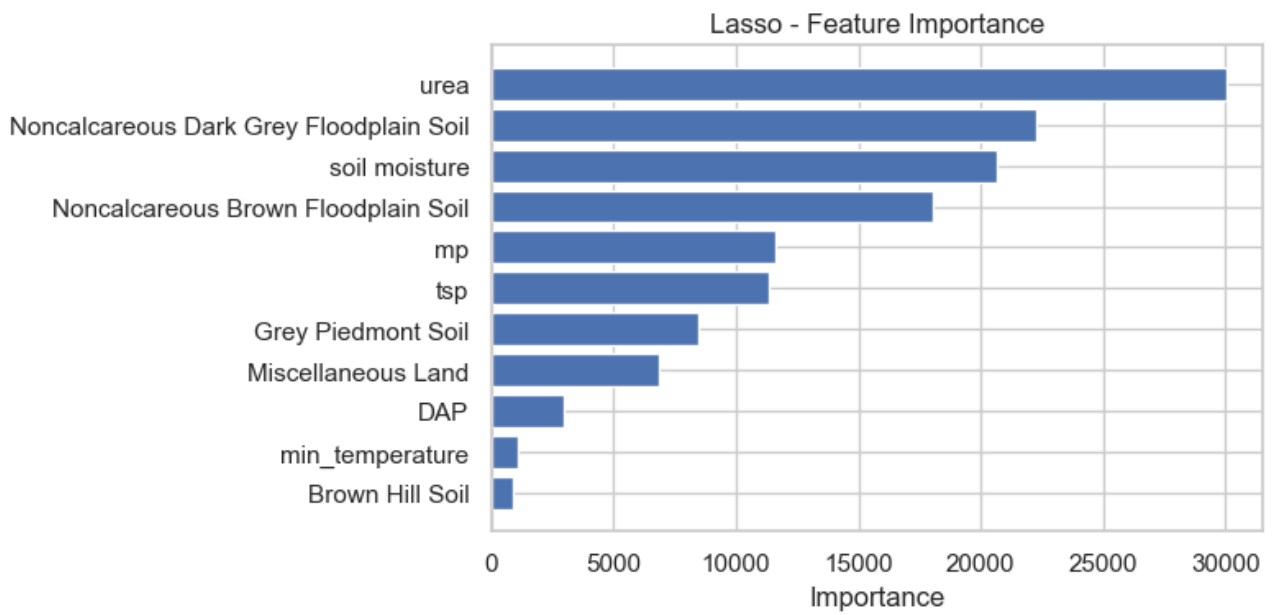


Figure 54: Aus - Lasso:Feature Importance

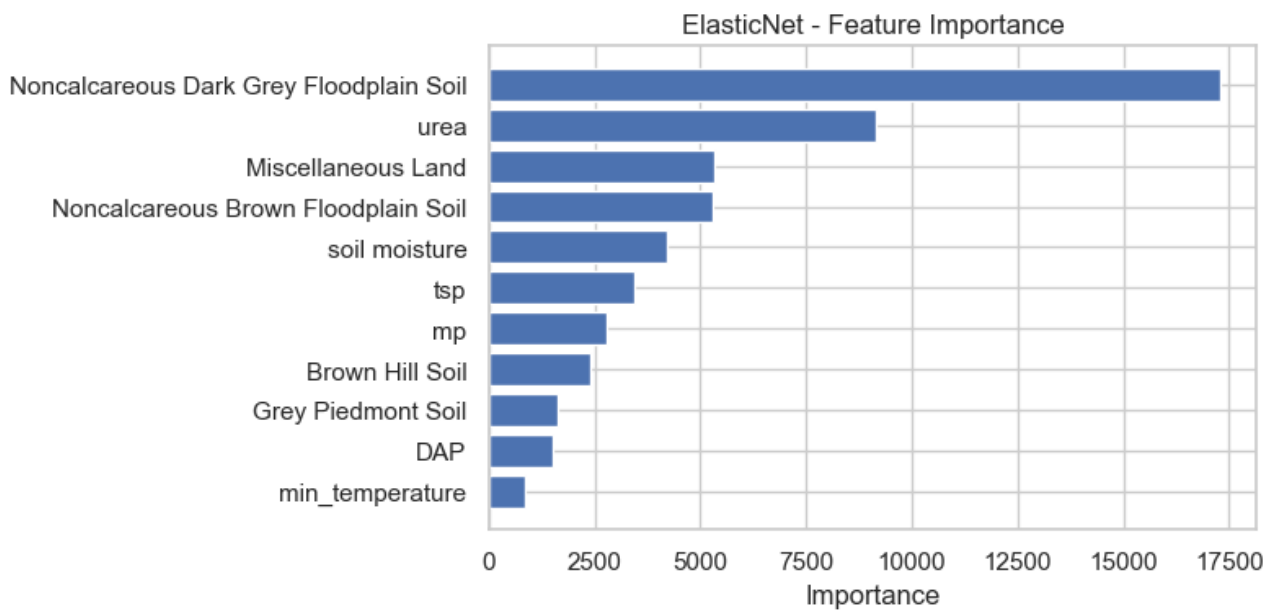


Figure 55: Aus - Elastic Net:Feature Importance

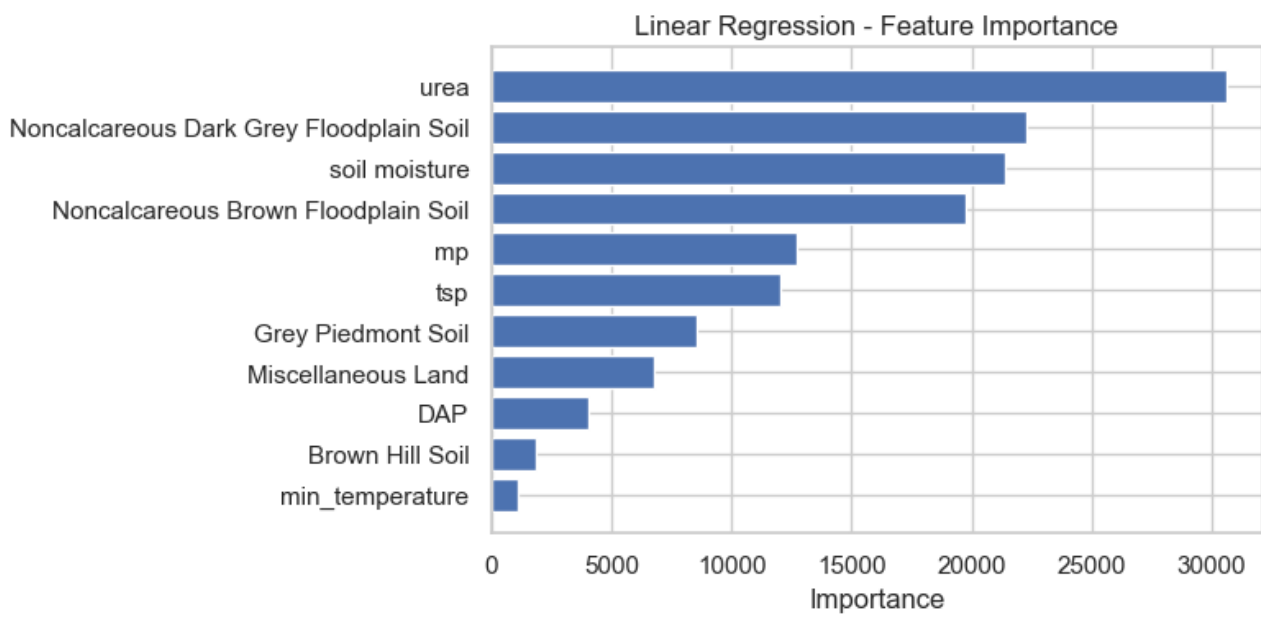


Figure 56: Aus - Linear Regression:Feature Importance

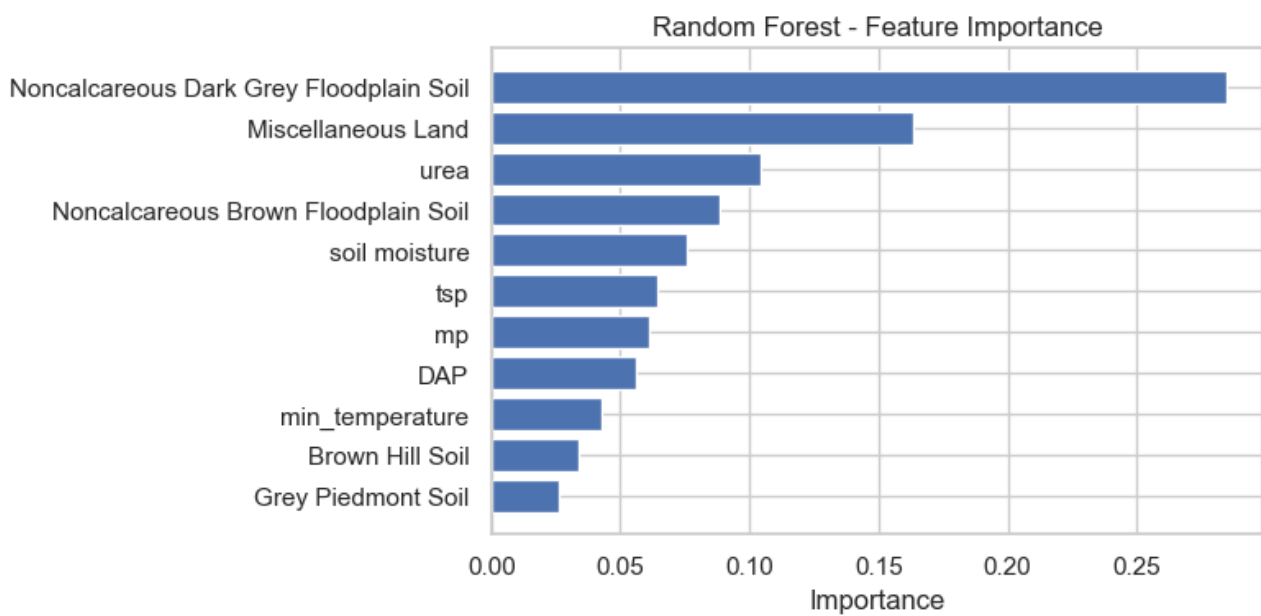


Figure 57: Aus - Random Forest:Feature Importance

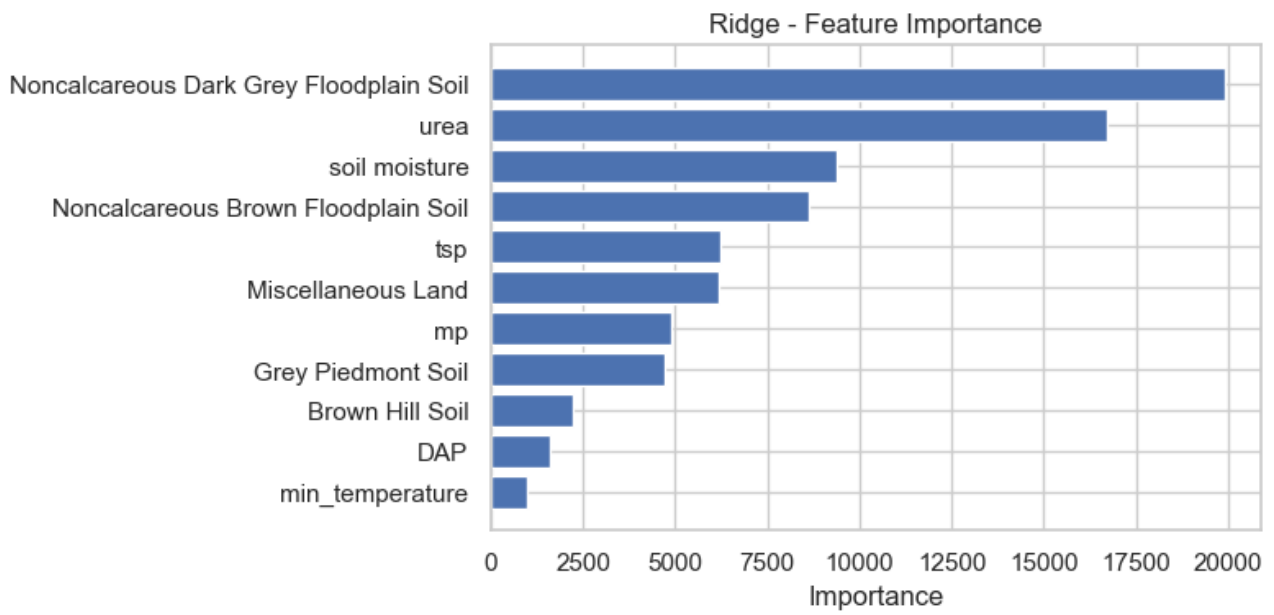


Figure 58: Aman - Ridge: Feature Importance

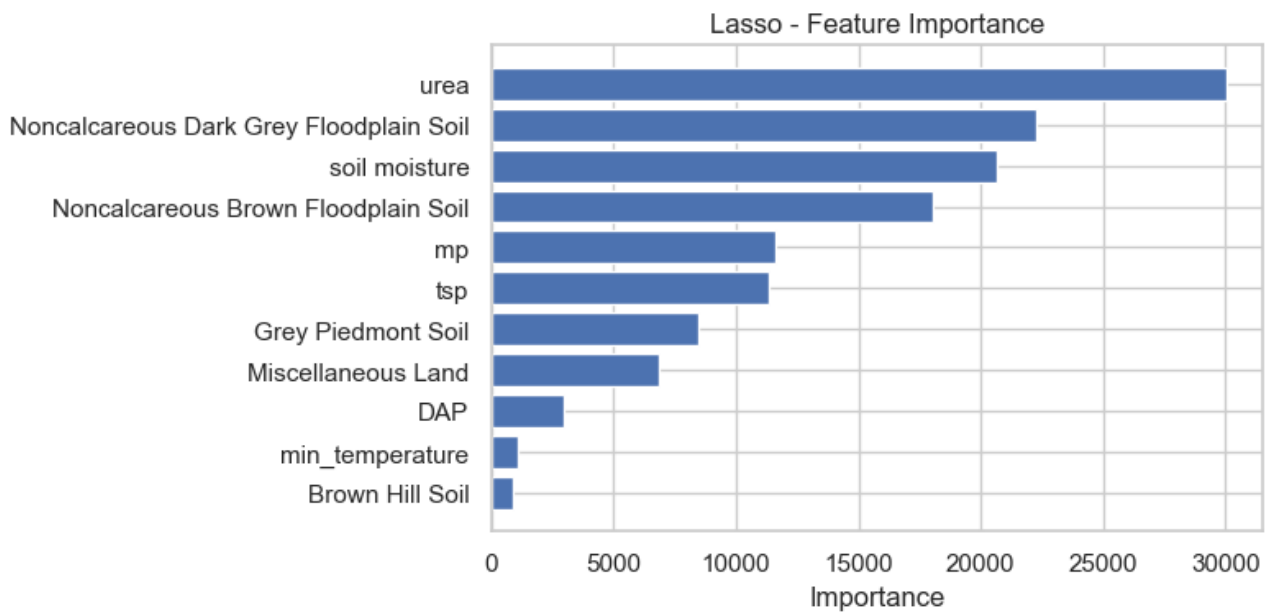


Figure 59: Aman - Lasso: Feature Importance

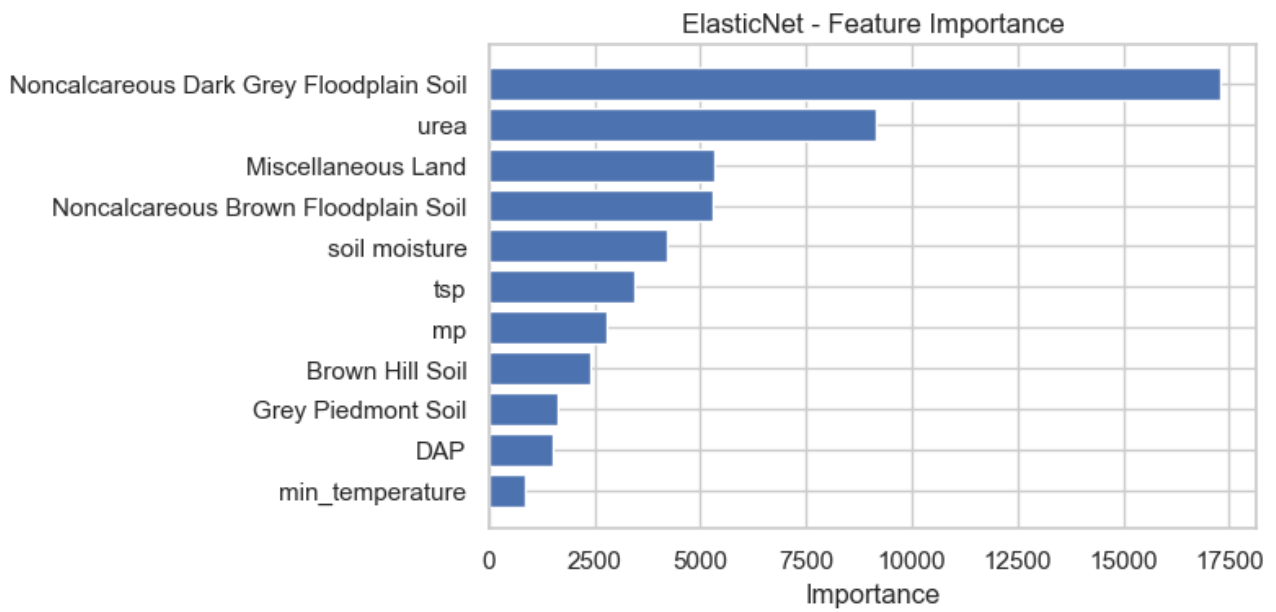


Figure 60: Aman - Elastic Net:Feature Importance

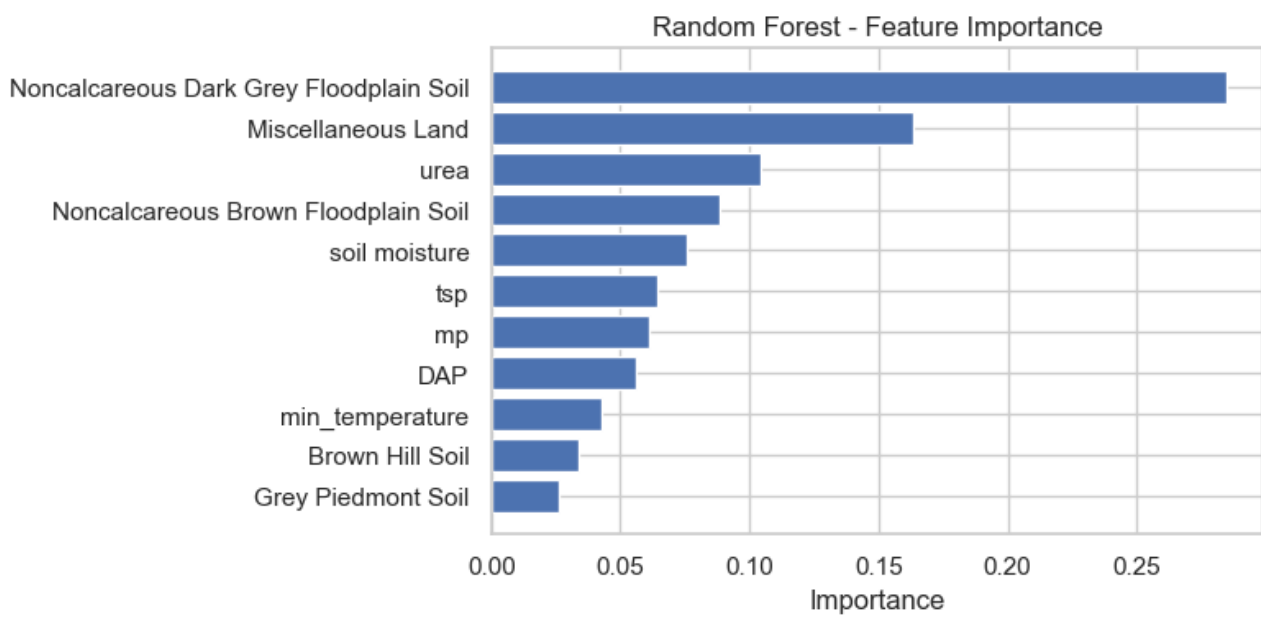


Figure 61: Aman - Random Forest: Feature Importance

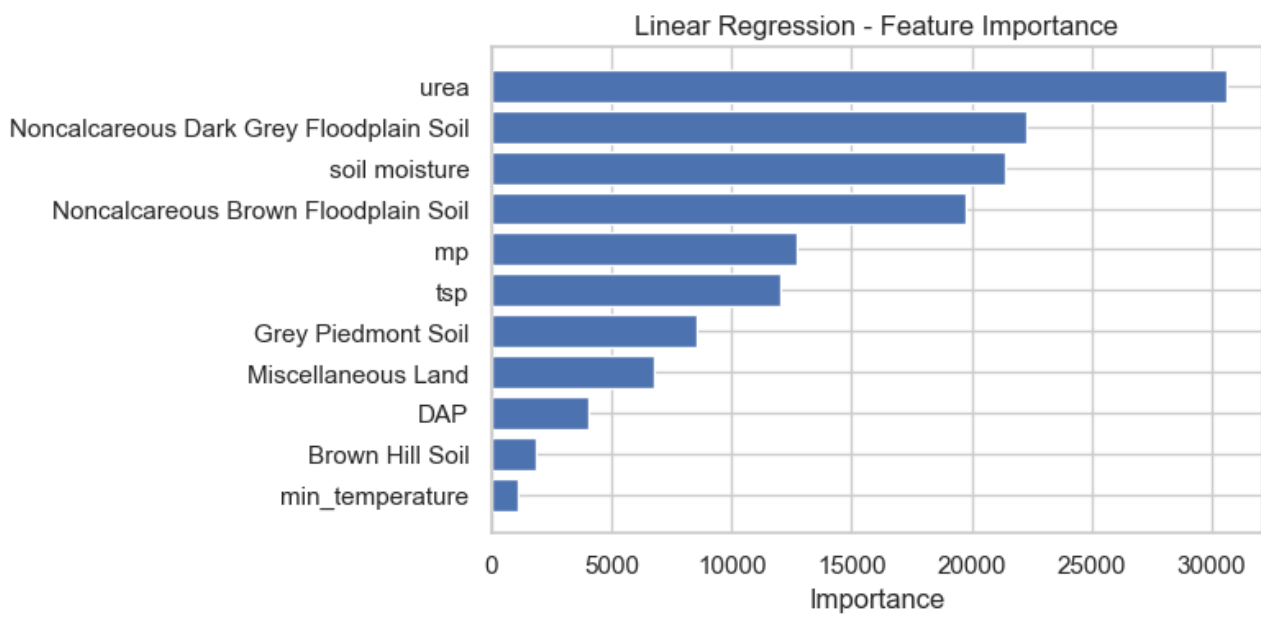


Figure 62: Aman - Linear Regression: Feature Importance

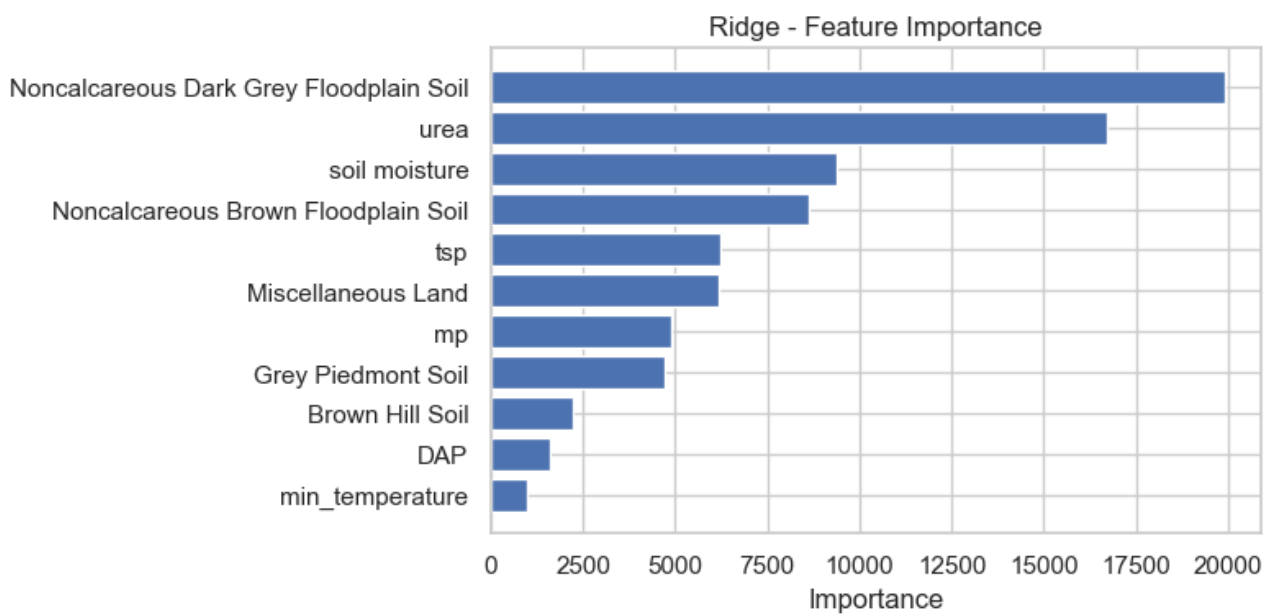


Figure 63: Boro - Ridge: Feature Importance

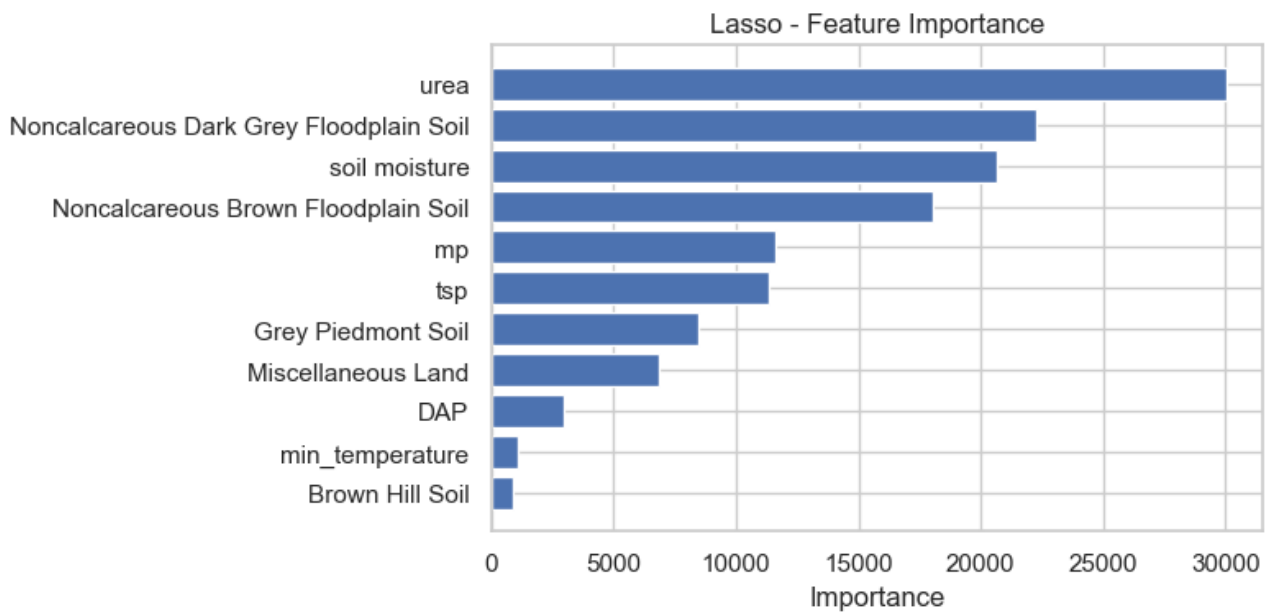


Figure 64: Boro - Lasso: Feature Importance

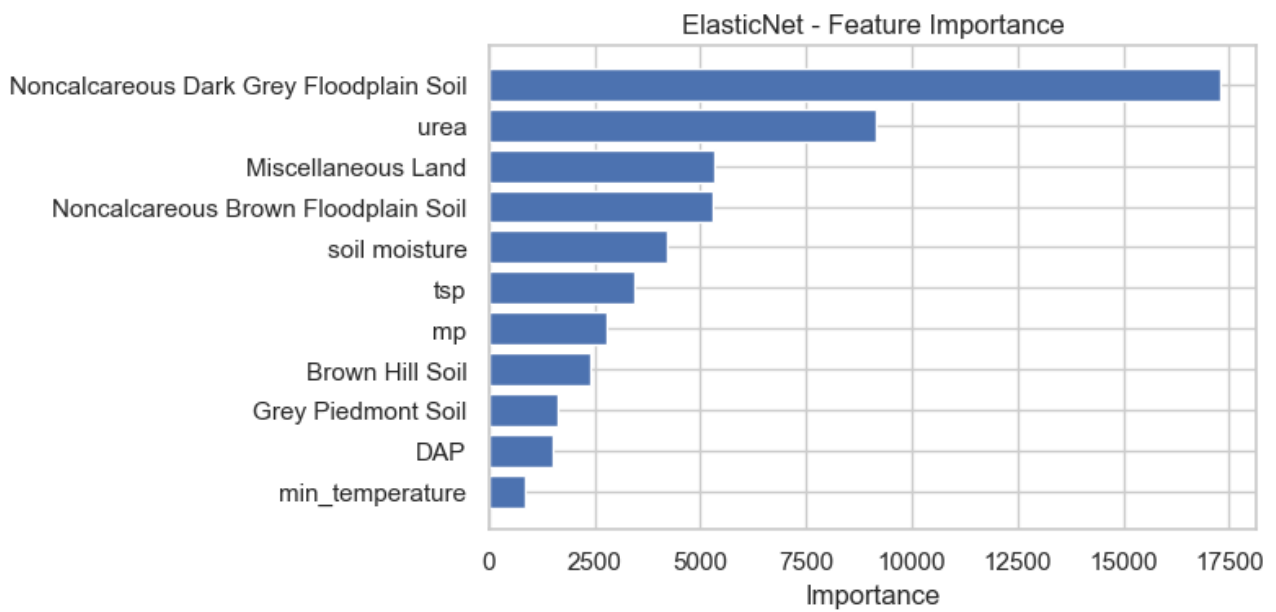


Figure 65: Boro - Elastic Net: Feature Importance

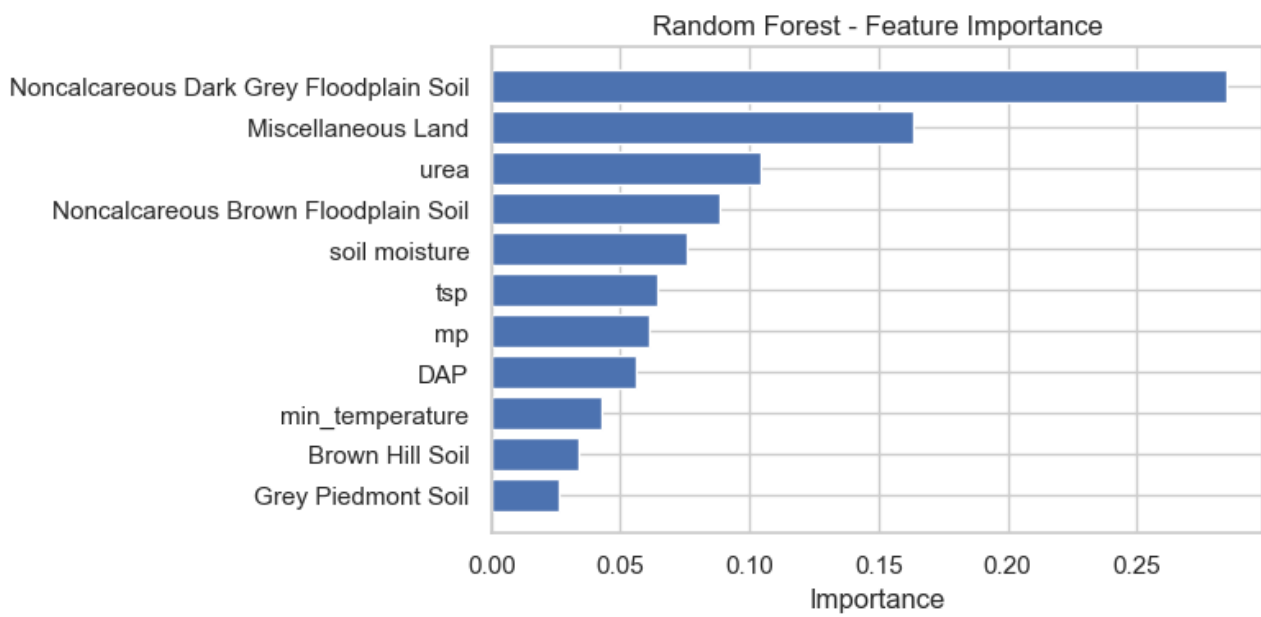


Figure 66: Boro - Random Forest: Feature Importance

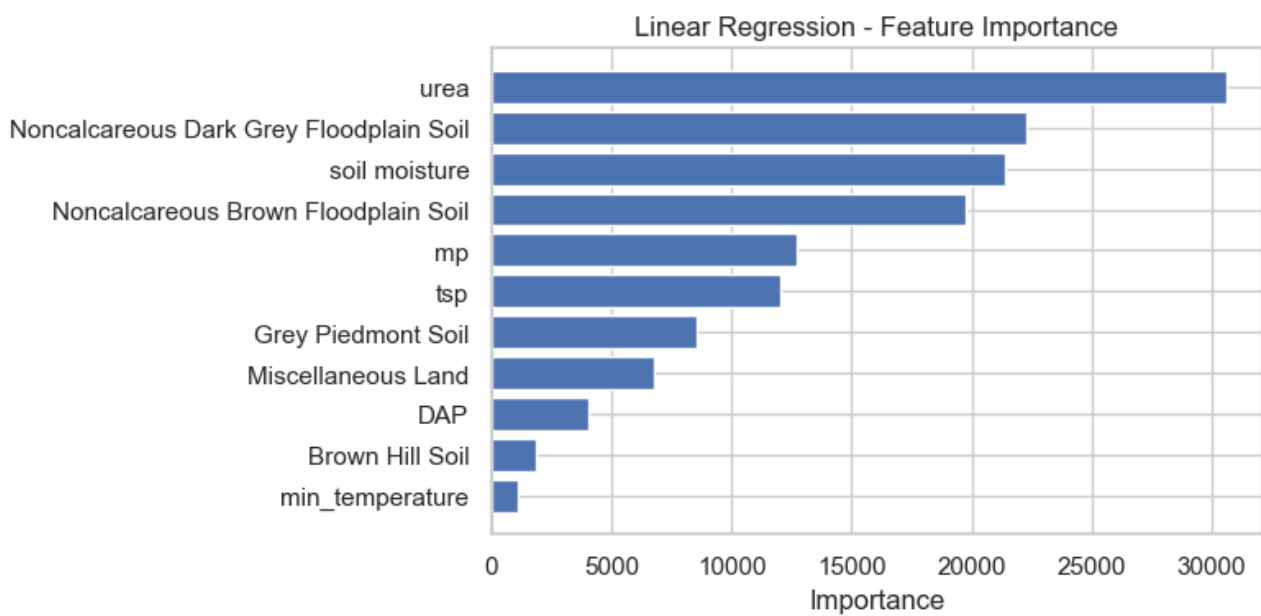


Figure 67: Boro - Linear Regression:Feature Importance

5.2 Tables

5.2.1 Model Performance Metrics

Table 1: Model Performance Metrics Across Datasets

Model	Train MSE	Test MSE	Train MAE	Test MAE	Train R ²	Test R ²
Dataset: aus						
Ridge	6.92e7	2.24e8	6610	9573	0.889	0.873
Lasso	5.92e7	2.80e8	5714	9155	0.905	0.841
ElasticNet	8.36e7	2.17e8	7289	10358	0.866	0.876
KNN	0	2.18e8	0	7227	1.000	0.876
Random Forest	9.76e6	2.06e8	1494	7933	0.984	0.883
SVR	4.11e8	1.09e9	11764	18712	0.340	0.381
Linear Regression	5.91e7	2.90e8	5696	9240	0.905	0.835
Dataset: aman						
Ridge	3.48e9	1.91e9	31675	30928	0.894	0.963
Lasso	2.94e9	2.38e9	31697	29521	0.910	0.954
ElasticNet	3.70e9	2.03e9	33717	33662	0.887	0.960
KNN	0	1.20e9	0	22487	1.000	0.977
Random Forest	6.71e8	1.49e9	13439	27997	0.980	0.971
SVR	3.10e10	5.08e10	112101	167667	0.055	0.013
Linear Regression	2.94e9	2.41e9	31725	29595	0.910	0.953
Dataset: boro						
Ridge	2.04e10	1.43e10	78533	59743	0.845	0.873
Lasso	1.72e10	9.70e9	79770	65816	0.870	0.914
ElasticNet	2.19e10	1.63e10	81643	68653	0.834	0.856
KNN	0	4.27e9	0	38576	1.000	0.962
Random Forest	8.40e9	5.91e9	44178	48613	0.936	0.948
SVR	1.72e11	1.59e11	265254	298273	-0.305	-0.409
Linear Regression	1.72e10	9.72e9	79811	65896	0.870	0.914

Note: MSE = Mean Squared Error, MAE = Mean Absolute Error, R² = Coefficient of Determination. Metrics are shown for various models across three datasets: aus (Australia), aman, and boro.

Table 2: Model Performance Metrics

Model	Train MSE	Test MSE	Train MAE	Test MAE	Train R ²	Test R ²
KNN (aman)	0	1.20e9	0	22487	1.000	0.977
Random Forest (aus)	9.76e6	2.06e8	1494	7933	0.984	0.883
KNN (boro)	0	4.27e9	0	38576	1.000	0.962

Note: MSE = Mean Squared Error, MAE = Mean Absolute Error, R² = Coefficient of Determination. Values in parentheses indicate dataset origin.

5.3 Discussion

Machine Learning based comparison and analysis of crop yield and prices of Aus, Aman and Boro Rice is an undergraduate thesis of the authors and this paper is an extension of one of the factors contributing to crop yield. The authors have faces limitations such as the unavailability of Bangladesh specific seasonally cursted crop yield datasets for applied machine learning research and more data of higher quality will improve research result. In the future, the authors may explore the opportunity to involve electronic sensors to isolate yield factors and generate real world datasets for far superior machine learning models.

References

- [1] Md. Shah Alam, Md. Mahbubur Rahman, and Md. Anamul Hoque. “Bayesian Network Modeling of Soil Salinity Effects on Aman Rice Yield in Coastal Bangladesh”. In: *Science of the Total Environment* 807 (2022), p. 150750. DOI: 10.1016/j.scitotenv.2021.150750.
- [2] D. L. J. Alexander, A. Tropsha, and David A. Winkler. “Beware of R^2 : simple, unambiguous assessment of the prediction accuracy of QSAR and QSPR models”. In: *Journal of Chemical Information and Modeling* 55.7 (2015), pp. 1316–1322. DOI: 10.1021/acs.jcim.5b00206.
- [3] Centre for Policy Dialogue. *State of the Bangladesh Economy in FY2024 (Third Reading)*. Research Report. Dhaka, Bangladesh: Centre for Policy Dialogue (CPD), 2025. URL: https://cpd.org.bd/resources/2025/04/State-of-the-Bangladesh-Economy-in-FY2024_Third-Reading.pdf.
- [4] Wei Chen, Xiaogang Li, and Hong Zhang. “A Hybrid Deep Learning Model for Rice Yield Prediction Integrating Multi-temporal Remote Sensing and Soil Data”. In: *Remote Sensing of Environment* 280 (2022), p. 113204. DOI: 10.1016/j.rse.2022.113204.
- [5] Subhadeep Ghosh, Subhasish Mondal, and Debashis Chatterjee. “Soil Moisture Characteristics and Drought Resilience in Aus Rice Cultivation: A Machine Learning Approach”. In: *Agricultural Water Management* 276 (2023), p. 108067. DOI: 10.1016/j.agwat.2022.108067.
- [6] Tanhim Islam. *Agricultural Dataset Bangladesh (44 parameters)*. Kaggle. Dataset accessed on 2026-01-28. 2019. URL: <https://www.kaggle.com/datasets/tanhim/agricultural-dataset-bangladesh-44-parameters>.
- [7] M. K. Johnson, R. Patel, and L. Chen. “Evaluation Metrics for Machine Learning Models: A Contemporary Review”. In: *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. Contemporary review of evaluation metrics including regression and classification measures. ACM, 2022, pp. 789–801. DOI: 10.1145/3531146.3533156.
- [8] Sandeep Kumar, Sunil Yadav, and Anirban Ghosh. “A Generalized Machine Learning Framework for Multi-variety Rice Yield Prediction Using Soil Health Data”. In: *Expert Systems with Applications* 185 (2021), p. 115641. DOI: 10.1016/j.eswa.2021.115641.
- [9] Soo-Jin Lee, Minh Kim, and Jisung Park. “Transfer Learning for Multi-season Rice Yield Prediction with Limited Data”. In: *IEEE Access* 12 (2024), pp. 12345–12356. DOI: 10.1109/ACCESS.2024.3356789.
- [10] A. C. Muller and S. Guido. *Practical Machine Learning for Data Scientists*. 2nd. Practical implementation guide for evaluation metrics in Python. O’Reilly Media, 2023. Chap. 5, pp. 287–332. ISBN: 978-1-098-11365-7.
- [11] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011). Main library for machine learning tools., pp. 2825–2830. URL: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [12] Md. Mizanur Rahman, Tofazzal Islam, and Mohammad Hossain. “Predicting Boro Rice Yield in Bangladesh Using Machine Learning and Soil Property Data”. In: *Agricultural Systems* 205 (2023), p. 103583. DOI: 10.1016/j.agsy.2022.103583.
- [13] M. A. Razzaque, J. Rahman, and S. H. Bidisha. *Creating employment opportunities for the youth: Urgent policy imperatives for Bangladesh*. Policy Brief. Research and Policy Integration for Development (RAPID), 2025. URL: https://www.rapidbd.org/wp-content/uploads/2025/04/Razzaque-et-al-Policy-Brief-March_2025_youth-employment.pdf.
- [14] J. D. Rodriguez, A. Perez, and J. A. Lozano. “A Comparative Analysis of Regression Metrics for Predictive Modeling”. In: *Journal of Machine Learning Research* 22.209 (2021). Comprehensive analysis of MAE, MSE, and other regression metrics with practical recommendations., pp. 1–32. URL: <https://jmlr.org/papers/v22/20-091.html>.
- [15] Jharna Rani Sarker, Asim Biswas, and Md. Jahangir Alam. “Soil Spectroscopy and Machine Learning for Aman Rice Yield Prediction in Southeast Asia”. In: *Geoderma* 406 (2022), p. 115495. DOI: 10.1016/j.geoderma.2021.115495.
- [16] scikit-learn developers. *Metrics and scoring: quantifying the quality of predictions*. Official documentation for MAE, MSE, and F-beta score implementations in scikit-learn 1.4+. scikit-learn. 2024. URL: https://scikit-learn.org/stable/modules/model_evaluation.html (visited on 10/27/2024).
- [17] scikit-learn developers. *sklearn.impute: Imputation of missing values*. Module for data imputation. scikit-learn. 2024. URL: <https://scikit-learn.org/stable/modules/classes.html#module-sklearn.impute>.

- [18] scikit-learn developers. *sklearn.inspection: Inspection utilities*. Module for permutation importance. scikit-learn. 2024. URL: <https://scikit-learn.org/stable/modules/classes.html#module-sklearn.inspection>.
- [19] scikit-learn developers. *sklearn.preprocessing: Preprocessing and Normalization*. Module containing scalers and normalizers. scikit-learn. 2024. URL: <https://scikit-learn.org/stable/modules/classes.html#module-sklearn.preprocessing>.
- [20] Rajesh Sharma, Neha Patel, and Amit Singh. “Comparative Analysis of Machine Learning Algorithms for Rice Yield Prediction Using Multi-source Data”. In: *Computers and Electronics in Agriculture* 204 (2023), p. 107540. DOI: 10.1016/j.compag.2022.107540.
- [21] The pandas development team. *pandas-dev/pandas: Pandas*. Library for data manipulation and analysis. 2024. DOI: 10.5281/zenodo.10957244. URL: <https://doi.org/10.5281/zenodo.10957244>.